Zero Touch Deployment

A Project Report submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering
Specialization in
Cloud Computing and Virtualization

By

Aakash Chaudhary (2215100001)
Aditi Saraswat (2215100002)
Aditya Upadhyay (2215100003)
Kanhiya Sharma (2215100009)
Mansi Agrawal (2215100013)

Group No.01
Under the Guidance of
Mr. Garvit Dohere

Department of Computer Engineering & Applications

Institute of Engineering & Technology

GLA UNIVERSITY
MATHURA
Recognised by UGC Under Section 2(f)
Accredited with A+ Grade by NAAC
3.46 Score
12-B Status from UGC

GLA University

Mathura- 281406, INDIA

May, 2026

Department of Computer Engineering and Applications
GLA University, 17 km Stone, NH#2, Mathura-Delhi Road,
P.O. Chaumuhan, Mathura-281406 (U.P.)

Declaration

I hereby declare that the work which is being presented in the B.Tech. Project **"Zero Touch Deployment"**, in partial fulfillment of the requirements for the award of the ***Bachelor of Technology*** in Computer Science and Engineering Specialization in Cloud Computing and Virtualization submitted to the Department of Computer Engineering and Applications of GLA University, Mathura, is an authentic record of my own work carried under the supervision of **Mr. Garvit Dohere-Technical Trainer**

The contents of this project report, in full or in parts, have not been submitted to any other Institute or University for the award of any degree.

Sign _____

Name of Student: Aakash Chaudhary
University Roll No.: 2215100001

Sign _____

Name of Student: Aditi Saraswat
University Roll No.:2215100002

Sign _____

Name of Student: Aditya Upadhyay
University Roll No.:2215100003

Sign _____

Name of Student: Kanhiya Sharma
University Roll No.: 2215100009

Sign _____

Name of Student: Mansi Agrawal
University Roll No.:2215100013

Certificate

This is to certify that the above statements made by the candidate are correct to the best of my/our knowledge and belief.

_____

Supervisor
Garvit Dohere
Technical Trainer
Dept. of Computer Engg, & App.

_____

Project Co-ordinator
(Dr. Mayank Srivastava)
Associate Professor
Dept. of Computer Engg, & App.

_____

Program Co-ordinator
(Dr. Nikhil Govil)
Associate Professor
Dept. of Computer Engg, & App.

Date:

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who helped and supported us in completing this project successfully.
First of all, we would like to thank our faculty members and mentors for their valuable guidance, support, and encouragement throughout the project. Their suggestions and feedback helped us in understanding the concepts clearly and improving our work at every stage.
We are also thankful to our institution for providing us with the necessary resources and environment to carry out this project. The learning experience gained during this work has been very helpful in enhancing our practical knowledge.
Finally, we would like to thank all our team members for their cooperation, coordination, and continuous efforts in completing this project. Without teamwork and mutual support, this project would not have been possible.

_____
Sign _____
Name of Candidate: Aakash Chaudhary
University Roll No: 2215100001

Sign _____
Name of Candidate: Aditi Saraswat
University Roll No: 2215100002

Sign _____
Name of Candidate: Aditya Upadhyay
University Roll No: 2215100003

Sign _____
Name of Candidate: Kanhiya Sharma
University Roll No: 2215100009

Sign _____
Name of Candidate: Mansi Agrawal
University Roll No: 2215100013

ABSTRACT

This project focuses on the implementation of a complete DevOps pipeline using Jenkins to automate the software development lifecycle. The main objective of the project is to design and develop a Continuous Integration and Continuous Deployment (CI/CD) system that reduces manual effort and improves efficiency in application development and deployment. In traditional methods, software development involves multiple manual steps which can lead to delays and errors. This project aims to overcome those challenges by introducing automation at every stage.

In this project, a sample web application is used and integrated with various DevOps tools to create a smooth and continuous workflow. Git is used for version control, which helps in managing and tracking code changes. Whenever the developer updates the code and pushes it to the repository, Jenkins automatically triggers the pipeline. Jenkins plays a key role in automating the entire process, including fetching the latest code, building the application, and running tests.

During the pipeline execution, the system ensures that the application is properly built and tested before moving to the next stage. This helps in identifying errors at an early stage and improves the quality of the application. After successful testing, Docker is used for containerization. It packages the application along with all its dependencies, ensuring that it runs consistently across different environments without any issues.

The deployment process is also automated, where the application is delivered to the target environment without manual intervention. This makes the process faster and more reliable. The use of automation not only saves time but also reduces the chances of human errors. It ensures that the same process is followed every time, which improves consistency in software delivery.

This project also highlights the importance of DevOps practices in modern software development. Continuous Integration ensures that code changes are regularly tested and integrated, while Continuous Deployment ensures that updates are delivered quickly and efficiently. These practices help in improving collaboration between development and operations teams and make the overall process more efficient.

Overall, this project provides a clear understanding of how a real-world DevOps pipeline works. It demonstrates how different tools like Git, Jenkins, and Docker can be integrated to automate the software development lifecycle. The project not only improves technical knowledge but also gives practical experience of implementing automation in software development, which is widely used in the industry today.

CONTENTS

CHAPTER 1: Introduction

1.1 Overview and Motivation

In today's fast-paced software development environment, delivering high-quality applications quickly and efficiently has become a major requirement for organizations. Traditional development methods, where code is written, tested, and deployed manually, are often slow, error-prone, and difficult to manage. To overcome these challenges, DevOps practices and automation tools have become essential in modern software engineering.

This project focuses on implementing a DevOps pipeline using Jenkins, which is one of the most widely used open-source automation tools. Jenkins helps automate different stages of the software development lifecycle, including building, testing, and deployment of applications. It acts as a central platform where developers can integrate their code, run automated tests, and deploy applications continuously without manual intervention.

The project demonstrates how a complete CI/CD (Continuous Integration and Continuous Deployment) pipeline can be created using Jenkins. It includes integration with tools like Git for version control, build tools for compiling code, and deployment mechanisms for delivering the application to servers. The pipeline ensures that every code change is automatically tested and deployed, improving efficiency and reducing human errors.

Overall, this project provides a practical understanding of how DevOps works in real-world scenarios and how automation tools like Jenkins can streamline the entire development process.

The main motivation behind this project comes from the limitations of traditional software development processes. In manual workflows, developers often face issues such as delayed testing, integration conflicts, and deployment failures. These problems not only slow down the development cycle but also affect the quality of the final product.

With the increasing demand for faster software delivery, organizations are shifting towards DevOps practices where automation plays a key role. Jenkins is highly valuable in this context because it automates repetitive tasks like building, testing, and deploying applications, saving time and reducing errors.

Another important motivation is the need for continuous feedback. In a CI/CD pipeline, every change made by developers is immediately tested, allowing early detection of bugs and issues. This improves software reliability and helps teams deliver better products more frequently.

Additionally, learning and implementing Jenkins-based projects is important from a career perspective. DevOps and CI/CD tools are in high demand in the industry, and having hands-on experience with such tools enhances technical skills and job opportunities.

In summary, this project is motivated by the need to:

- Automate the software development process
- Reduce manual effort and errors
- Improve software quality through continuous testing
- Gain practical knowledge of DevOps tools and practices

1.2 Objective

The main objective of this project is to build a complete DevOps pipeline that automates the entire software development process, starting from writing code to deploying it on a live environment. The project focuses on creating a smooth and continuous workflow where all stages of development are connected and executed automatically without manual intervention.

In this project, the aim is to integrate different tools and technologies in a proper sequence so that once the developer pushes the code to the repository, the entire process starts automatically. This includes fetching the latest code, building the application, running tests, checking code quality, performing security scanning, creating Docker images, and finally deploying the application on a Kubernetes cluster. Each stage is designed to work in coordination with the next stage, ensuring a proper flow of execution.

Another important objective of this project is to understand how real-world systems are designed and managed. In traditional development processes, tasks like building, testing, and deployment are done manually, which can lead to delays and errors. This project replaces those manual steps with automation using tools like Jenkins for CI/CD, Docker for containerization, Kubernetes for deployment, and ArgoCD for continuous delivery. These tools are widely used in the industry, and their integration in this project helps in understanding their practical usage.

The project also aims to improve development efficiency by reducing manual work and minimizing the chances of human errors. By automating repetitive tasks, the system ensures that the same process is followed every time, which increases reliability and consistency. It also helps in saving time, as the pipeline runs automatically whenever there is a change in the code.

In addition to this, monitoring is an important part of the system. In this project, monitoring tools like Prometheus and Grafana are used to track the performance and health of the application. These tools are deployed using the Helm package manager, which simplifies the installation and management of applications in the Kubernetes cluster. With the help of these tools, system metrics can be visualized and analyzed easily, helping in identifying issues and maintaining system stability.

Overall, the objective of this project is not only to build a working DevOps pipeline but also to gain practical knowledge of how modern DevOps practices are implemented in real-world scenarios. It provides a clear understanding of how different tools work together to automate the software development lifecycle and make the process faster, more efficient, and reliable..

1.3 Summary of Similar Applications

Before building this project, it is important to understand that many similar systems already exist in the industry which follow the same idea of automating the software development process. These systems are mainly based on CI/CD (Continuous Integration and Continuous Deployment), where the goal is to reduce manual work and make development faster and more reliable.

One of the most commonly used tools is Jenkins, which is an open-source automation server. It allows developers to create pipelines where code is automatically built, tested, and deployed. It is highly flexible and supports a large number of plugins, which makes it suitable for complex and customized projects. Even today, it is widely used in many companies because it can be integrated with almost any tool or technology.

Another similar application is GitHub Actions, which provides CI/CD features directly inside GitHub. It works in a simple way—whenever code is pushed to the repository, the workflow starts automatically and performs tasks like build and testing. It is easy to set up and is mostly used for small to medium-level projects where quick setup is needed.

GitLab CI/CD is also widely used and provides an all-in-one platform where code management and pipeline automation are available together. It allows developers to define different stages like build, test, and deployment in a structured manner. It also includes additional features like security scanning and monitoring, which makes it useful for larger and more secure environments.

Apart from these, there are other tools like CircleCI, Travis CI, and Azure DevOps, which also perform similar functions. All these tools follow the same basic working process:
first, the developer writes code → then the system automatically builds it → runs tests → and finally deploys it to a server. This automation helps in detecting errors early and improves the overall quality of the software.

Even though different tools are available, the main idea behind all of them is the same—to automate the software development lifecycle and reduce manual effort. This project is based on the same concept and tries to implement a similar system in a simple and understandable way, so that the complete flow from code to deployment can be clearly understood.

CHAPTER 2: System Requirement Analysis

2.1 Technical Feasibility

This project is technically feasible because it uses tools and technologies that are easily available and commonly used in the industry. The complete system is based on DevOps practices where different tools are connected together to automate the workflow from development to deployment. Since all the tools used in this project are well-documented and widely supported, it becomes easier to understand and implement the system step by step.

Tools like Jenkins, Docker, Kubernetes, and Git are open-source or freely available, which makes them easy to access and use. These tools can be installed on a normal system or configured using cloud platforms without requiring any special setup. Even though the overall system may look complex at first, each component has a specific role, and when implemented in sequence, the process becomes simple and manageable. Jenkins is used to automate the CI/CD pipeline, Git manages the source code, Docker is used for containerization, and Kubernetes handles deployment and orchestration.

The project mainly focuses on automation, where once the developer pushes the code to the repository, the system automatically takes care of building, testing, and deployment. This reduces the need for manual intervention and minimizes the chances of errors. The automated pipeline ensures that all steps are performed in a fixed order, which improves consistency and reliability in the development process.

Another important aspect of technical feasibility is the use of containers and orchestration. Docker helps in packaging the application along with all its dependencies, so it can run consistently in any environment. Kubernetes manages these containers efficiently by distributing them across multiple nodes, ensuring high availability and scalability. This makes the system flexible and capable of handling increasing workloads.

In addition to this, monitoring tools like Prometheus and Grafana are used to track system performance and health. These tools are deployed using Helm, which simplifies their installation and management in the Kubernetes environment. This shows that the system is not only deployable but also maintainable in the long run.

Overall, the project is technically feasible because it uses standard DevOps tools, does not require high-end infrastructure, and can be implemented using basic knowledge of these technologies. The step-by-step integration of tools and the use of automation make the system practical, efficient, and suitable for real-world applications.

2.2 System Requirements

To successfully implement this project, some basic system requirements are needed. These requirements include both hardware and software components. Since the project is based on DevOps tools and automation, the system should be capable of handling multiple tools running together smoothly. However, the requirements are not very high and can be fulfilled using a normal system with a proper setup.

Hardware Requirements:

- Laptop or Desktop System:

A standard laptop or desktop is sufficient to perform development, testing, and deployment tasks for this project.

- Minimum 8 GB RAM (recommended for better performance):

RAM is important because multiple tools like Jenkins, Docker, and Kubernetes may run at the same time. At least 8 GB RAM ensures smooth execution without lag.

- At least 30 GB Free Storage:

Storage is required to install tools, store code repositories, Docker images, and logs generated during pipeline execution.

- Stable Internet Connection:

Internet is necessary for downloading tools, accessing repositories, pulling Docker images, and managing cloud or remote services.

Software Requirements:

- Operating System (Linux preferred, Windows/macOS supported):

The project can run on any operating system, but Linux is preferred because most DevOps tools are easier to configure and manage in a Linux environment.

- Java (for running Jenkins):

Jenkins is a Java-based application, so Java is required to install and run Jenkins on the system.

- Jenkins (for CI/CD pipeline):

Jenkins is used to automate the process of building, testing, and deploying the application. It acts as the core of the pipeline.

- Git (for version control):

Git is used to manage the source code. It helps in tracking changes and allows developers to push and pull code easily.

- Docker (for containerization):

Docker is used to package the application along with its dependencies into containers, ensuring consistent execution in different environments.

- Kubernetes (for deployment and orchestration):

Kubernetes is used to deploy and manage Docker containers. It handles scaling, load balancing, and ensures high availability of the application.

- ArgoCD (for continuous delivery):

ArgoCD is used to automate the deployment process by syncing the application state from the Git repository to the Kubernetes cluster.

- Monitoring Tools (Prometheus and Grafana):

These tools are used to monitor system performance and health. Prometheus collects metrics, and Grafana provides dashboards for visualization.

- Web Browser:

A web browser is required to access dashboards like Jenkins, ArgoCD, and Grafana, and to manage the system through their user interfaces.

CHAPTER 3: SOFTWARE DESIGN

3.1 System Architecture

The system is designed as a complete DevOps pipeline where different tools are connected to automate the entire process from development to deployment.

At the starting point, the developer writes the application code using technologies like React, Node.js, or other frameworks. This code is stored in a Git repository, which acts as the central place for managing all changes.

Once the code is pushed, the CI/CD process starts. Jenkins plays an important role here as it manages the pipeline. It fetches the code from the repository and starts the build process. During this stage, different checks are performed such as code quality analysis and security scanning using tools like SonarQube and OWASP. This helps in identifying issues at an early stage.

After successful checks, the application is packaged into a Docker container. Docker helps in creating a consistent environment so that the application can run smoothly on any system. These containers are then stored and used for deployment.

For deployment, Kubernetes is used to manage and run the containers. It handles scaling, load balancing, and ensures that the application runs without interruption. The cluster consists of multiple nodes, which makes the system more reliable and efficient.

ArgoCD is used for continuous delivery. It automatically takes the updated configuration from the Git repository and deploys it into the Kubernetes cluster. This makes the deployment process fully automated.

Finally, monitoring tools like Prometheus and Grafana are used to track system performance. They help in checking the health of the application and provide useful insights through dashboards.

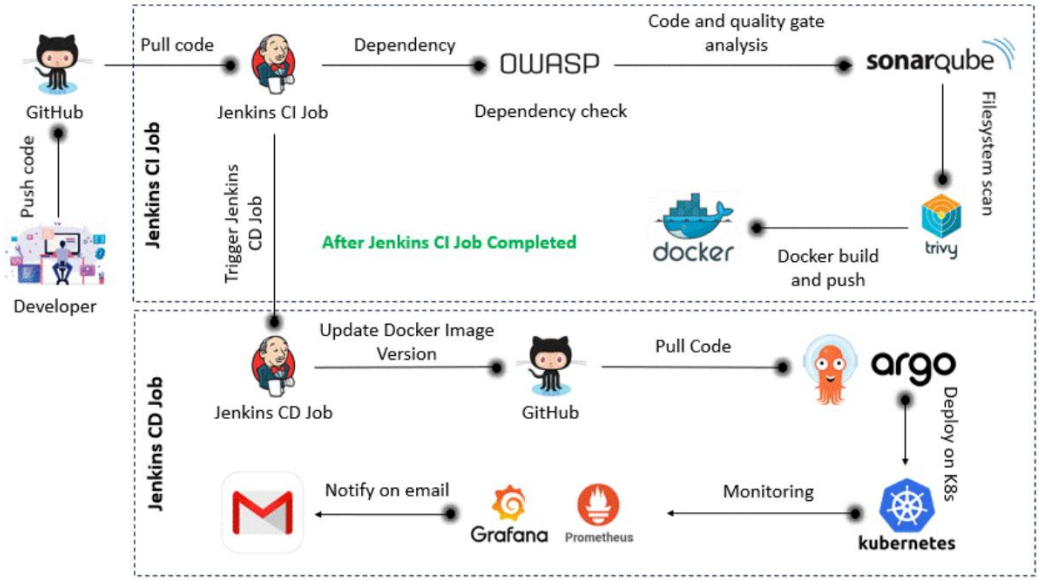
Overall, the architecture is designed in a way that each tool has a specific role, and all of them work together to create a smooth and automated workflow.

3.2 System Design Flow

The working of the system follows a step-by-step flow:

1. Developer writes and pushes code to Git
2. Jenkins pipeline gets triggered automatically
3. Code is built and tested
4. Security and quality checks are performed
5. Docker image is created
6. Application is deployed on Kubernetes
7. ArgoCD manages continuous delivery
8. Monitoring tools track performance

This flow ensures that the entire process is automated and there is minimal manual intervention.



3.3 Design Considerations

While designing this system, some important points are kept in mind:

- The system should be easy to understand and manage
- Each component should be independent but well-connected
- The pipeline should be automated to reduce manual work
- The system should be scalable to handle more load in future
- Proper monitoring should be included for better performance tracking

These considerations help in making the system efficient and reliable.

CHAPTER 4: IMPLEMENTATION AND USER INTERFACE

4.1 Implementation

The implementation of this project is done step by step by connecting different DevOps tools to create a complete automated pipeline.

First, the application code is written and stored in a Git repository. Git is used to manage the code and track all changes. Whenever the developer updates the code and pushes it to the repository, it acts as a trigger for the next steps.

After that, Jenkins is used to create the CI/CD pipeline. In Jenkins, a pipeline is configured which defines all the stages like build, test, and deployment. When the code is pushed, Jenkins automatically starts the pipeline and fetches the latest code from the repository.

In the next step, the build process takes place. The application is compiled and checked for errors. Along with this, code quality and security checks are also performed using tools like SonarQube and OWASP. This helps in identifying issues early before deployment.

Once everything is successful, a Docker image is created. Docker is used to package the application along with all its dependencies, so it can run in any environment without issues. This image is then used for deployment.

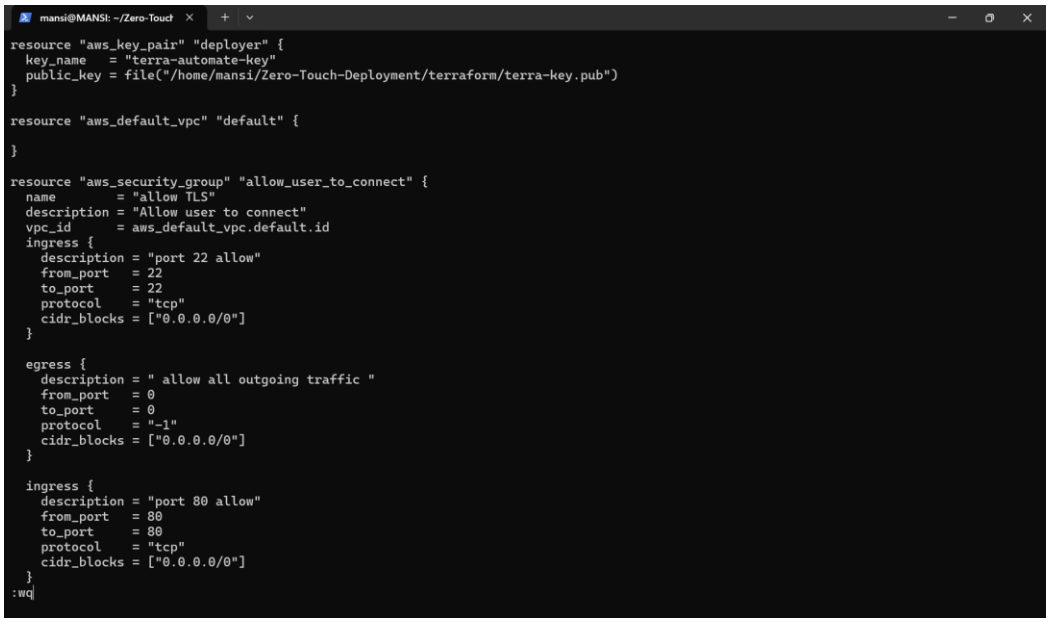
The deployment is handled using Kubernetes. It runs the Docker containers and manages them efficiently. It also helps in scaling the application and ensures that it remains available even if some part of the system fails.

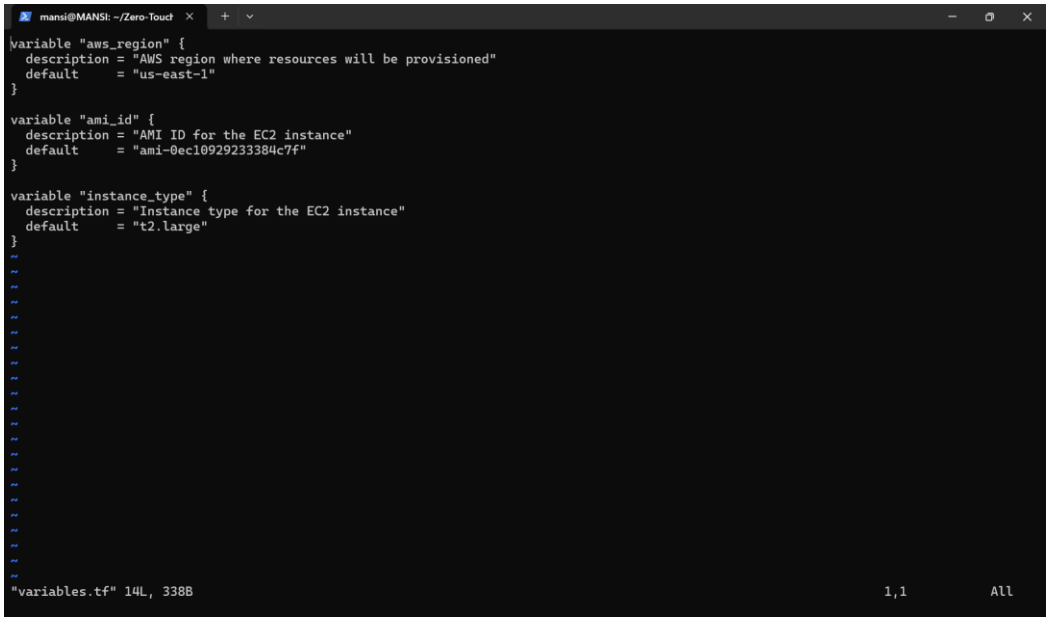
For continuous delivery, ArgoCD is used. It automatically updates the application in the Kubernetes cluster whenever there is a change in the configuration. This makes the deployment process smooth and fully automated.

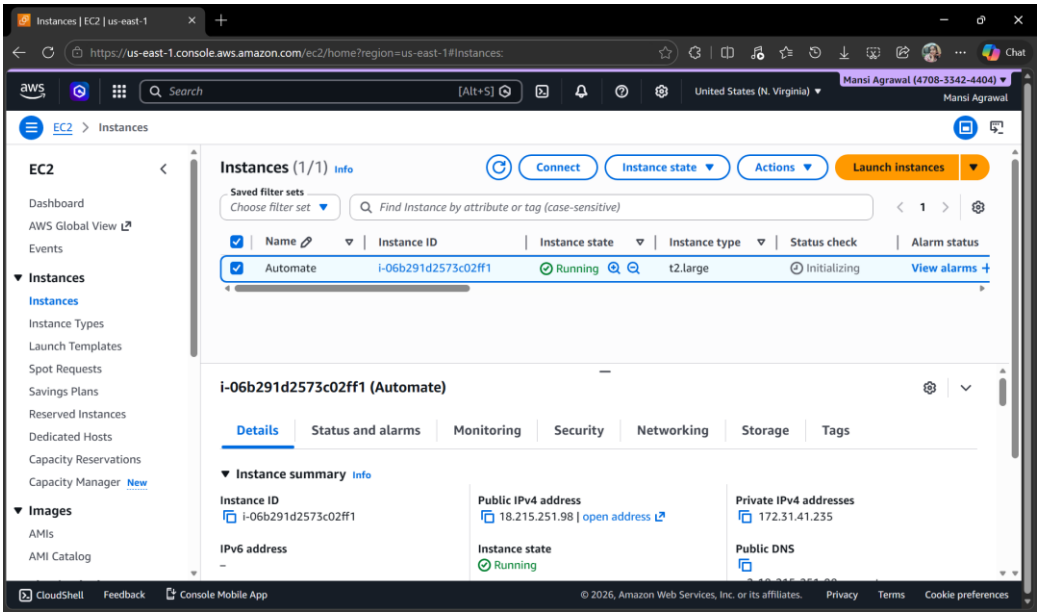
Finally, monitoring tools like Prometheus and Grafana are used to track the performance of the system. They provide information about system health, usage, and possible issues.

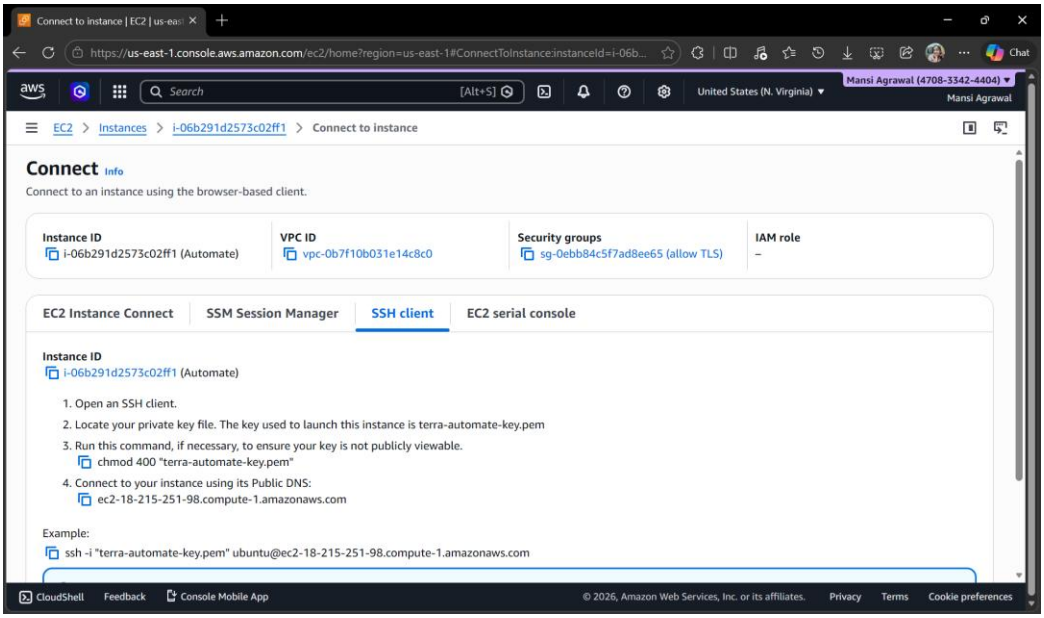
Pre-requisites to implement this project:

- Create 1 Master machine on AWS with 2CPU, 8GB of RAM (t2.large) and 29 GB of storage and install Docker on it.

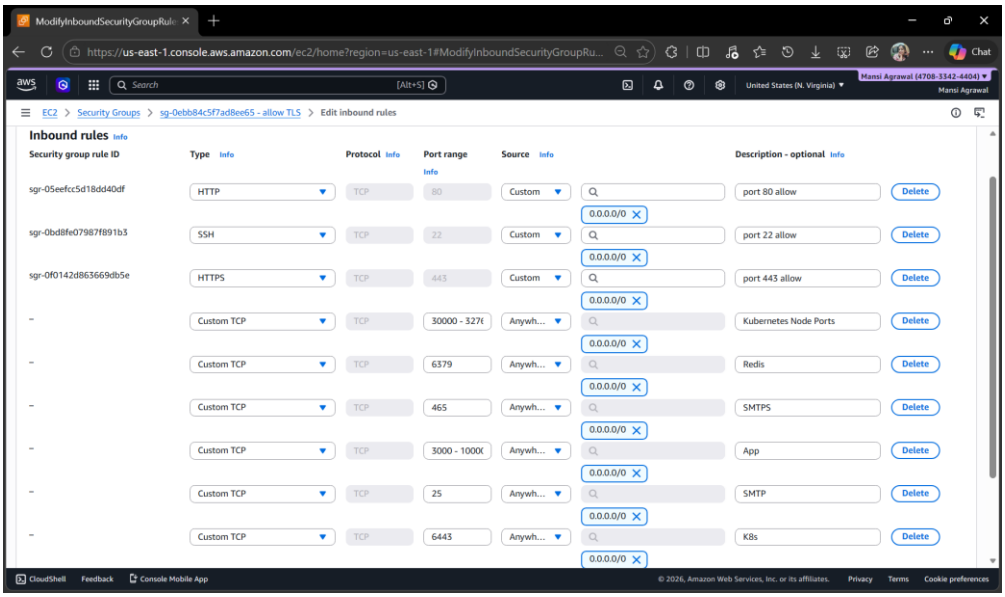




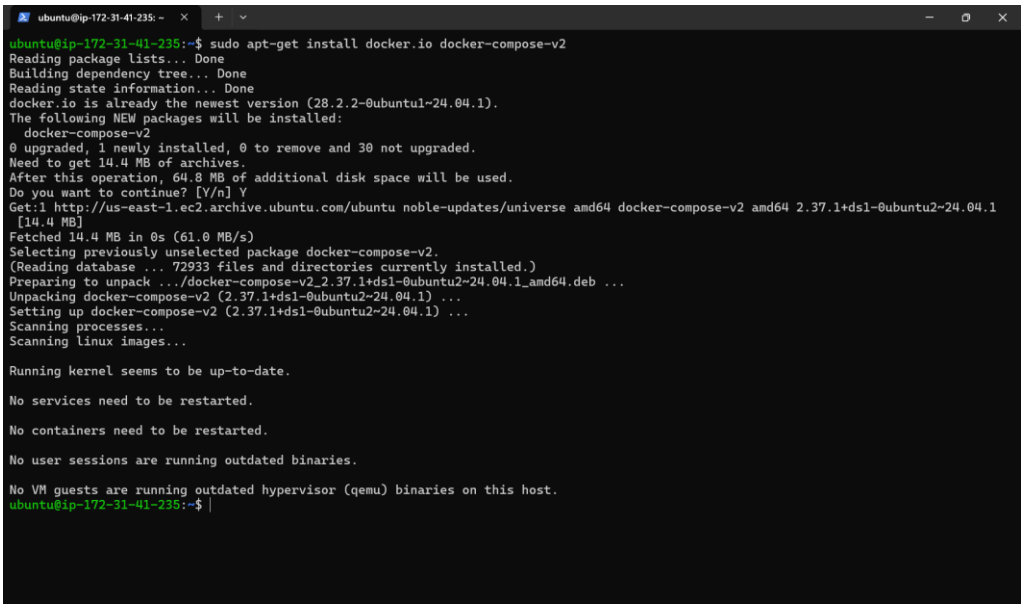

Instances | EC2 | us-east-1
https://us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#Instances:
Search
[Alt+S]
United States (N. Virginia)
Mansi Agrawal (4708-3342-4404)
Mansi Agrawal
EC2 > Instances
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
Capacity Manager New
Images
AMIs
AMI Catalog
Instances (1/1) Info
Connect
Instance state
Actions
Launch instances
Saved filter sets
Choose filter set
Find Instance by attribute or tag (case-sensitive)
Name
Instance ID
Instance state
Instance type
Status check
Alarm status
Automate
i-06b291d2573c02ff1
Running
t2.large
Initializing
View alarms
i-06b291d2573c02ff1 (Automate)
Details
Status and alarms
Monitoring
Security
Networking
Storage
Tags
Instance summary Info
Instance ID
i-06b291d2573c02ff1
Public IPv4 address
18.215.251.98 | open address
Private IPv4 addresses
172.31.41.235
IPv6 address
Instance state
Running
Public DNS
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

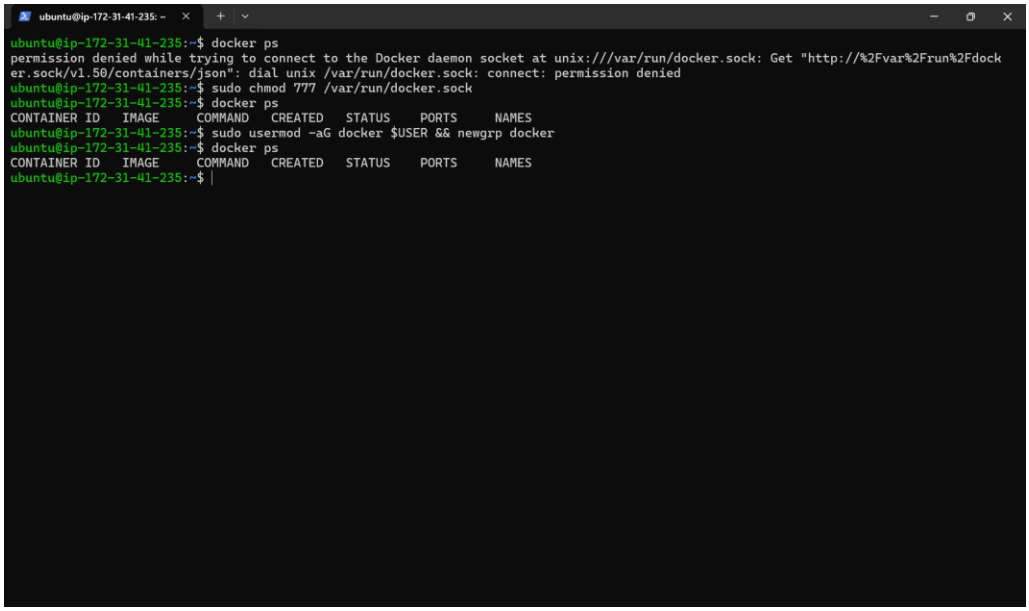

Connect to instance | EC2 | us-east
https://us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#ConnectToInstance:instanceId=i-06b...
Search
[Alt+S]
United States (N. Virginia)
Mansi Agrawal (4708-3342-4404)
Mansi Agrawal
EC2 > Instances > i-06b291d2573c02ff1 > Connect to instance
Connect Info
Connect to an instance using the browser-based client.
Instance ID
i-06b291d2573c02ff1 (Automate)
VPC ID
vpc-0b7f10b031e14c8c0
Security groups
sg-0ebb84c5f7ad8ee65 (allow TLS)
IAM role
–
EC2 Instance Connect
SSM Session Manager
SSH client
EC2 serial console
Instance ID
i-06b291d2573c02ff1 (Automate)
1. Open an SSH client.
2. Locate your private key file. The key used to launch this instance is terra-automate-key.pem
3. Run this command, if necessary, to ensure your key is not publicly viewable.
chmod 400 "terra-automate-key.pem"
4. Connect to your instance using its Public DNS:
ec2-18-215-251-98.compute-1.amazonaws.com
Example:
ssh -i "terra-automate-key.pem" ubuntu@ec2-18-215-251-98.compute-1.amazonaws.com
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

- Open the below ports in security group of master machine and also attach same security group to Jenkins worker node (We will create worker node shortly)

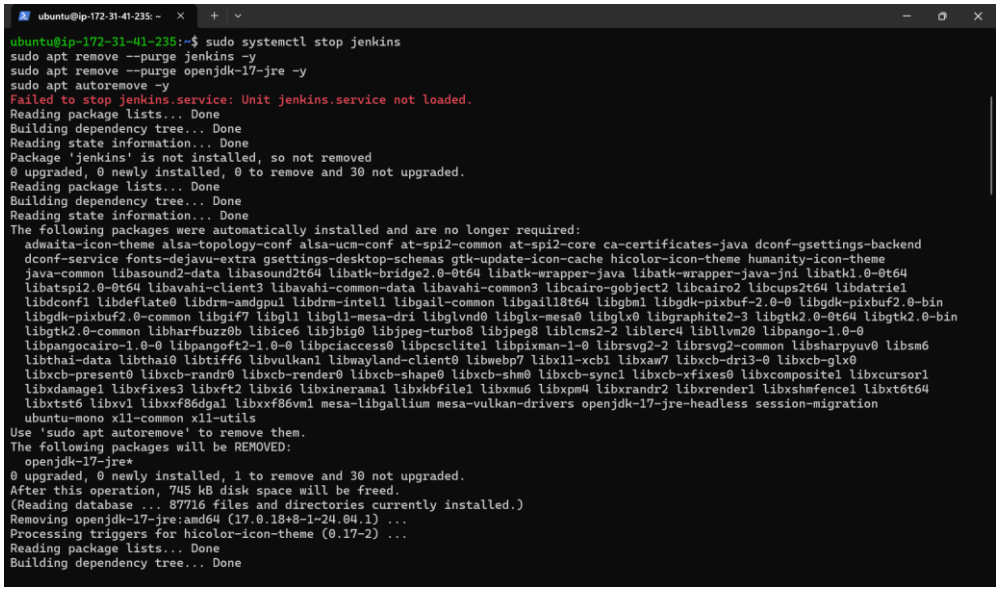


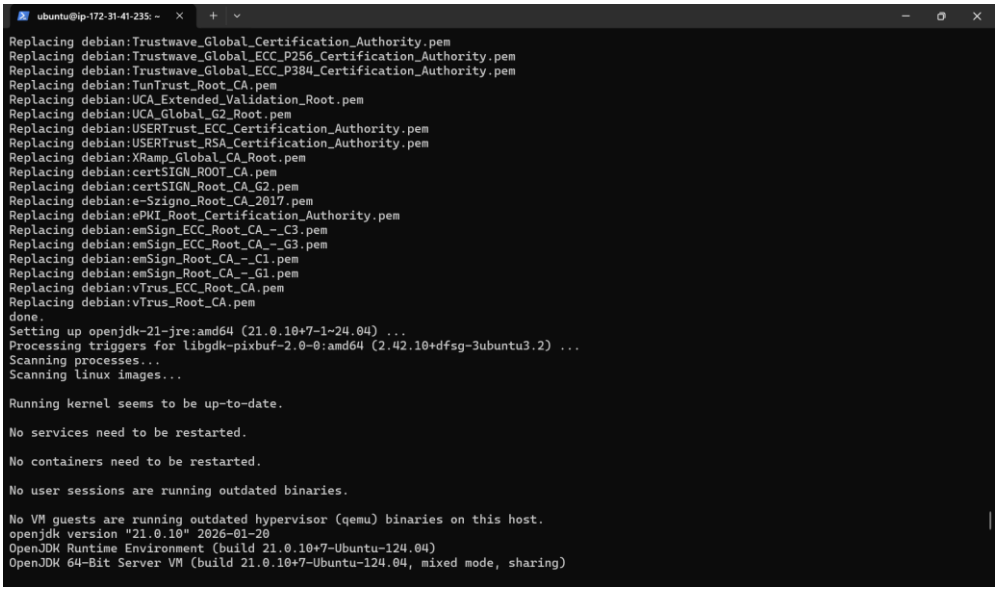
- Install & Configure Docker by using below command.

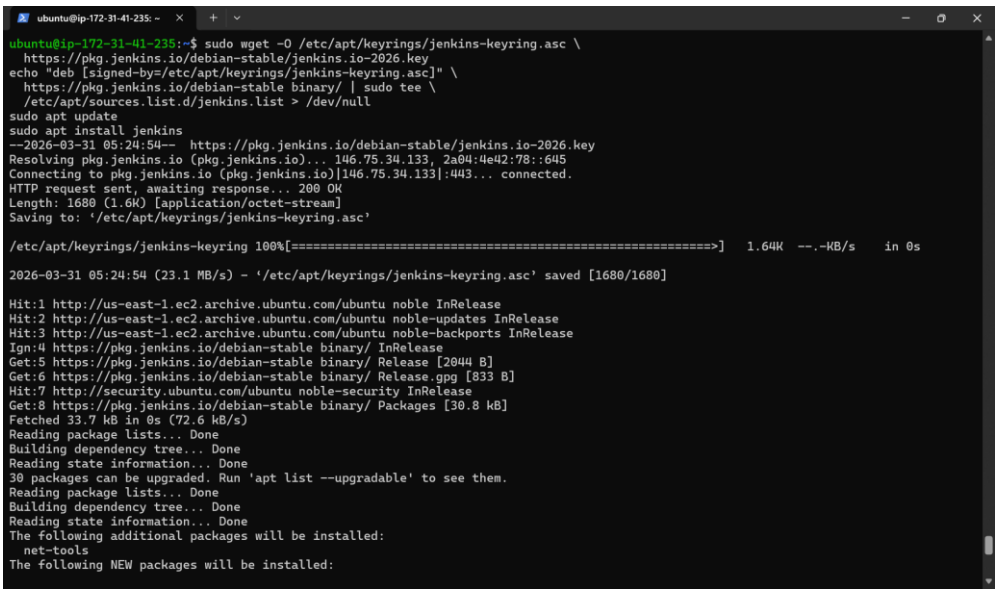


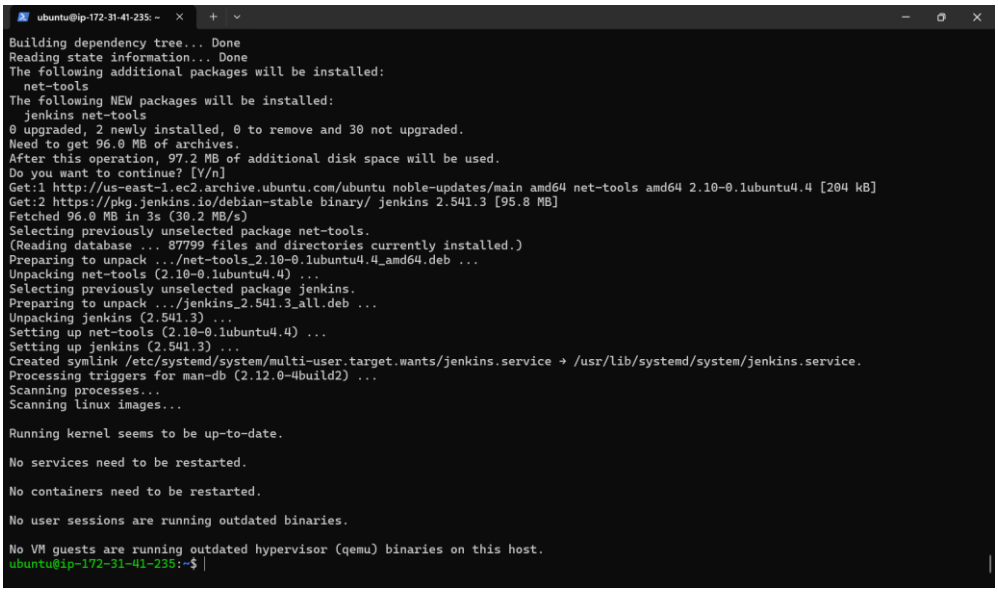


- Install and configure Jenkins (Master machine)



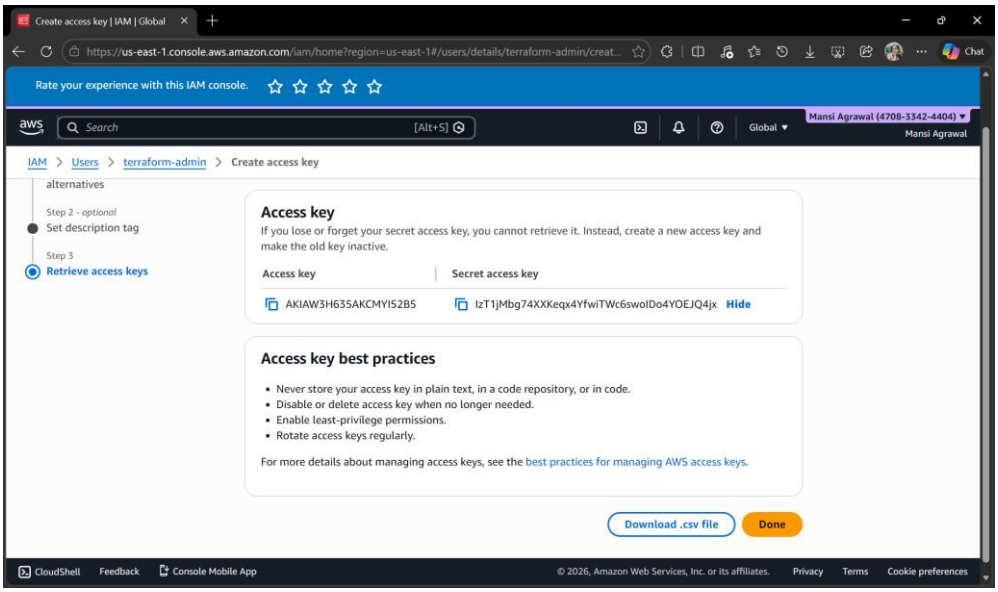


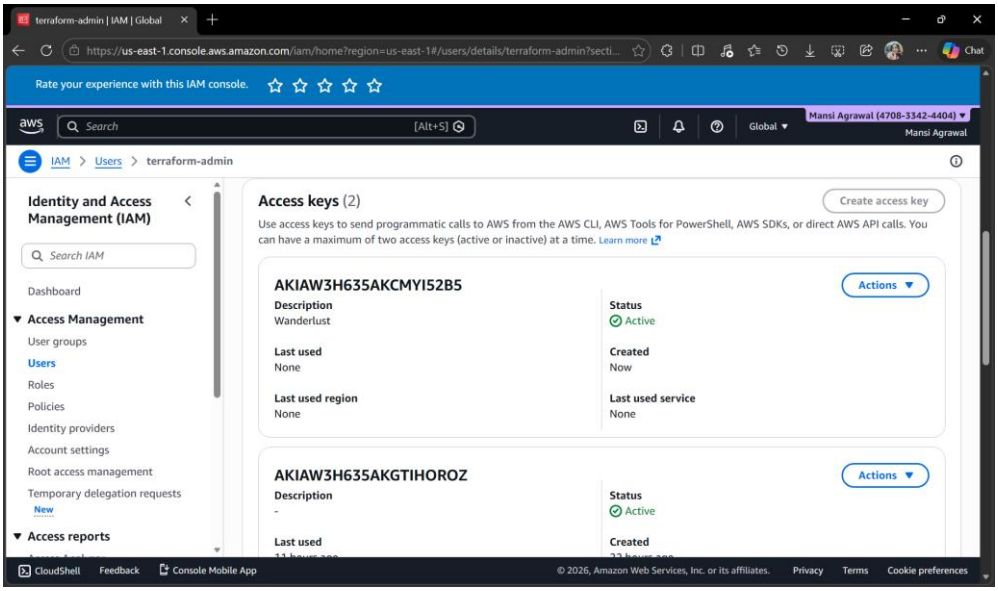




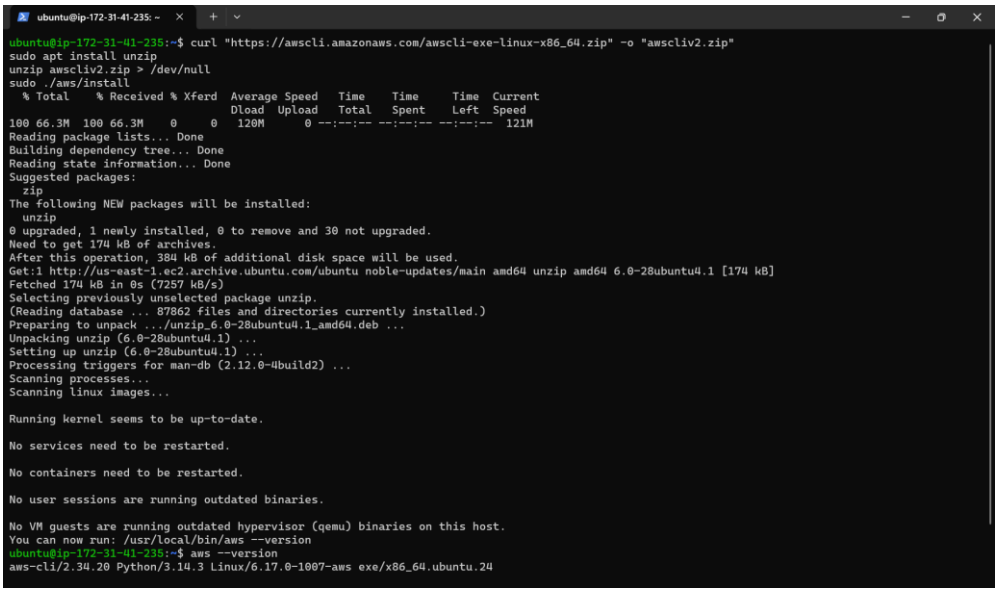
Now, access Jenkins Master on the browser on port 8080 and configure it.

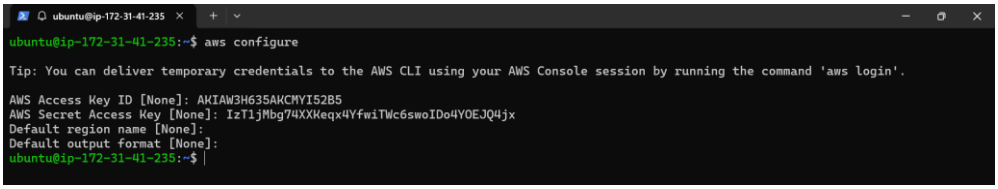
- Create EKS Cluster on AWS (Master machine)
    - IAM user with access keys and secret access keys



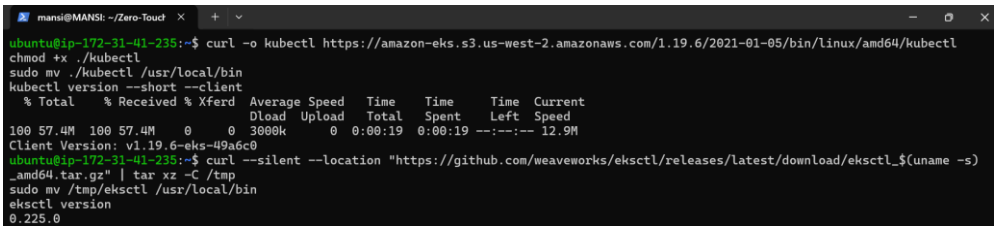


- AWSCLI should be configured

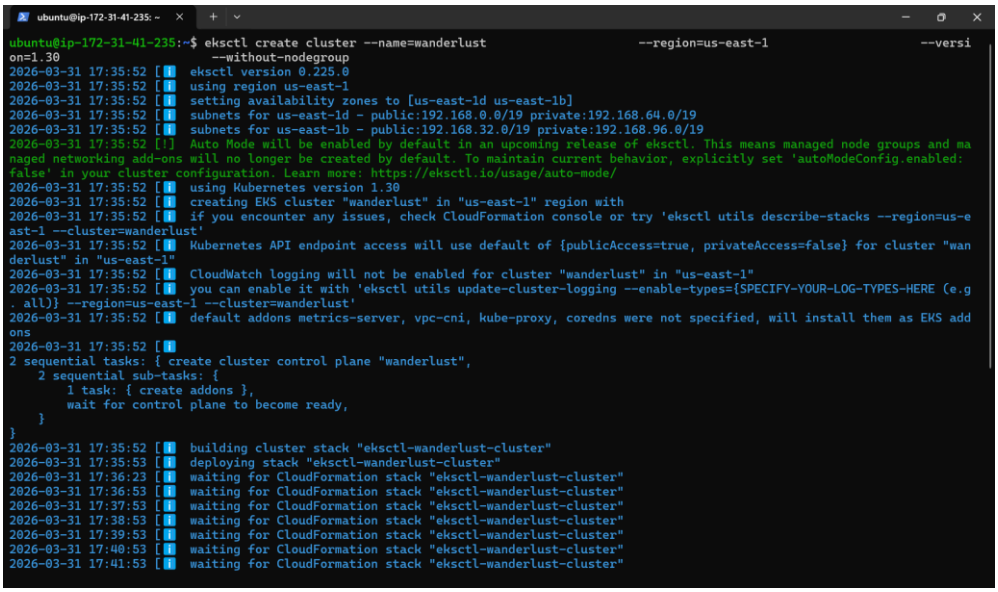


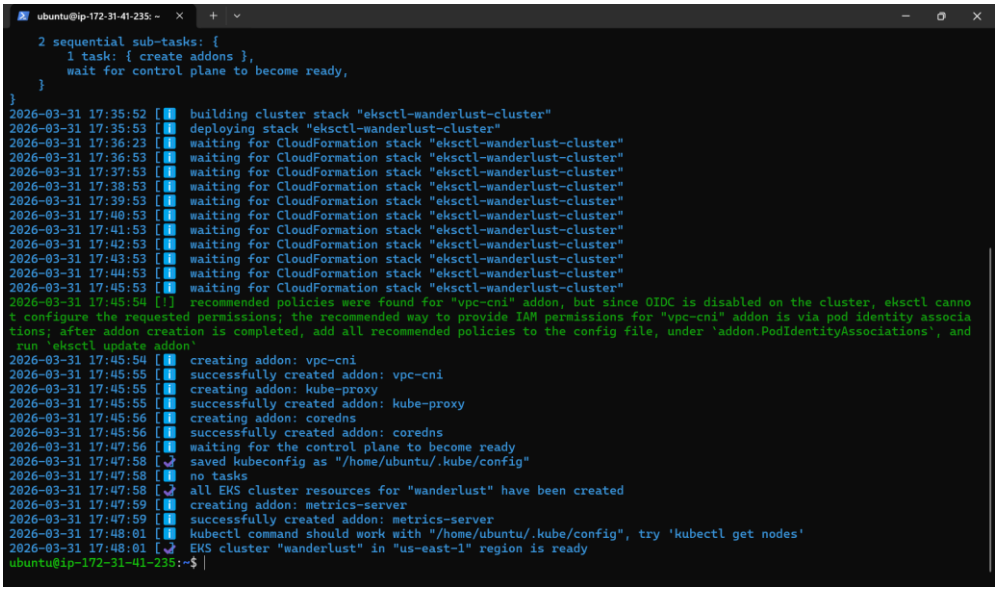


- Install kubectl and eksctl (Master machine)

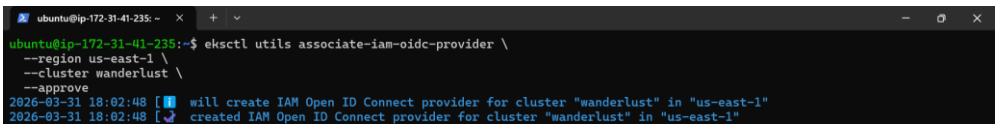


- Create EKS Cluster (Master machine)

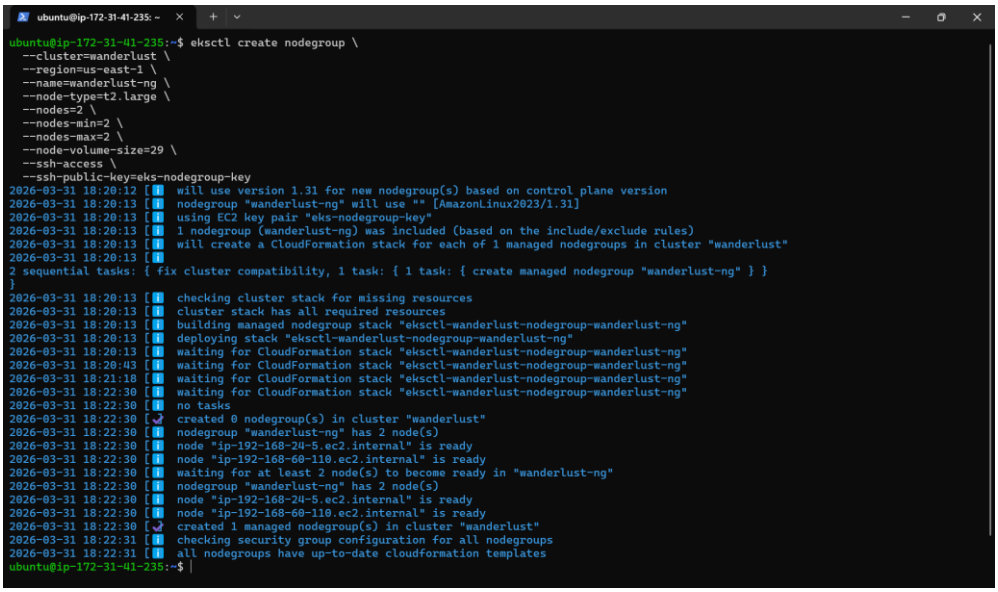




- Associate IAM OIDC Provider (Master machine)



- Create Nodegroup (Master machine)



4.2 User Interface

The user interface of this project is mainly based on the dashboards of the tools used in the system.

Jenkins provides a simple web interface where users can create and manage pipelines. It shows the status of each stage, such as whether the build is successful or failed. Users can also view logs to understand what is happening at each step.

Git platforms like GitHub or GitLab provide an interface to manage code repositories. Users can upload code, track changes, and collaborate with others easily.

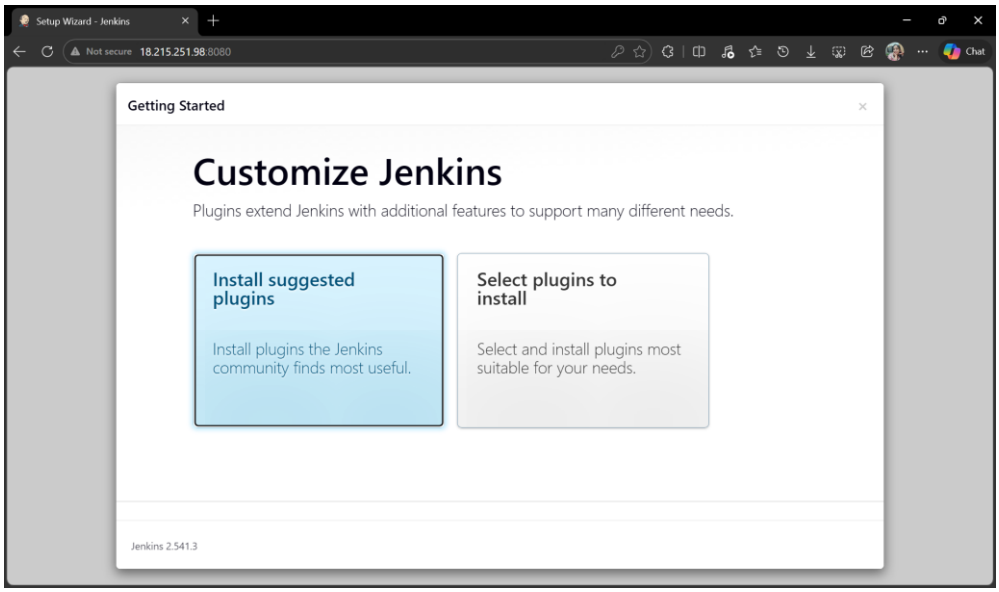
Docker and Kubernetes also provide dashboards or command-line tools to manage containers and deployments. These interfaces help users check whether the application is running properly.

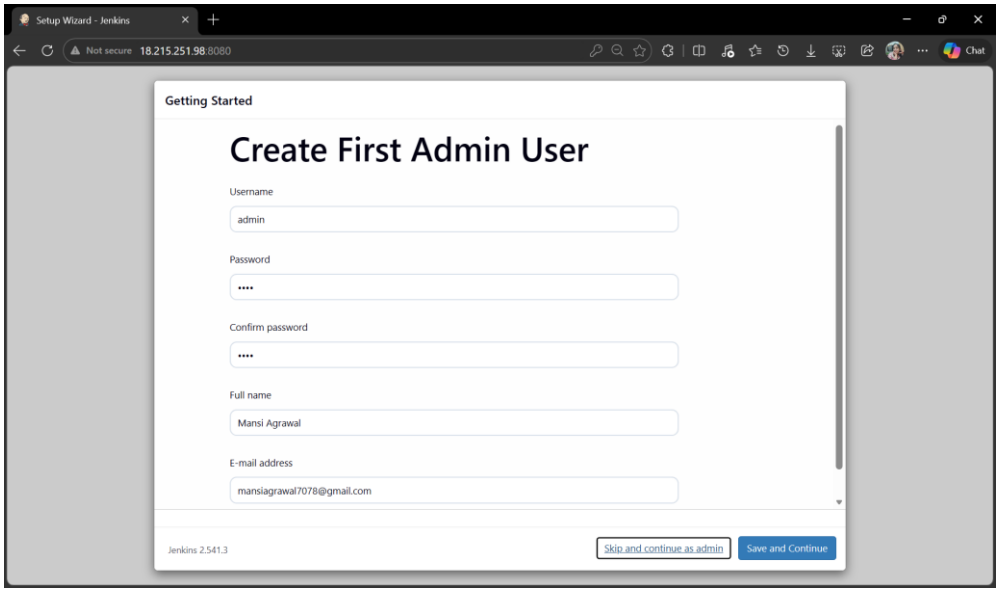
ArgoCD has a user-friendly dashboard where users can see the deployment status and sync updates with the Kubernetes cluster.

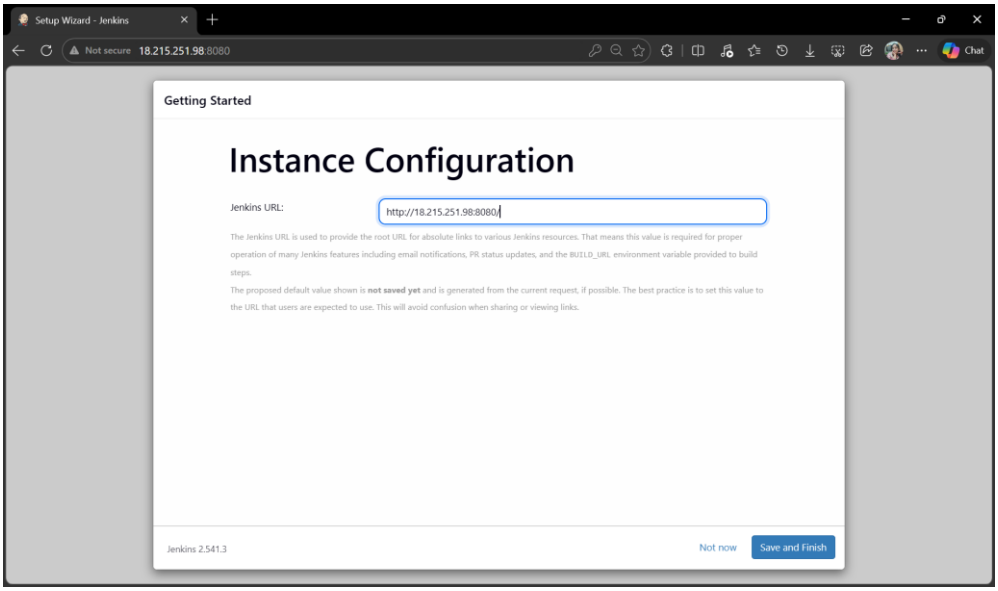
Monitoring tools like Grafana provide visual dashboards with graphs and charts. These help users understand system performance in a simple way.

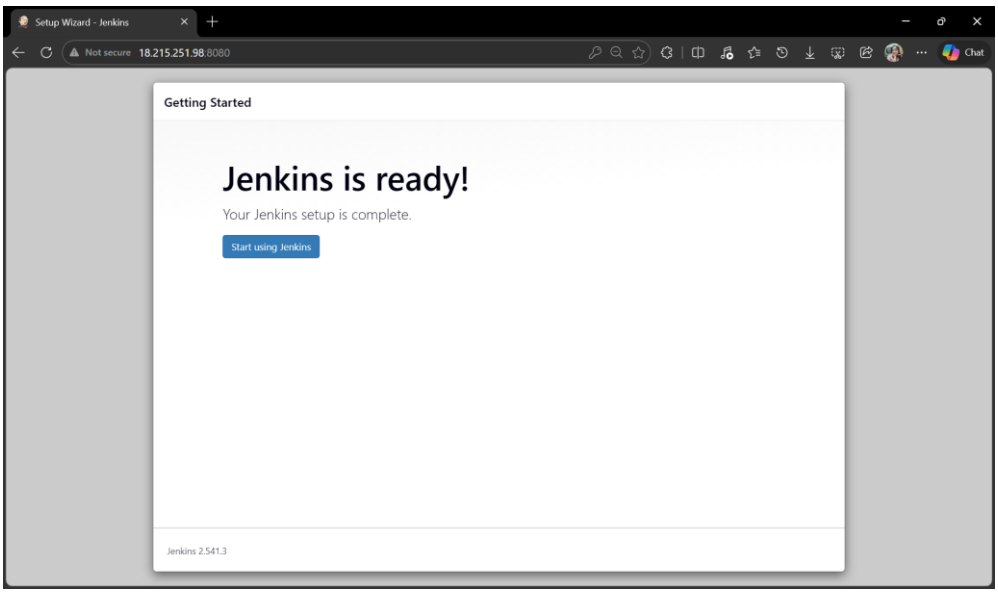
Overall, the user interface is not a single screen but a combination of different dashboards. All of them are easy to use and help in managing the system efficiently.

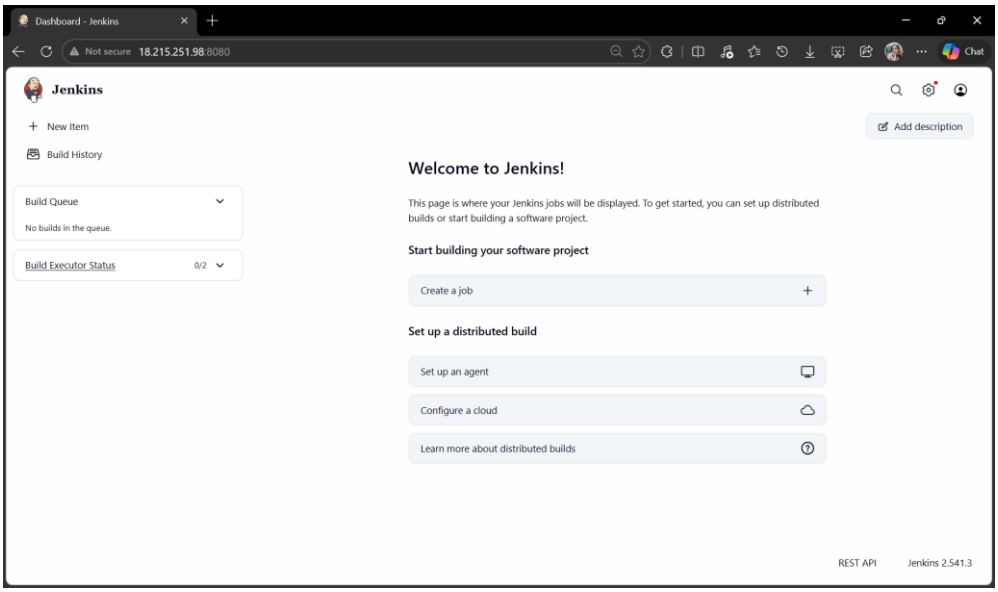
- Jenkins Setup



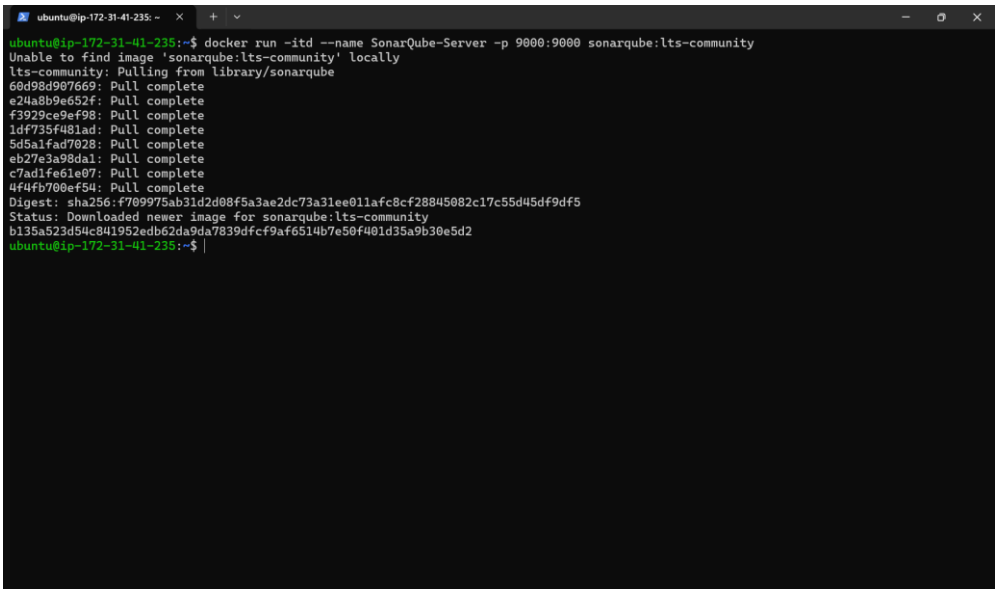


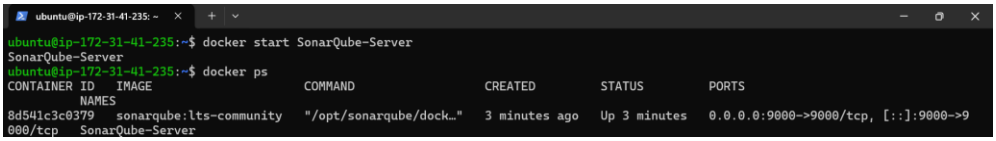

Setup Wizard - Jenkins
Not secure 18.215.251.98:8080
Chat
Getting Started
Instance Configuration
Jenkins URL:
http://18.215.251.98:8080/
The Jenkins URL is used to provide the root URL for absolute links to various Jenkins resources. That means this value is required for proper operation of many Jenkins features including email notifications, PR status updates, and the BUILD_URL environment variable provided to build steps.
The proposed default value shown is not saved yet and is generated from the current request, if possible. The best practice is to set this value to the URL that users are expected to use. This will avoid confusion when sharing or viewing links.
Jenkins 2.541.3
Not now
Save and Finish

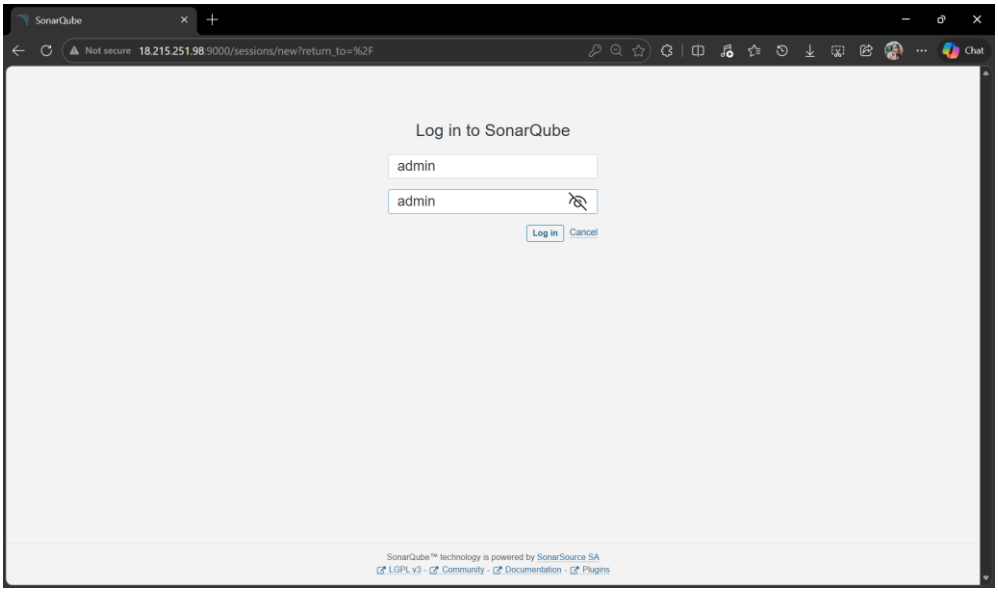

Setup Wizard - Jenkins
Not secure 18.215.251.98:8080
Chat
Getting Started
Jenkins is ready!
Your Jenkins setup is complete.
Start using Jenkins
Jenkins 2.541.3

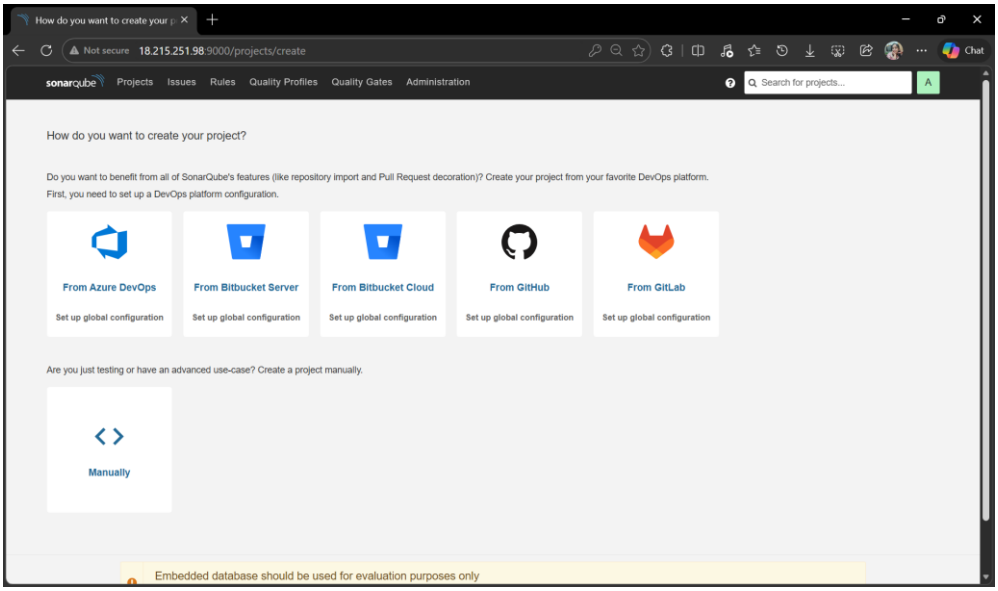

Dashboard - Jenkins
Not secure 18.215.251.98:8080
Chat
Jenkins
New Item
Build History
Build Queue
No builds in the queue.
Build Executor Status
0/2
Add description
Welcome to Jenkins!
This page is where your Jenkins jobs will be displayed. To get started, you can set up distributed builds or start building a software project.
Start building your software project
Create a job
Set up a distributed build
Set up an agent
Configure a cloud
Learn more about distributed builds
REST API
Jenkins 2.541.3

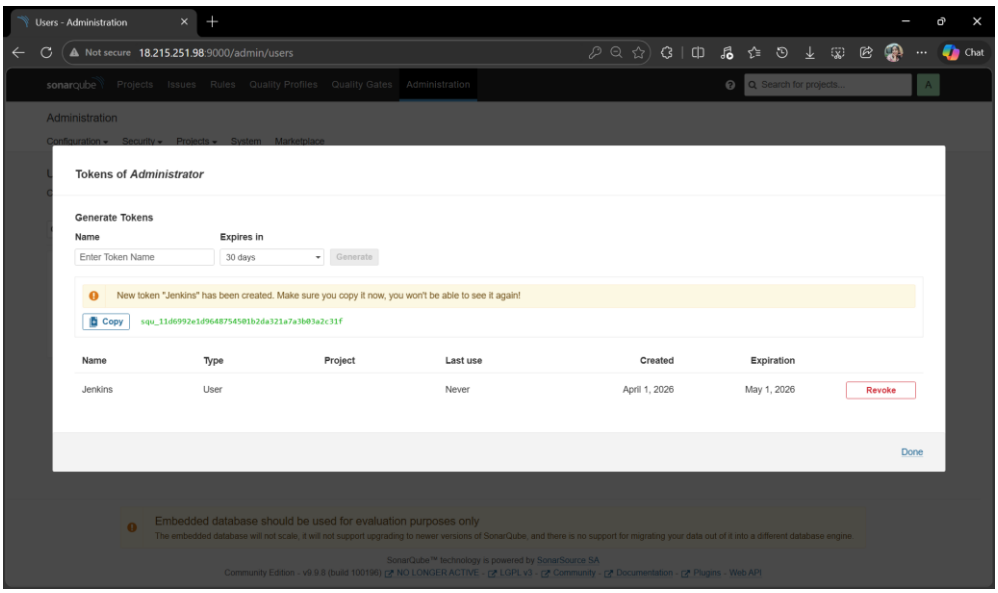
- SonarQube Setup

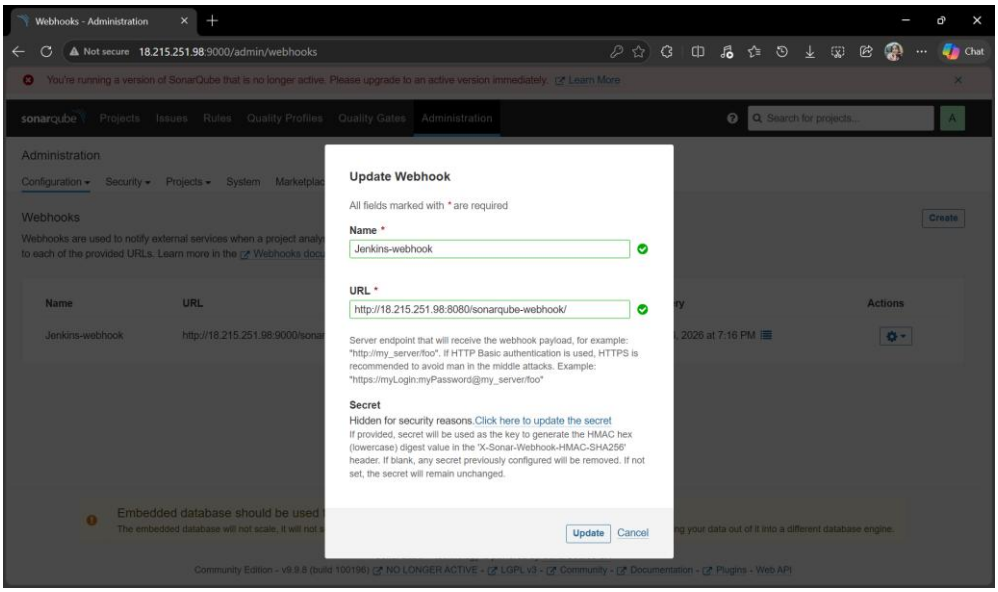


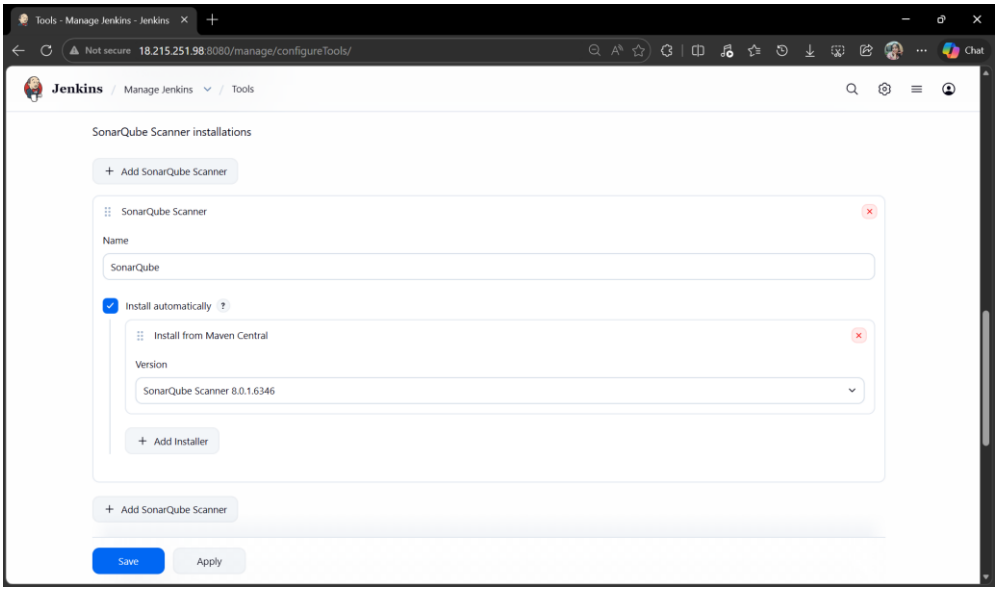


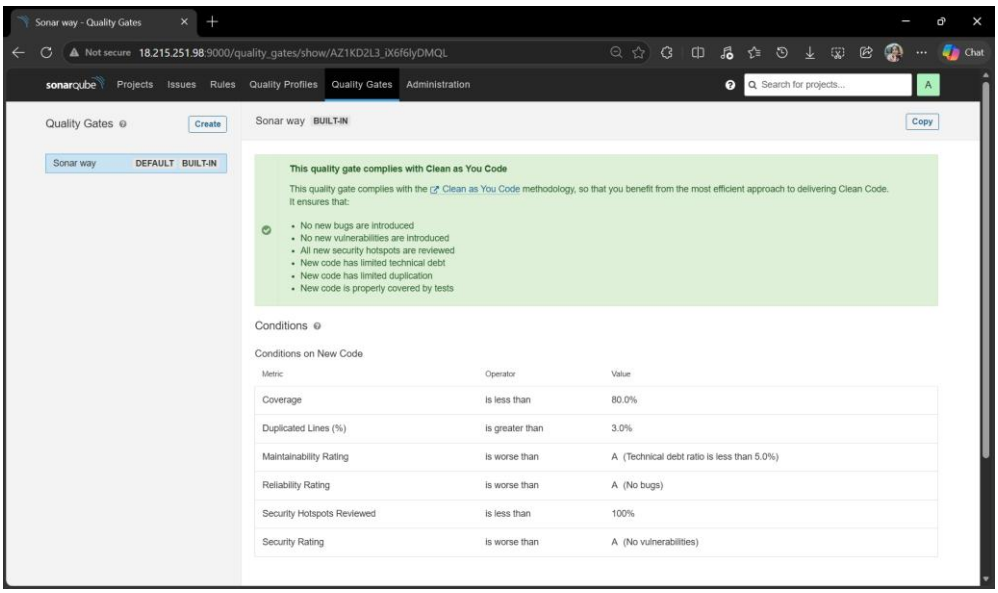



How do you want to create your p
Not secure 18.215.251.98:9000/projects/create
sonarqube Projects Issues Rules Quality Profiles Quality Gates Administration
Search for projects...
How do you want to create your project?
Do you want to benefit from all of SonarQube's features (like repository import and Pull Request decoration)? Create your project from your favorite DevOps platform.
First, you need to set up a DevOps platform configuration.
From Azure DevOps
Set up global configuration
From Bitbucket Server
Set up global configuration
From Bitbucket Cloud
Set up global configuration
From GitHub
Set up global configuration
From GitLab
Set up global configuration
Are you just testing or have an advanced use-case? Create a project manually.
Manually
Embedded database should be used for evaluation purposes only

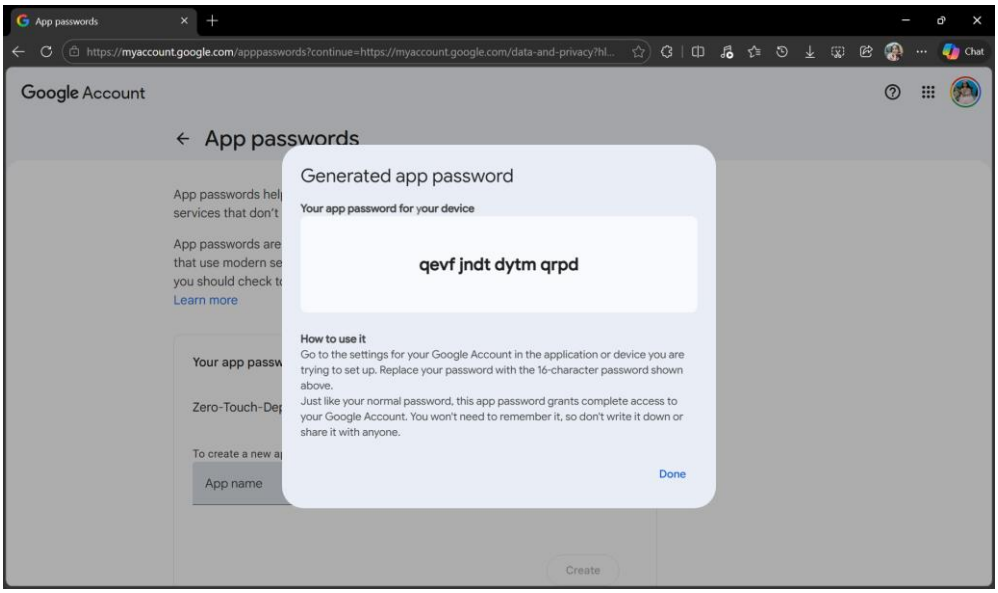

Users - Administration
Not secure 18.215.251.98:9000/admin/users
sonarqube Projects Issues Rules Quality Profiles Quality Gates Administration
Search for projects...
Administration
Configuration Security Projects System Marketplace
Tokens of Administrator
Generate Tokens
Name
Expires in
Enter Token Name
30 days
Generate
New token "Jenkins" has been created. Make sure you copy it now, you won't be able to see it again!
Copy
squ_11d6992e1d9648754501b2da321a7a3b03a2c31f
Name Type Project Last use Created Expiration
Jenkins User Never April 1, 2026 May 1, 2026
Revoke
Done
Embedded database should be used for evaluation purposes only
The embedded database will not scale, it will not support upgrading to newer versions of SonarQube, and there is no support for migrating your data out of it into a different database engine.
SonarQube™ technology is powered by SonarSource SA
Community Edition - v9.9.8 (build 100196) NO LONGER ACTIVE - LGPL v3 - Community - Documentation - Plugins - Web API

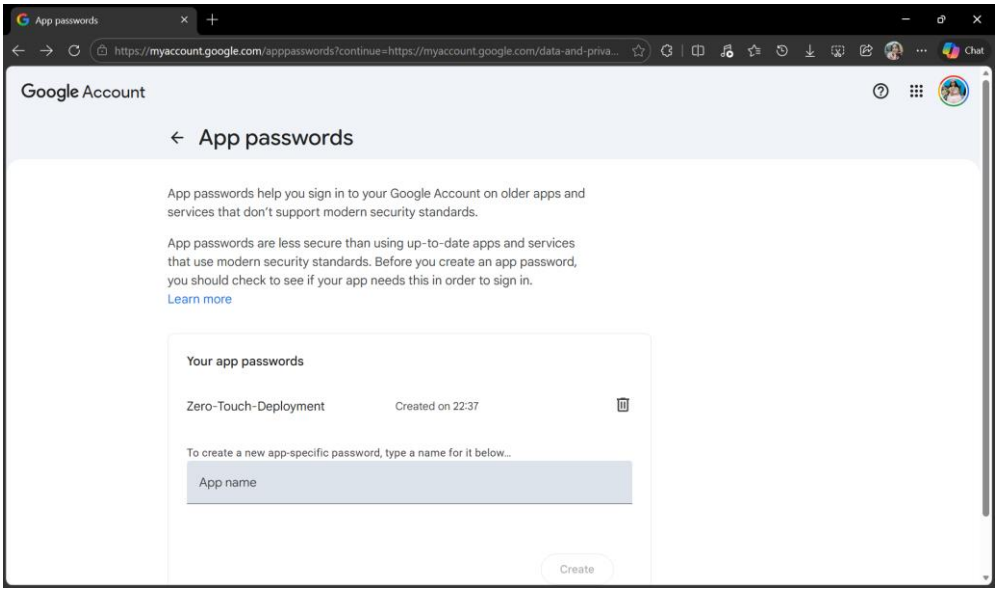

Webhooks - Administration
Not secure 18.215.251.98:9000/admin/webhooks
You're running a version of SonarQube that is no longer active. Please upgrade to an active version immediately. Learn More
sonarqube Projects Issues Rules Quality Profiles Quality Gates Administration
Search for projects...
Administration
Configuration Security Projects System Marketplace
Webhooks
Webhooks are used to notify external services when a project analy
to each of the provided URLs. Learn more in the Webhooks doc
Create
Name URL Actions
Jenkins-webhook http://18.215.251.98:9000/sonar 2026 at 7:16 PM
Update Webhook
All fields marked with * are required
Name *
Jenkins-webhook
URL *
http://18.215.251.98:8080/sonarqube-webhook/
Server endpoint that will receive the webhook payload, for example: "http://my_server/foo". If HTTP Basic authentication is used, HTTPS is recommended to avoid man in the middle attacks. Example: "https://myLogin:myPassword@my_server/foo"
Secret
Hidden for security reasons. Click here to update the secret
If provided, secret will be used as the key to generate the HMAC hex (lowercase) digest value in the 'X-Sonar-Webhook-HMAC-SHA256' header. If blank, any secret previously configured will be removed. If not set, the secret will remain unchanged.
Update
Cancel
Embedded database should be used
The embedded database will not scale, it will not
your data out of it into a different database engine.
Community Edition - v9.9.8 (build 100196) NO LONGER ACTIVE - LGPL v3 - Community - Documentation - Plugins - Web API

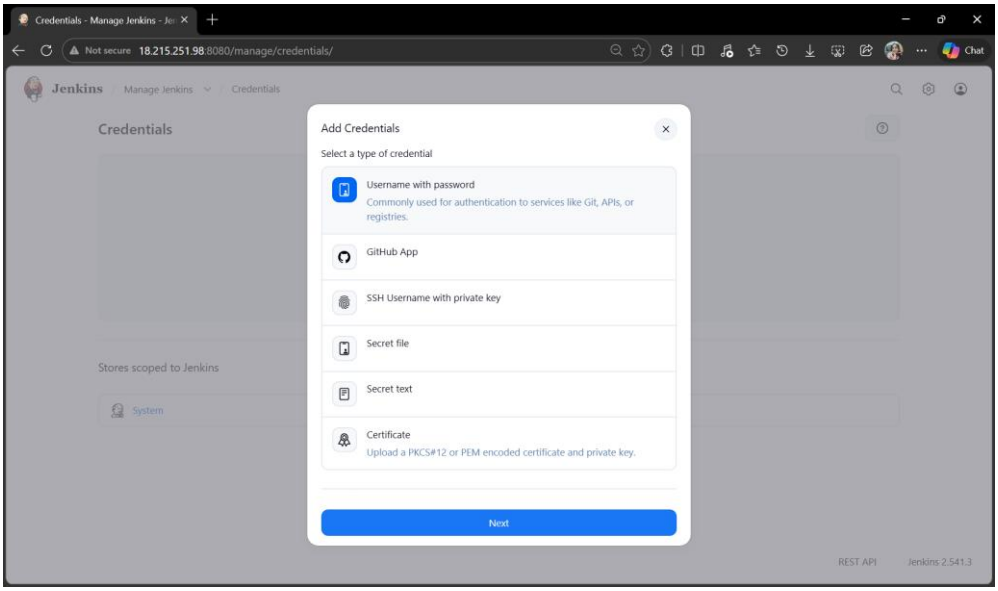

Tools - Manage Jenkins - Jenkins
Not secure 18.215.251.98:8080/manage/configureTools/
Jenkins / Manage Jenkins / Tools
SonarQube Scanner installations
Add SonarQube Scanner
SonarQube Scanner
Name
SonarQube
Install automatically
Install from Maven Central
Version
SonarQube Scanner 8.0.1.6346
Add Installer
Add SonarQube Scanner
Save
Apply

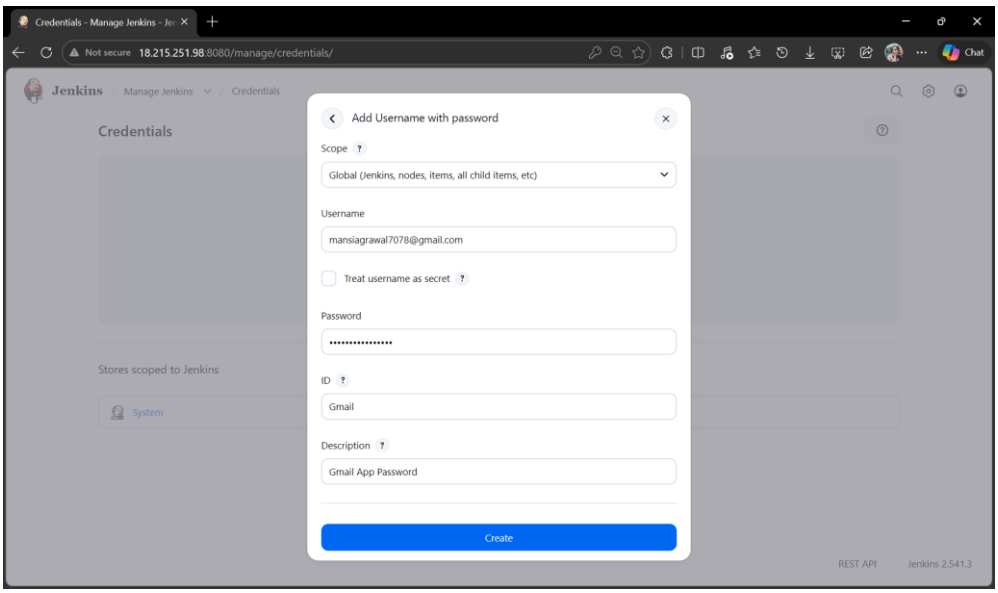

Sonar way - Quality Gates
Not secure 18.215.251.98:9000/quality_gates/show/AZ1KD2L3_iX6f6lyDMQL
sonarqube Projects Issues Rules Quality Profiles Quality Gates Administration
Search for projects...
Quality Gates
Create
Sonar way DEFAULT BUILT-IN
Sonar way BUILT-IN
Copy
This quality gate complies with Clean as You Code
This quality gate complies with the Clean as You Code methodology, so that you benefit from the most efficient approach to delivering Clean Code.
It ensures that:
No new bugs are introduced
No new vulnerabilities are introduced
All new security hotspots are reviewed
New code has limited technical debt
New code has limited duplication
New code is properly covered by tests
Conditions
Conditions on New Code
Metric
Operator
Value
Coverage
is less than
80.0%
Duplicated Lines (%)
is greater than
3.0%
Maintainability Rating
is worse than
A (Technical debt ratio is less than 5.0%)
Reliability Rating
is worse than
A (No bugs)
Security Hotspots Reviewed
is less than
100%
Security Rating
is worse than
A (No vulnerabilities)

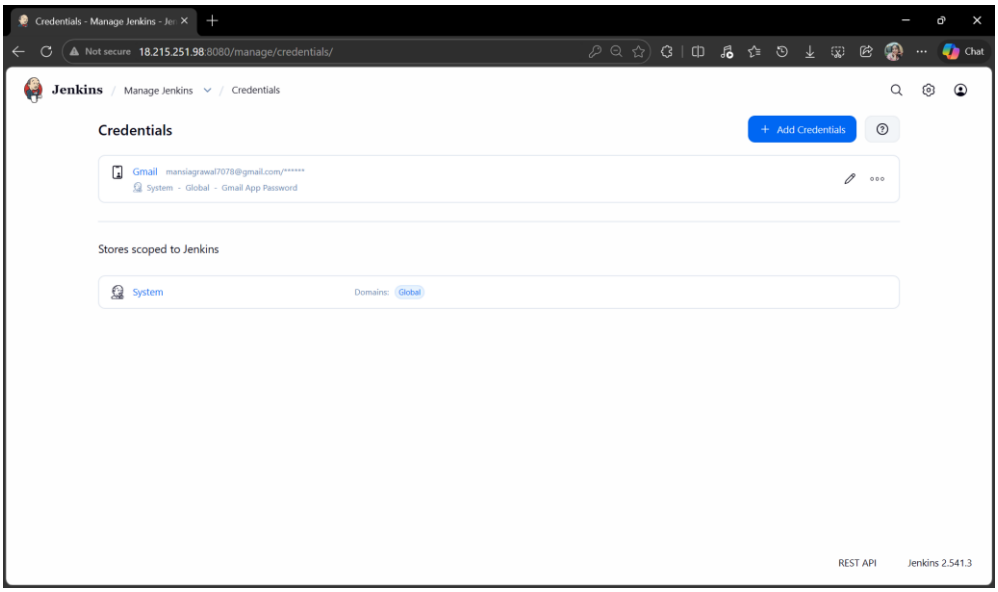
- Email Notification Setup

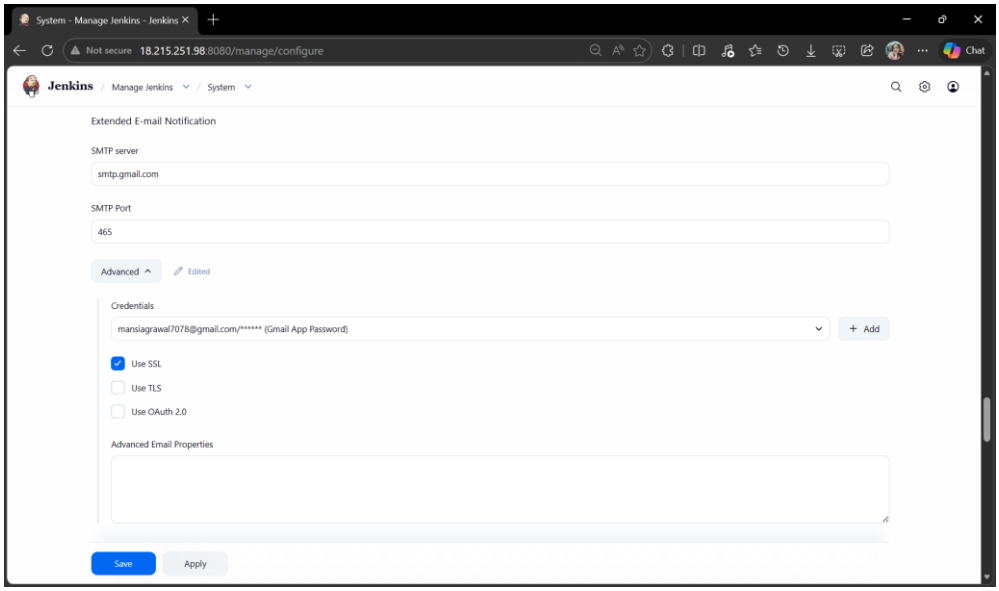


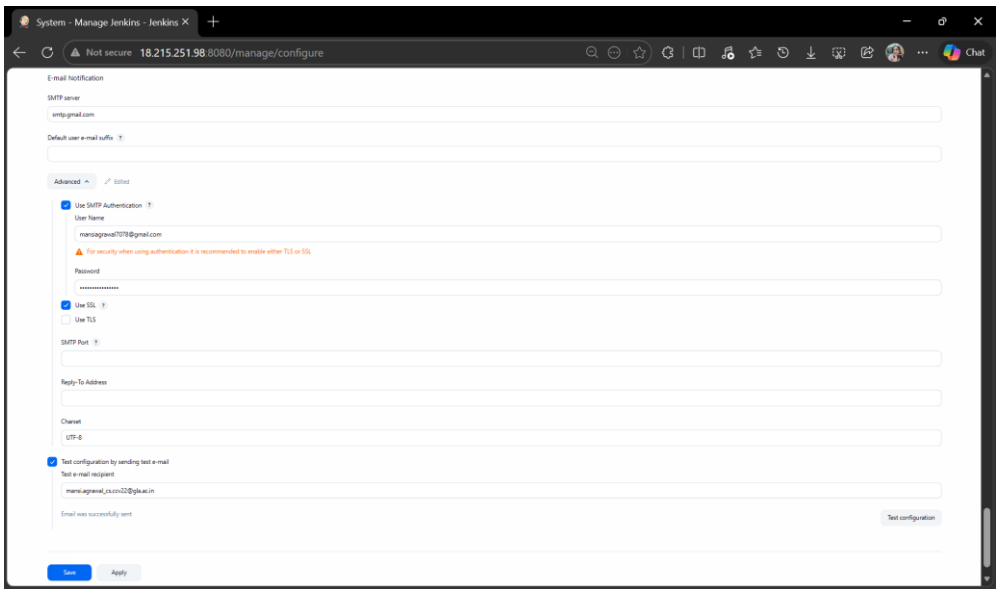


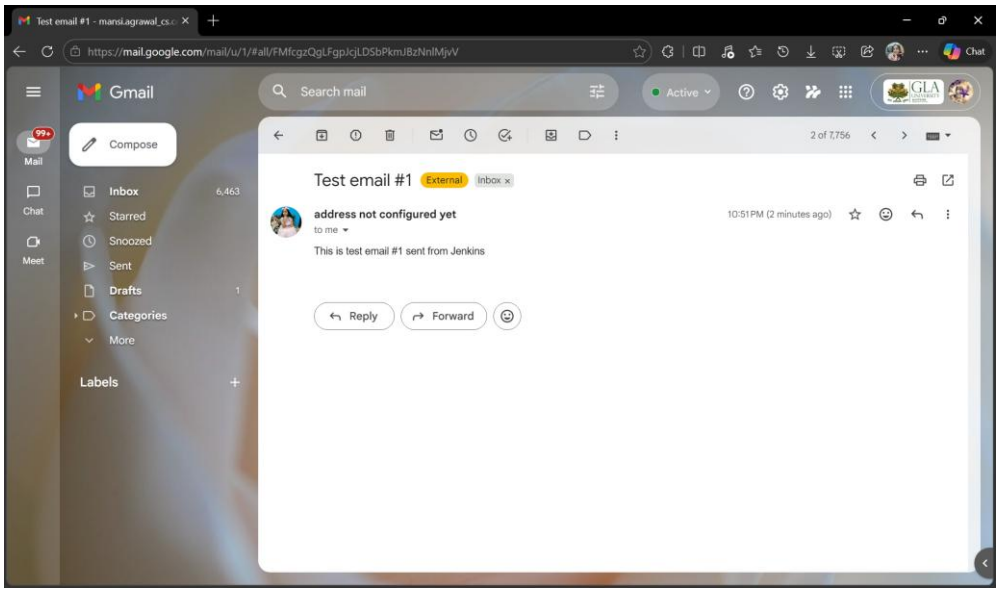

Credentials - Manage Jenkins
Not secure 18.215.251.98:8080/manage/credentials/
Jenkins
Manage Jenkins
Credentials
Add Credentials
Select a type of credential
Username with password
Commonly used for authentication to services like Git, APIs, or registries.
GitHub App
SSH Username with private key
Secret file
Secret text
Certificate
Upload a PKCS#12 or PEM encoded certificate and private key.
Next
Stores scoped to Jenkins
System
REST API
Jenkins 2.541.3


Credentials - Manage Jenkins
Not secure 18.215.251.98:8080/manage/credentials/
Jenkins
Manage Jenkins
Credentials
Add Username with password
Scope
Global (Jenkins, nodes, items, all child items, etc)
Username
mansiagrawal7078@gmail.com
Treat username as secret
Password
ID
Gmail
Description
Gmail App Password
Create
Stores scoped to Jenkins
System
REST API
Jenkins 2.541.3

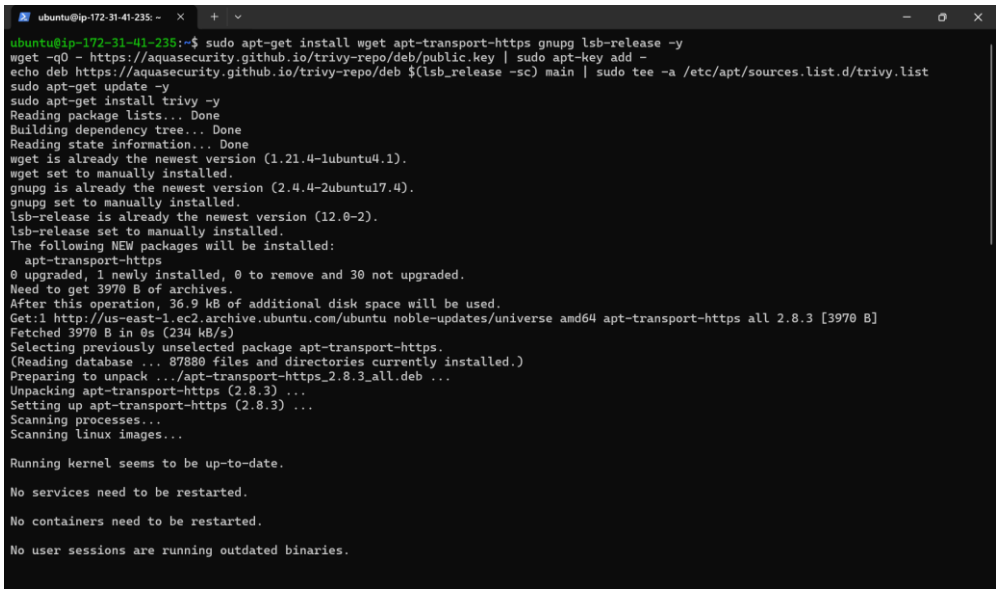

Credentials - Manage Jenkins
Not secure 18.215.251.98:8080/manage/credentials/
Jenkins
Manage Jenkins
Credentials
Add Credentials
Gmail
mansiagrawal7078@gmail.com/******
System - Global - Gmail App Password
Stores scoped to Jenkins
System
Domains: Global
REST API
Jenkins 2.541.3

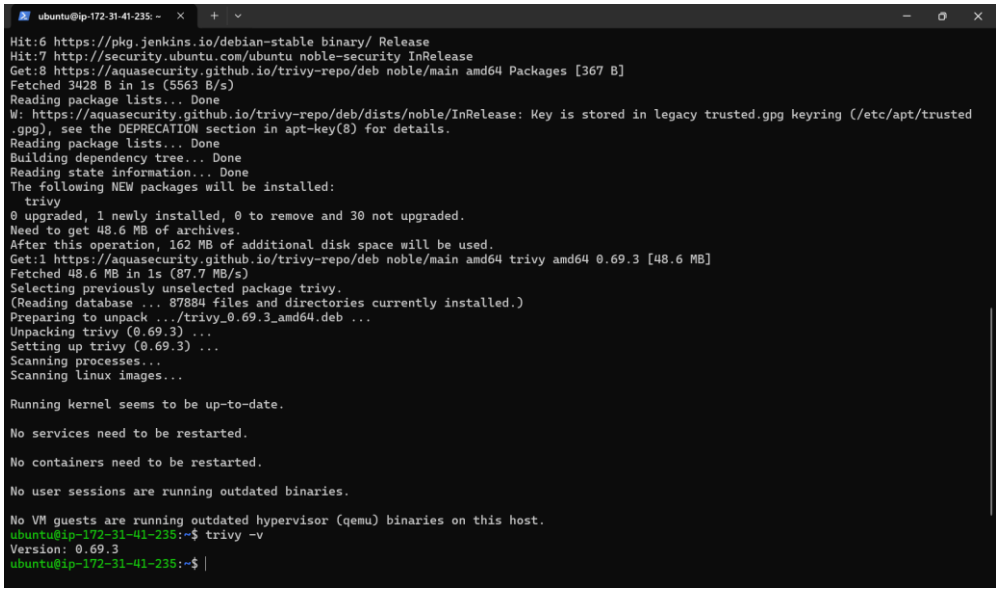

System - Manage Jenkins - Jenkins
Not secure 18.215.251.98:8080/manage/configure
Jenkins / Manage Jenkins / System
Extended E-mail Notification
SMTP server
smtp.gmail.com
SMTP Port
465
Advanced
Edited
Credentials
mansiagrawal7078@gmail.com/****** (Gmail App Password)
Add
Use SSL
Use TLS
Use OAuth 2.0
Advanced Email Properties
Save
Apply


System - Manage Jenkins - Jenkins
Not secure 18.215.251.98:8080/manage/configure
E-mail Notification
SMTP server
smtp.gmail.com
Default user e-mail suffix
Advanced
Edited
Use SMTP Authentication
User Name
mansiagrawal7078@gmail.com
Password
Use SSL
Use TLS
SMTP Port
Reply-To Address
Charset
UTF-8
Test configuration by sending test e-mail
Test e-mail recipient
Email was successfully sent
Test configuration
Save
Apply

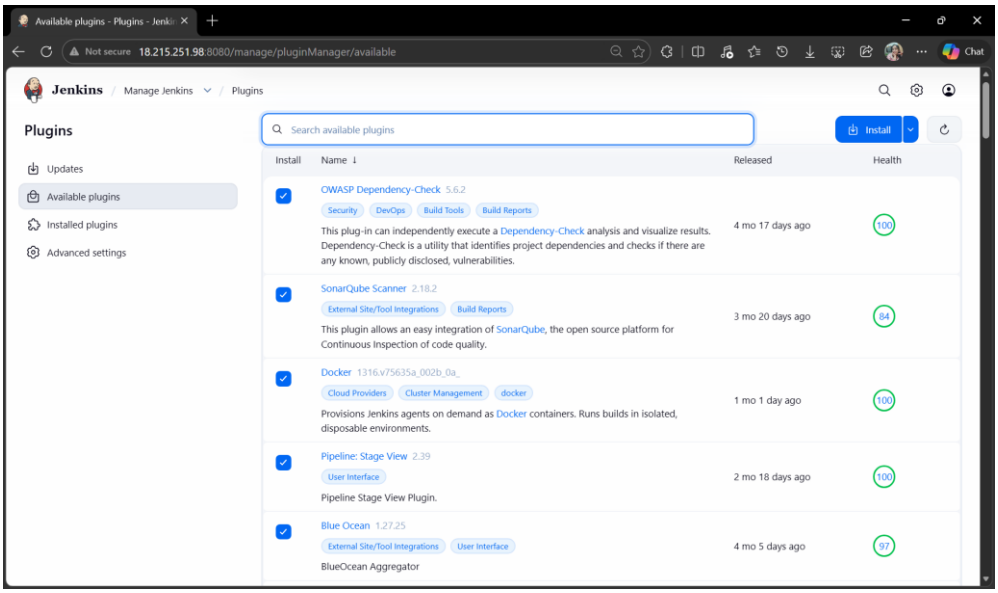

Test email #1 - mansi.agrawal_cs.c
https://mail.google.com/mail/u/1/#all/FMfcgzQgLFgpJcjLDSbPkmJBzNnlMjvV
Gmail
Search mail
Active
Compose
Mail
Chat
Meet
Inbox 6,463
Starred
Snoozed
Sent
Drafts 1
Categories
More
Labels
2 of 7,756
Test email #1
External
Inbox
address not configured yet
to me
10:51 PM (2 minutes ago)
This is test email #1 sent from Jenkins
Reply
Forward

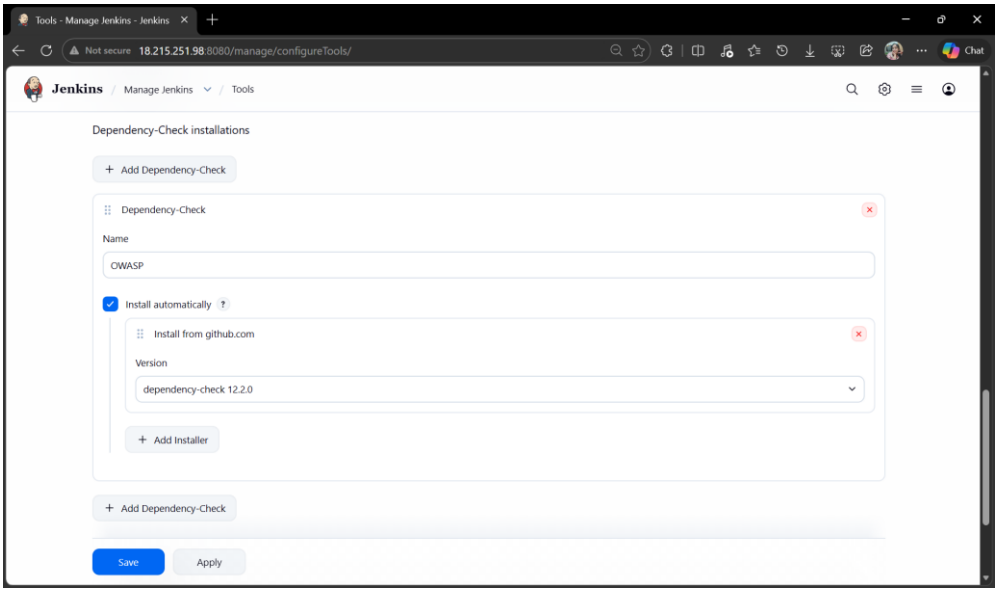
- Trivy Setup


ubuntu@ip-172-31-41-235: ~
ubuntu@ip-172-31-41-235:~$ sudo apt-get install wget apt-transport-https gnupg lsb-release -y
wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key | sudo apt-key add -
echo deb https://aquasecurity.github.io/trivy-repo/deb $(lsb_release -sc) main | sudo tee -a /etc/apt/sources.list.d/trivy.list
sudo apt-get update -y
sudo apt-get install trivy -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
wget is already the newest version (1.21.4-1ubuntu4.1).
wget set to manually installed.
gnupg is already the newest version (2.4.4-2ubuntu17.4).
gnupg set to manually installed.
lsb-release is already the newest version (12.0-2).
lsb-release set to manually installed.
The following NEW packages will be installed:
apt-transport-https
0 upgraded, 1 newly installed, 0 to remove and 30 not upgraded.
Need to get 3970 B of archives.
After this operation, 36.9 kB of additional disk space will be used.
Get:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 apt-transport-https all 2.8.3 [3970 B]
Fetched 3970 B in 0s (234 kB/s)
Selecting previously unselected package apt-transport-https.
(Reading database ... 87880 files and directories currently installed.)
Preparing to unpack .../apt-transport-https_2.8.3_all.deb ...
Unpacking apt-transport-https (2.8.3) ...
Setting up apt-transport-https (2.8.3) ...
Scanning processes...
Scanning linux images...
Running kernel seems to be up-to-date.
No services need to be restarted.
No containers need to be restarted.
No user sessions are running outdated binaries.

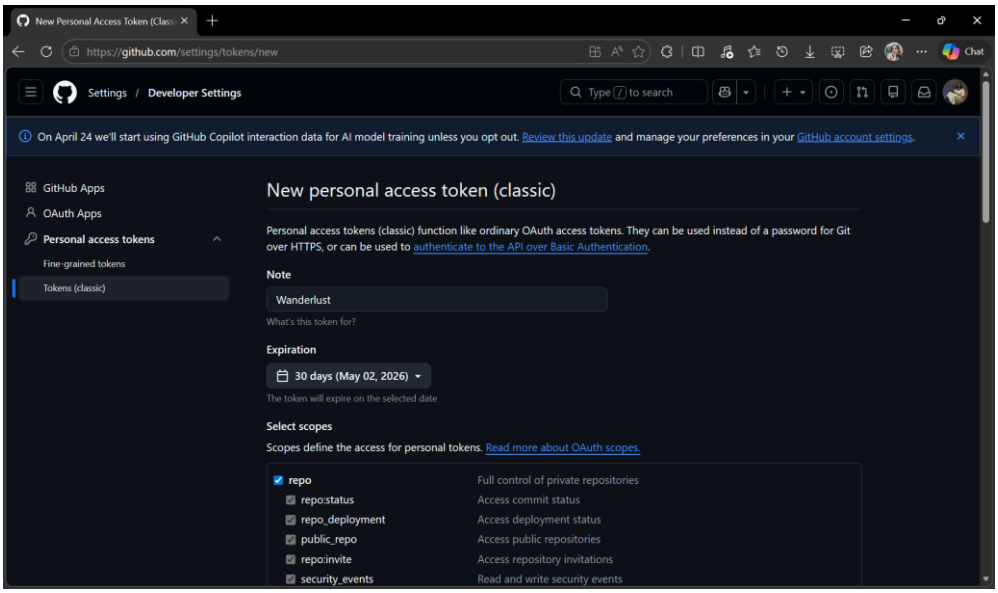

ubuntu@ip-172-31-41-235: ~
Hit:6 https://pkg.jenkins.io/debian-stable binary/ Release
Hit:7 http://security.ubuntu.com/ubuntu noble-security InRelease
Get:8 https://aquasecurity.github.io/trivy-repo/deb noble/main amd64 Packages [367 B]
Fetched 3428 B in 1s (5563 B/s)
Reading package lists... Done
W: https://aquasecurity.github.io/trivy-repo/deb/dists/noble/InRelease: Key is stored in legacy trusted.gpg keyring (/etc/apt/trusted.gpg), see the DEPRECATION section in apt-key(8) for details.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
trivy
0 upgraded, 1 newly installed, 0 to remove and 30 not upgraded.
Need to get 48.6 MB of archives.
After this operation, 162 MB of additional disk space will be used.
Get:1 https://aquasecurity.github.io/trivy-repo/deb noble/main amd64 trivy amd64 0.69.3 [48.6 MB]
Fetched 48.6 MB in 1s (87.7 MB/s)
Selecting previously unselected package trivy.
(Reading database ... 87884 files and directories currently installed.)
Preparing to unpack .../trivy_0.69.3_amd64.deb ...
Unpacking trivy (0.69.3) ...
Setting up trivy (0.69.3) ...
Scanning processes...
Scanning linux images...
Running kernel seems to be up-to-date.
No services need to be restarted.
No containers need to be restarted.
No user sessions are running outdated binaries.
No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-41-235:~$ trivy -v
Version: 0.69.3
ubuntu@ip-172-31-41-235:~$

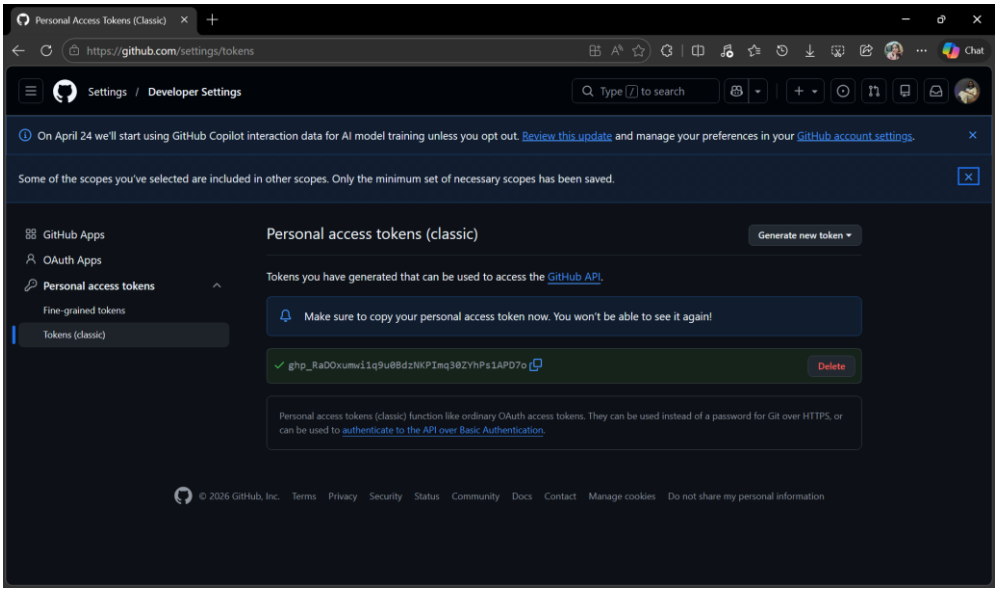
- Plugins & OWASP Setup

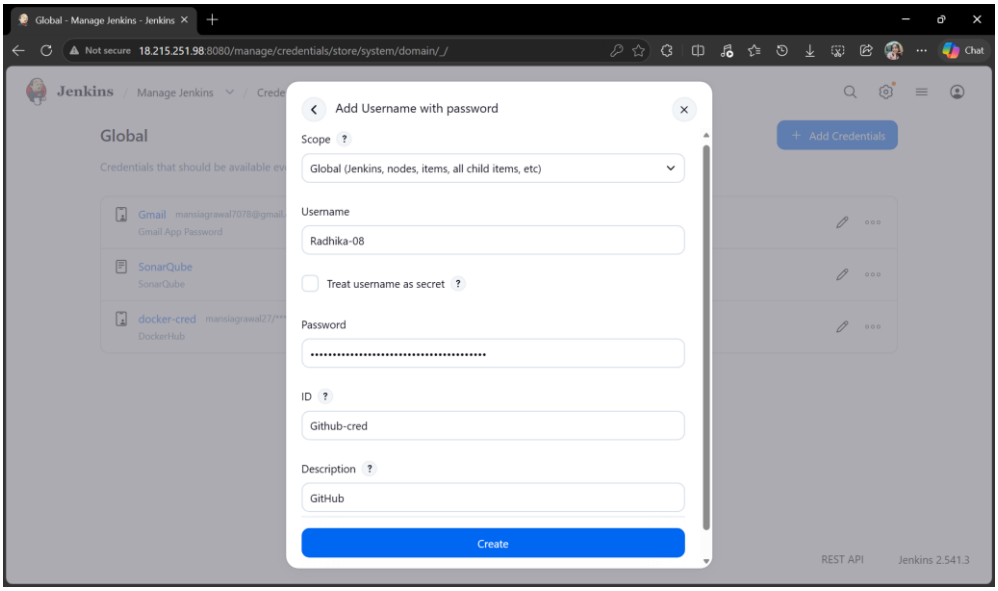




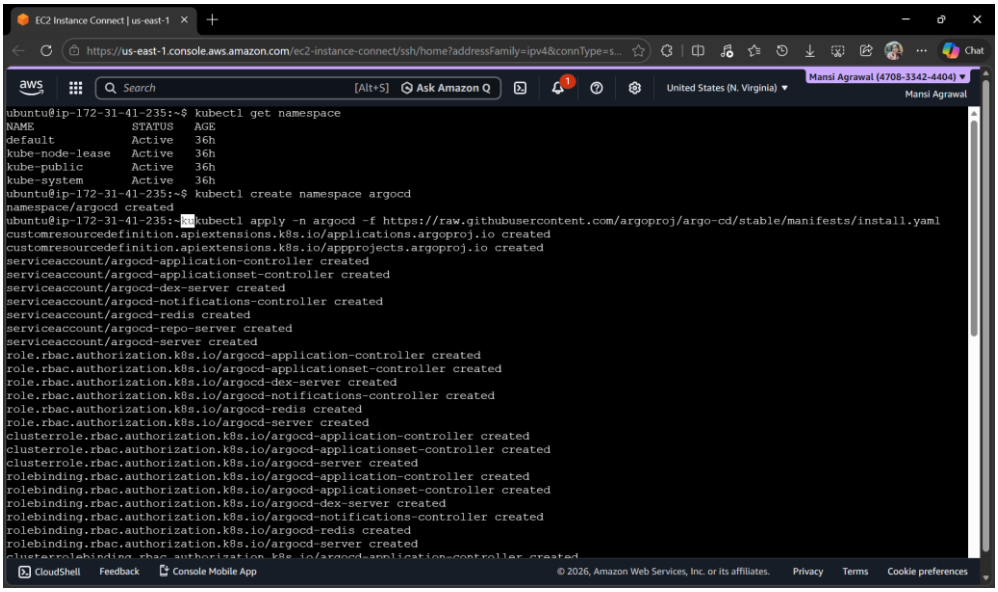
- GitHub Setup

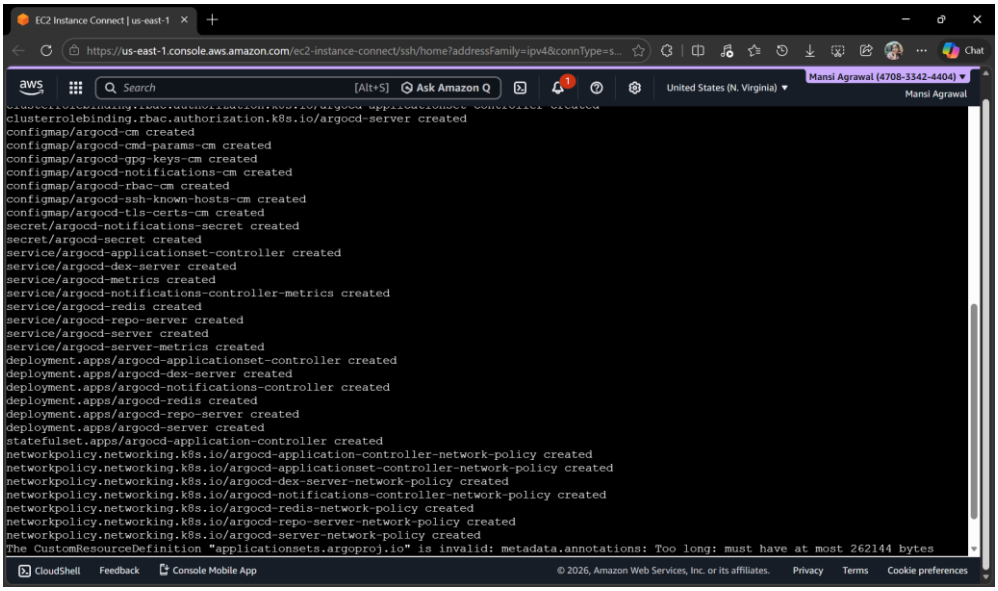


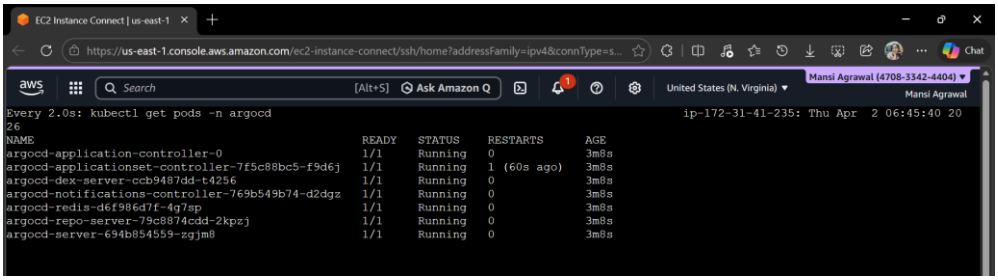


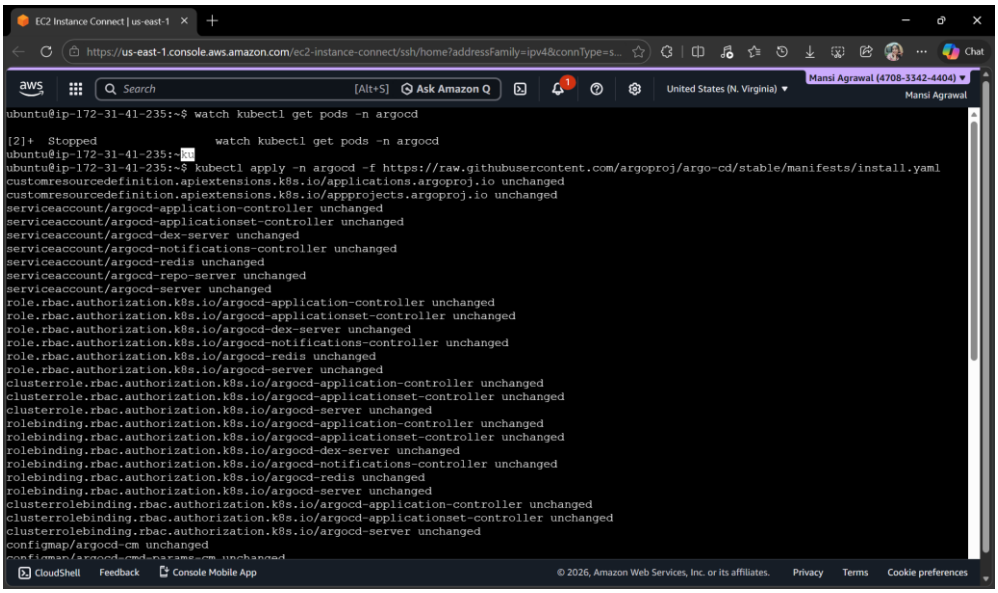


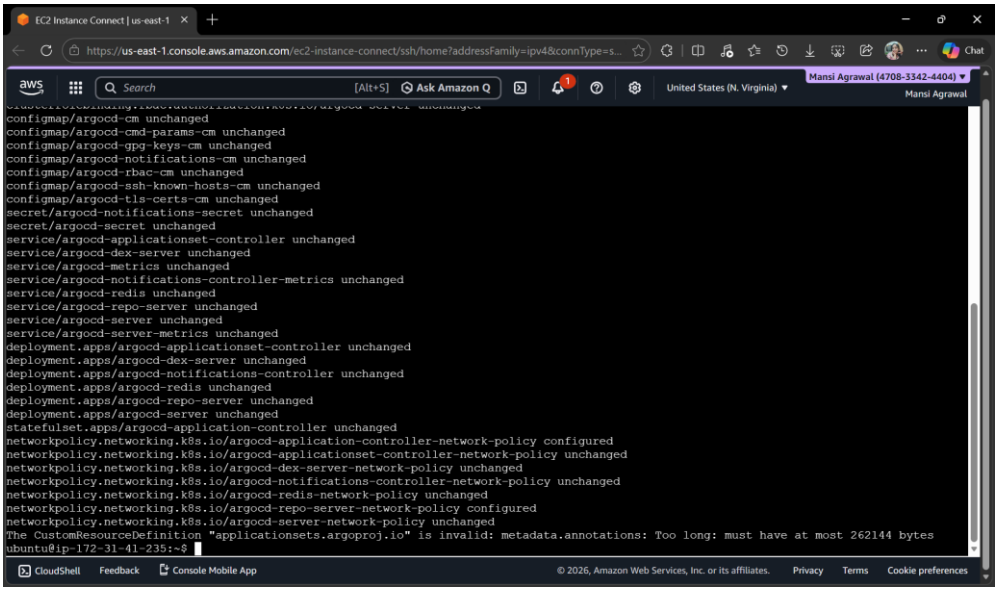
- ArgoCD Setup

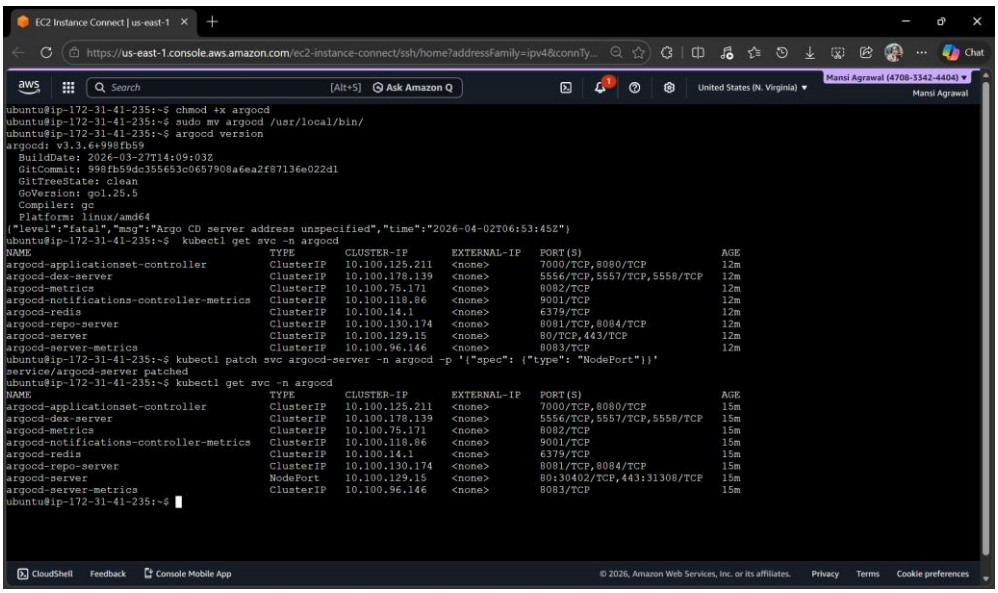


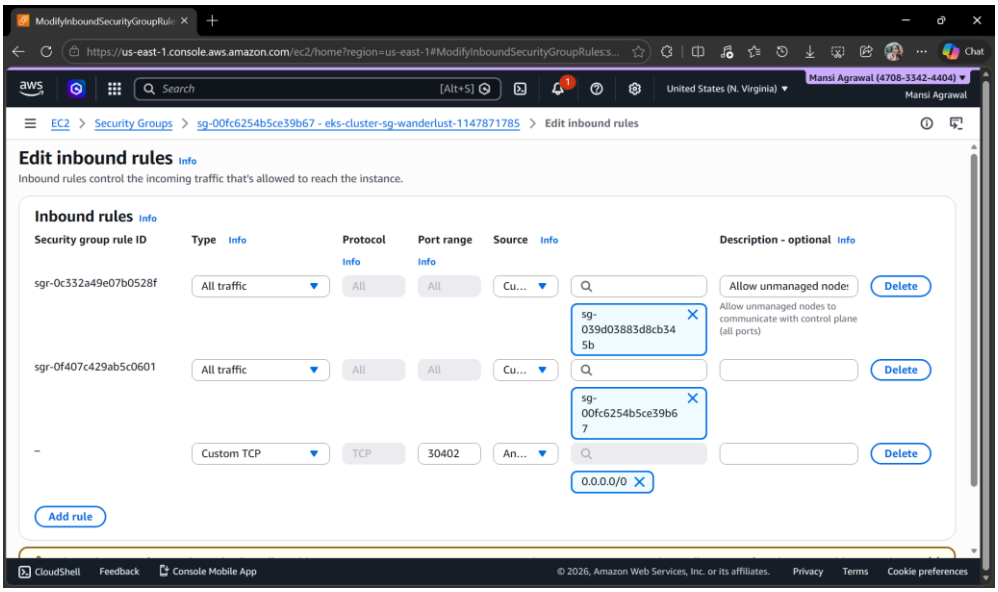

EC2 Instance Connect | us-east-1
clusterrolebinding.rbac.authorization.k8s.io/argocd-server created
configmap/argocd-cm created
configmap/argocd-cmd-params-cm created
configmap/argocd-gpg-keys-cm created
configmap/argocd-notifications-cm created
configmap/argocd-rbac-cm created
configmap/argocd-ssh-known-hosts-cm created
configmap/argocd-tls-certs-cm created
secret/argocd-notifications-secret created
secret/argocd-secret created
service/argocd-applicationset-controller created
service/argocd-dex-server created
service/argocd-metrics created
service/argocd-notifications-controller-metrics created
service/argocd-redis created
service/argocd-repo-server created
service/argocd-server created
service/argocd-server-metrics created
deployment.apps/argocd-applicationset-controller created
deployment.apps/argocd-dex-server created
deployment.apps/argocd-notifications-controller created
deployment.apps/argocd-redis created
deployment.apps/argocd-repo-server created
deployment.apps/argocd-server created
statefulset.apps/argocd-application-controller created
networkpolicy.networking.k8s.io/argocd-application-controller-network-policy created
networkpolicy.networking.k8s.io/argocd-applicationset-controller-network-policy created
networkpolicy.networking.k8s.io/argocd-dex-server-network-policy created
networkpolicy.networking.k8s.io/argocd-notifications-controller-network-policy created
networkpolicy.networking.k8s.io/argocd-redis-network-policy created
networkpolicy.networking.k8s.io/argocd-repo-server-network-policy created
networkpolicy.networking.k8s.io/argocd-server-network-policy created
The CustomResourceDefinition "applicationsets.argoproj.io" is invalid: metadata.annotations: Too long: must have at most 262144 bytes
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

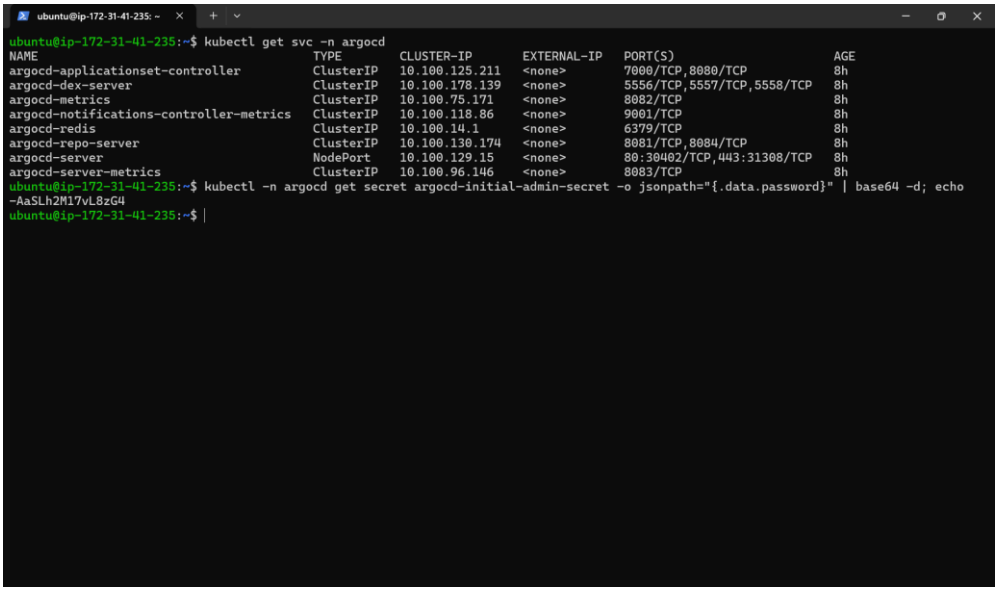

EC2 Instance Connect | us-east-1
Every 2.0s: kubectl get pods -n argocd
ip-172-31-41-235: Thu Apr 2 06:45:40 20
26
NAME READY STATUS RESTARTS AGE
argocd-application-controller-0 1/1 Running 0 3m8s
argocd-applicationset-controller-7f5c88bc5-f9d6j 1/1 Running 1 (60s ago) 3m8s
argocd-dex-server-ccb9487dd-t4256 1/1 Running 0 3m8s
argocd-notifications-controller-769b549b74-d2dgz 1/1 Running 0 3m8s
argocd-redis-d6f986d7f-4q7sp 1/1 Running 0 3m8s
argocd-repo-server-79c8874cdd-2kpzj 1/1 Running 0 3m8s
argocd-server-694b854559-zgjm8 1/1 Running 0 3m8s

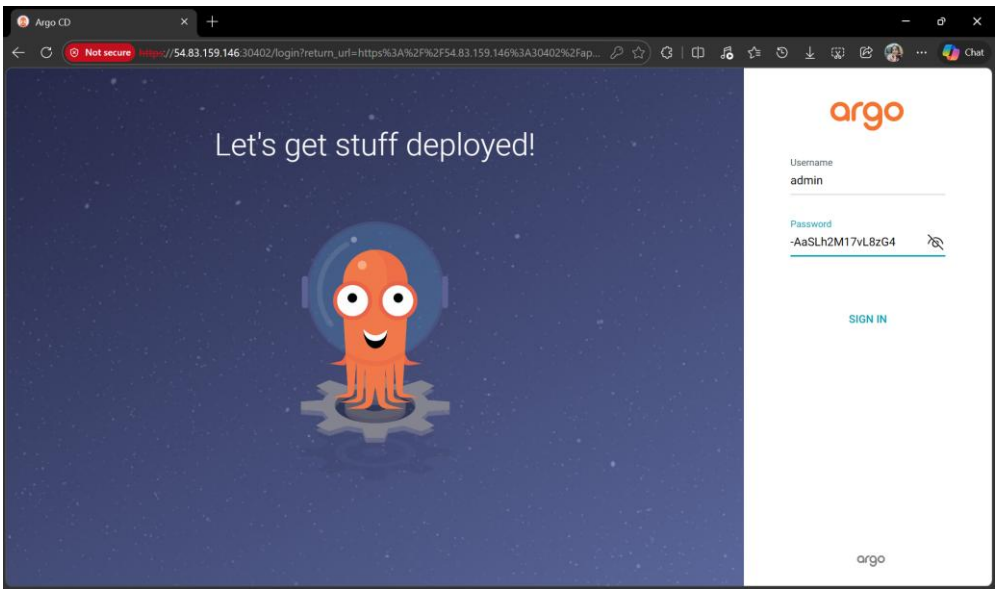

EC2 Instance Connect | us-east-1
ubuntu@ip-172-31-41-235:~$ watch kubectl get pods -n argocd
[2]+ Stopped watch kubectl get pods -n argocd
ubuntu@ip-172-31-41-235:~$
ubuntu@ip-172-31-41-235:~$ kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
customresourcedefinition.apiextensions.k8s.io/applications.argoproj.io unchanged
customresourcedefinition.apiextensions.k8s.io/appprojects.argoproj.io unchanged
serviceaccount/argocd-application-controller unchanged
serviceaccount/argocd-applicationset-controller unchanged
serviceaccount/argocd-dex-server unchanged
serviceaccount/argocd-notifications-controller unchanged
serviceaccount/argocd-redis unchanged
serviceaccount/argocd-repo-server unchanged
serviceaccount/argocd-server unchanged
role.rbac.authorization.k8s.io/argocd-application-controller unchanged
role.rbac.authorization.k8s.io/argocd-applicationset-controller unchanged
role.rbac.authorization.k8s.io/argocd-dex-server unchanged
role.rbac.authorization.k8s.io/argocd-notifications-controller unchanged
role.rbac.authorization.k8s.io/argocd-redis unchanged
role.rbac.authorization.k8s.io/argocd-server unchanged
clusterrole.rbac.authorization.k8s.io/argocd-application-controller unchanged
clusterrole.rbac.authorization.k8s.io/argocd-applicationset-controller unchanged
clusterrole.rbac.authorization.k8s.io/argocd-server unchanged
rolebinding.rbac.authorization.k8s.io/argocd-application-controller unchanged
rolebinding.rbac.authorization.k8s.io/argocd-applicationset-controller unchanged
rolebinding.rbac.authorization.k8s.io/argocd-dex-server unchanged
rolebinding.rbac.authorization.k8s.io/argocd-notifications-controller unchanged
rolebinding.rbac.authorization.k8s.io/argocd-redis unchanged
rolebinding.rbac.authorization.k8s.io/argocd-server unchanged
clusterrolebinding.rbac.authorization.k8s.io/argocd-application-controller unchanged
clusterrolebinding.rbac.authorization.k8s.io/argocd-applicationset-controller unchanged
clusterrolebinding.rbac.authorization.k8s.io/argocd-server unchanged
configmap/argocd-cm unchanged
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

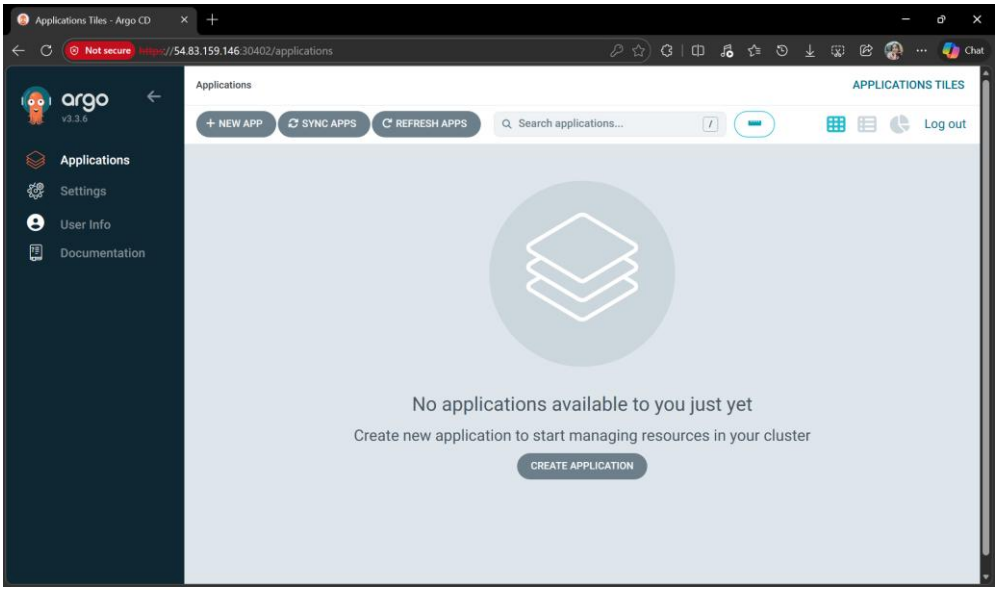

EC2 Instance Connect | us-east-1
configmap/argocd-cm unchanged
configmap/argocd-cmd-params-cm unchanged
configmap/argocd-gpg-keys-cm unchanged
configmap/argocd-notifications-cm unchanged
configmap/argocd-rbac-cm unchanged
configmap/argocd-ssh-known-hosts-cm unchanged
configmap/argocd-tls-certs-cm unchanged
secret/argocd-notifications-secret unchanged
secret/argocd-secret unchanged
service/argocd-applicationset-controller unchanged
service/argocd-dex-server unchanged
service/argocd-metrics unchanged
service/argocd-notifications-controller-metrics unchanged
service/argocd-redis unchanged
service/argocd-repo-server unchanged
service/argocd-server unchanged
service/argocd-server-metrics unchanged
deployment.apps/argocd-applicationset-controller unchanged
deployment.apps/argocd-dex-server unchanged
deployment.apps/argocd-notifications-controller unchanged
deployment.apps/argocd-redis unchanged
deployment.apps/argocd-repo-server unchanged
deployment.apps/argocd-server unchanged
statefulset.apps/argocd-application-controller unchanged
networkpolicy.networking.k8s.io/argocd-application-controller-network-policy configured
networkpolicy.networking.k8s.io/argocd-applicationset-controller-network-policy unchanged
networkpolicy.networking.k8s.io/argocd-dex-server-network-policy unchanged
networkpolicy.networking.k8s.io/argocd-notifications-controller-network-policy unchanged
networkpolicy.networking.k8s.io/argocd-redis-network-policy unchanged
networkpolicy.networking.k8s.io/argocd-repo-server-network-policy configured
networkpolicy.networking.k8s.io/argocd-server-network-policy unchanged
The CustomResourceDefinition "applicationsets.argoproj.io" is invalid: metadata.annotations: Too long: must have at most 262144 bytes
ubuntu@ip-172-31-41-235:~$

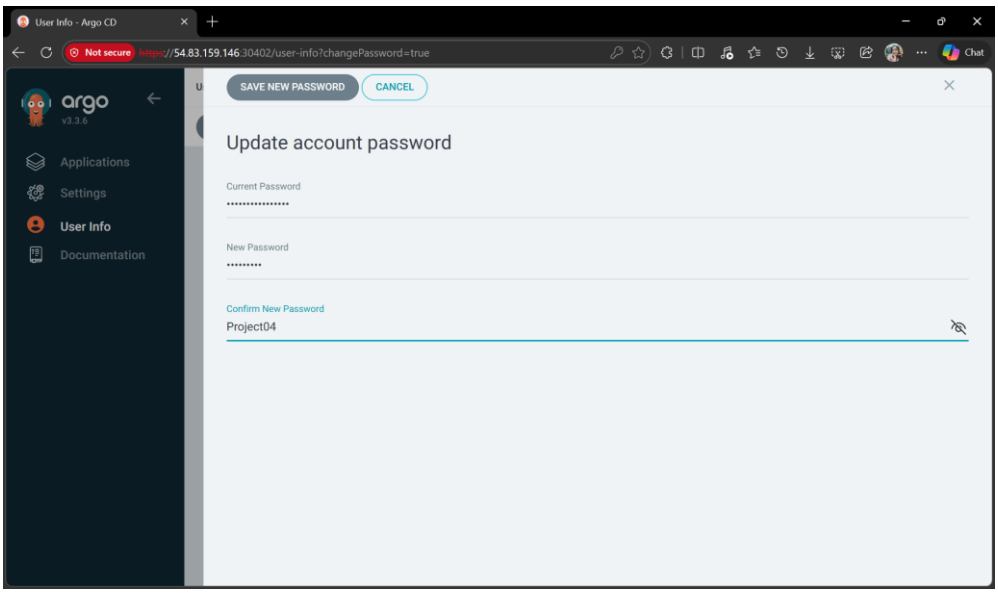

EC2 Instance Connect | us-east-1
ubuntu@ip-172-31-41-235:~$ chmod +x argocd
ubuntu@ip-172-31-41-235:~$ sudo mv argocd /usr/local/bin/
ubuntu@ip-172-31-41-235:~$ argocd version
argocd: v3.3.6+998fb59
BuildDate: 2026-03-27T14:09:03Z
GitCommit: 998fb59dc355653c0657908a6ea2f87136e022d1
GitTreeState: clean
GoVersion: go1.25.5
Compiler: gc
Platform: linux/amd64
{"level":"fatal","msg":"Argo CD server address unspecified","time":"2026-04-02T06:53:45Z"}
ubuntu@ip-172-31-41-235:~$ kubectl get svc -n argocd
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
argocd-applicationset-controller ClusterIP 10.100.125.211 <none> 7000/TCP,8080/TCP 12m
argocd-dex-server ClusterIP 10.100.178.139 <none> 5556/TCP,5557/TCP,5558/TCP 12m
argocd-metrics ClusterIP 10.100.75.171 <none> 8082/TCP 12m
argocd-notifications-controller-metrics ClusterIP 10.100.118.86 <none> 9001/TCP 12m
argocd-redis ClusterIP 10.100.14.1 <none> 6379/TCP 12m
argocd-repo-server ClusterIP 10.100.130.174 <none> 8081/TCP,8084/TCP 12m
argocd-server ClusterIP 10.100.129.15 <none> 80/TCP,443/TCP 12m
argocd-server-metrics ClusterIP 10.100.96.146 <none> 8083/TCP 12m
ubuntu@ip-172-31-41-235:~$ kubectl patch svc argocd-server -n argocd -p '{"spec": {"type": "NodePort"}}'
service/argocd-server patched
ubuntu@ip-172-31-41-235:~$ kubectl get svc -n argocd
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
argocd-applicationset-controller ClusterIP 10.100.125.211 <none> 7000/TCP,8080/TCP 15m
argocd-dex-server ClusterIP 10.100.178.139 <none> 5556/TCP,5557/TCP,5558/TCP 15m
argocd-metrics ClusterIP 10.100.75.171 <none> 8082/TCP 15m
argocd-notifications-controller-metrics ClusterIP 10.100.118.86 <none> 9001/TCP 15m
argocd-redis ClusterIP 10.100.14.1 <none> 6379/TCP 15m
argocd-repo-server ClusterIP 10.100.130.174 <none> 8081/TCP,8084/TCP 15m
argocd-server NodePort 10.100.129.15 <none> 80:30402/TCP,443:31308/TCP 15m
argocd-server-metrics ClusterIP 10.100.96.146 <none> 8083/TCP 15m
ubuntu@ip-172-31-41-235:~$

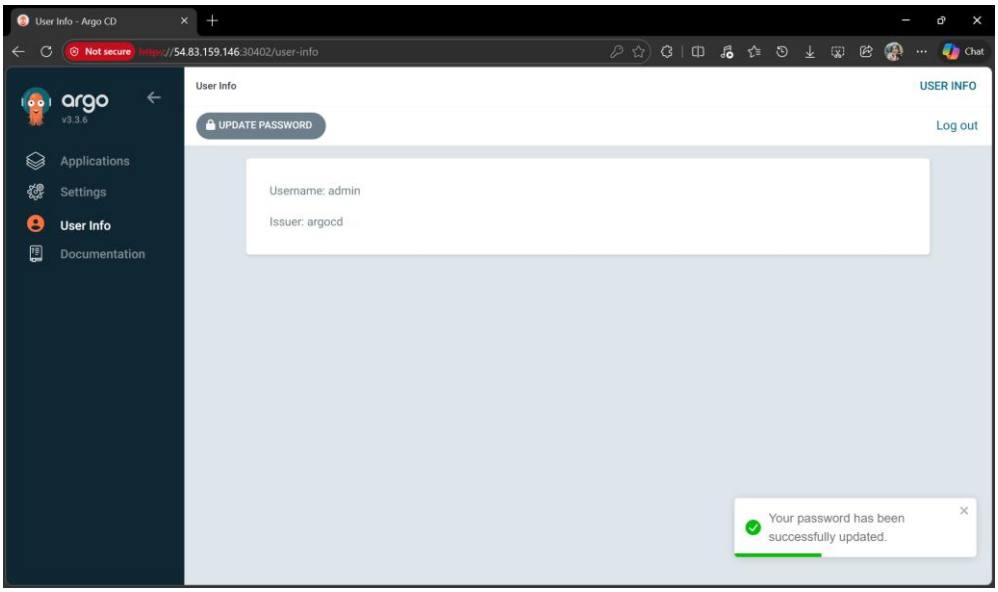

ModifyInboundSecurityGroupRules
EC2 > Security Groups > sg-00fc6254b5ce39b67 - eks-cluster-sg-wanderlust-1147871785 > Edit inbound rules
Edit inbound rules Info
Inbound rules control the incoming traffic that's allowed to reach the instance.
Inbound rules Info
Security group rule ID
Type Info
Protocol Info
Port range Info
Source Info
Description - optional Info
sgr-0c332a49e07b0528f
All traffic
All
All
Cu...
sg-039d03883d8cb345b
Allow unmanaged node
Allow unmanaged nodes to communicate with control plane (all ports)
Delete
sgr-0f407c429ab5c0601
All traffic
All
All
Cu...
sg-00fc6254b5ce39b67
Delete
-
Custom TCP
TCP
30402
An...
0.0.0.0/0
Delete
Add rule


ubuntu@ip-172-31-41-235: ~
ubuntu@ip-172-31-41-235:~$ kubectl get svc -n argocd
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
argocd-applicationset-controller ClusterIP 10.100.125.211 <none> 7000/TCP,8080/TCP 8h
argocd-dex-server ClusterIP 10.100.178.139 <none> 5556/TCP,5557/TCP,5558/TCP 8h
argocd-metrics ClusterIP 10.100.75.171 <none> 8082/TCP 8h
argocd-notifications-controller-metrics ClusterIP 10.100.118.86 <none> 9001/TCP 8h
argocd-redis ClusterIP 10.100.14.1 <none> 6379/TCP 8h
argocd-repo-server ClusterIP 10.100.130.174 <none> 8081/TCP,8084/TCP 8h
argocd-server NodePort 10.100.129.15 <none> 80:30402/TCP,443:31308/TCP 8h
argocd-server-metrics ClusterIP 10.100.96.146 <none> 8083/TCP 8h
ubuntu@ip-172-31-41-235:~$ kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d; echo
-AaSLh2M17vL8zG4
ubuntu@ip-172-31-41-235:~$

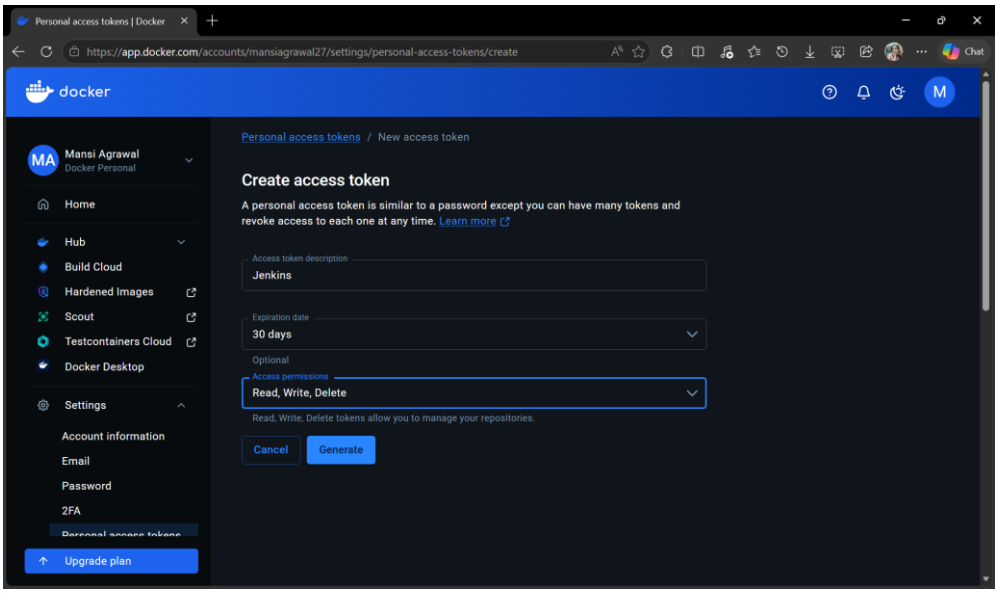

Argo CD
Not secure https://54.83.159.146:30402/login?return_url=https%3A%2F%2F54.83.159.146%3A30402%2Fap...
Chat
Let's get stuff deployed!
argo
Username
admin
Password
-AaSLh2M17vL8zG4
SIGN IN
argo

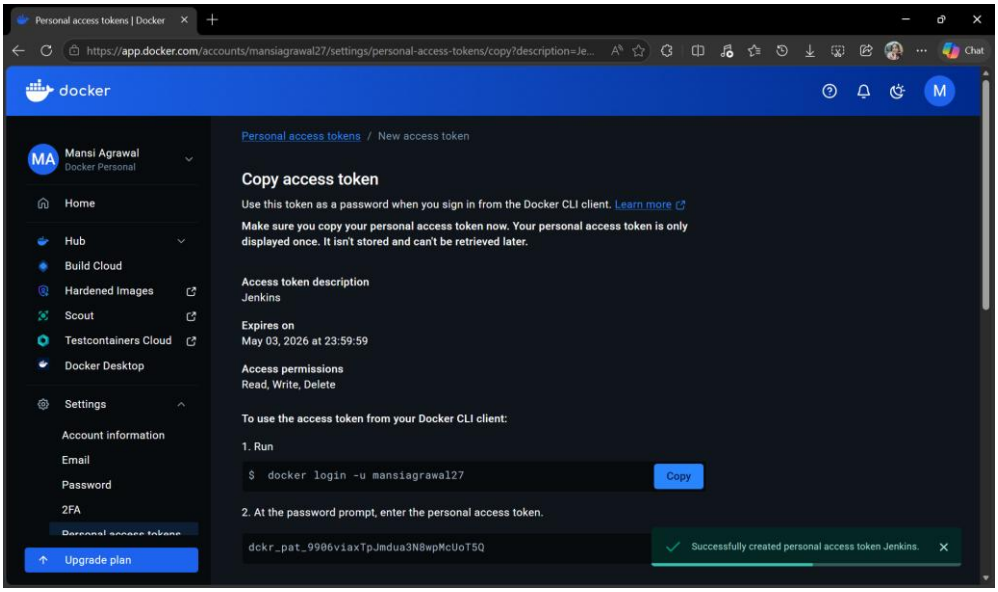

Applications Tiles - Argo CD
Not secure https://54.83.159.146:30402/applications
Chat
argo
v3.3.6
Applications
Settings
User Info
Documentation
Applications
APPLICATIONS TILES
+ NEW APP
SYNC APPS
REFRESH APPS
Search applications...
Log out
No applications available to you just yet
Create new application to start managing resources in your cluster
CREATE APPLICATION

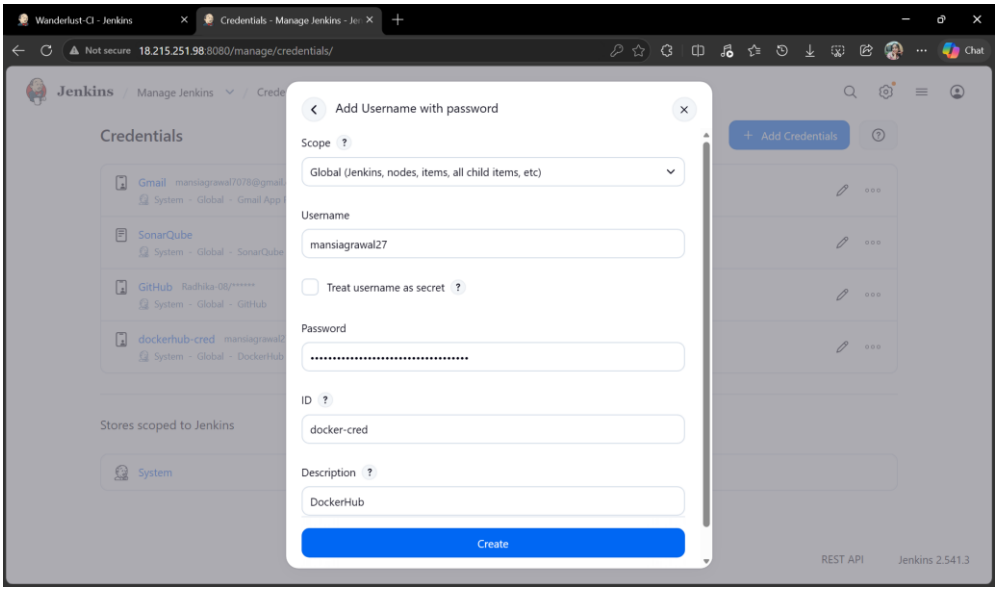

User Info - Argo CD
Not secure https://54.83.159.146:30402/user-info?changePassword=true
Chat
argo
v3.3.6
Applications
Settings
User Info
Documentation
SAVE NEW PASSWORD
CANCEL
Update account password
Current Password
New Password
Confirm New Password
Project04


User Info - Argo CD
Not secure https://54.83.159.146:30402/user-info
Chat
argo
v3.3.6
Applications
Settings
User Info
Documentation
User Info
USER INFO
UPDATE PASSWORD
Log out
Username: admin
Issuer: argocd
Your password has been successfully updated.

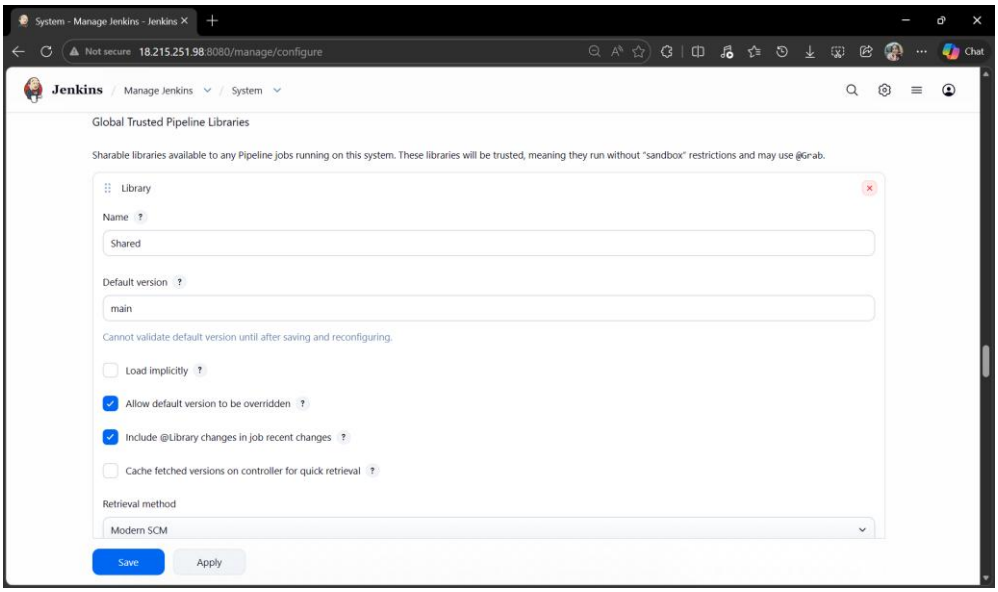
- Docker Setup

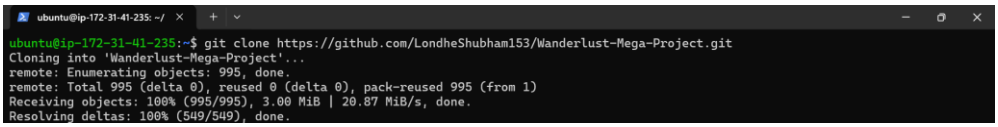






- Shared Library & Frontend and Backend Setup

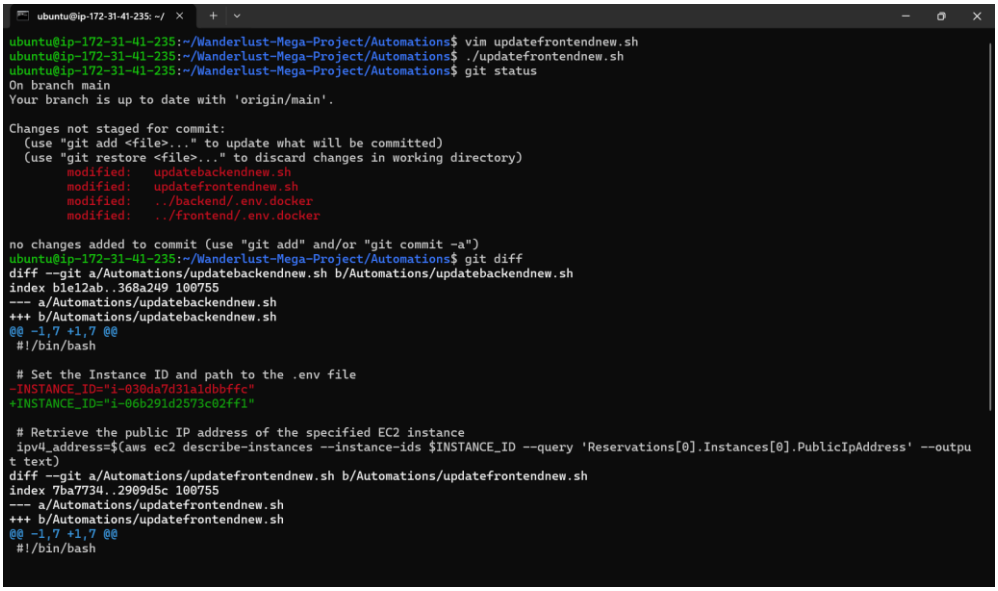


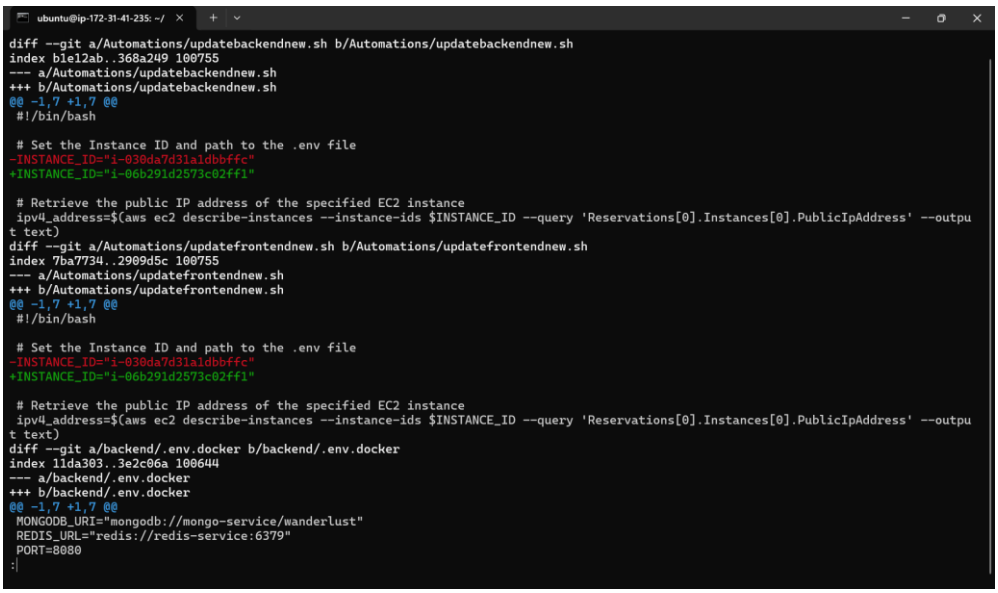


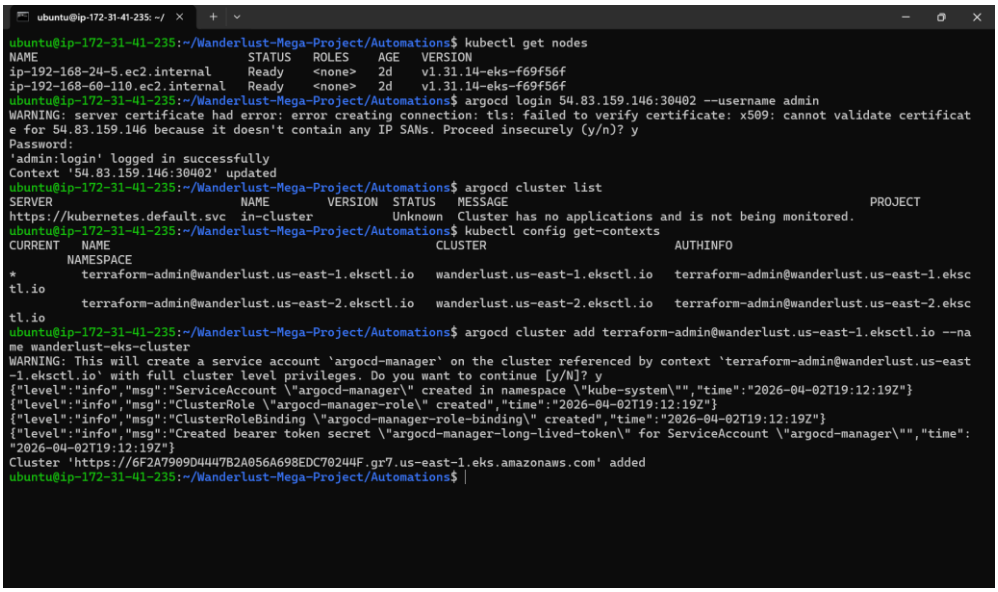
```
ubuntu@ip-172-31-41-235: ~/
#!/bin/bash

# Set the Instance ID and path to the .env file
INSTANCE_ID="i-06b291d2573c02ff1"

# Retrieve the public IP address of the specified EC2 instance
ipv4_address=$(aws ec2 describe-instances --instance-ids $INSTANCE_ID --query 'Reservations[0].Instances[0].PublicIpAddress' --output
 text)

# Path to the .env file
file_to_find="../backend/.env.docker"

# Check the current FRONTEND_URL in the .env file
current_url=$(sed -n "4p" $file_to_find)

# Update the .env file if the IP address has changed
if [[ "$current_url" != "FRONTEND_URL=\"http://${ipv4_address}:5173\"" ]]; then
    if [ -f $file_to_find ]; then
        sed -i -e "s|FRONTEND_URL.*|FRONTEND_URL=\"http://${ipv4_address}:5173\"|g" $file_to_find
    else
        echo "ERROR: File not found."
    fi
fi
~
~
~
~
~
~
~
~
~
~
~
~
~
-- INSERT --                                                                    4,34          All
```

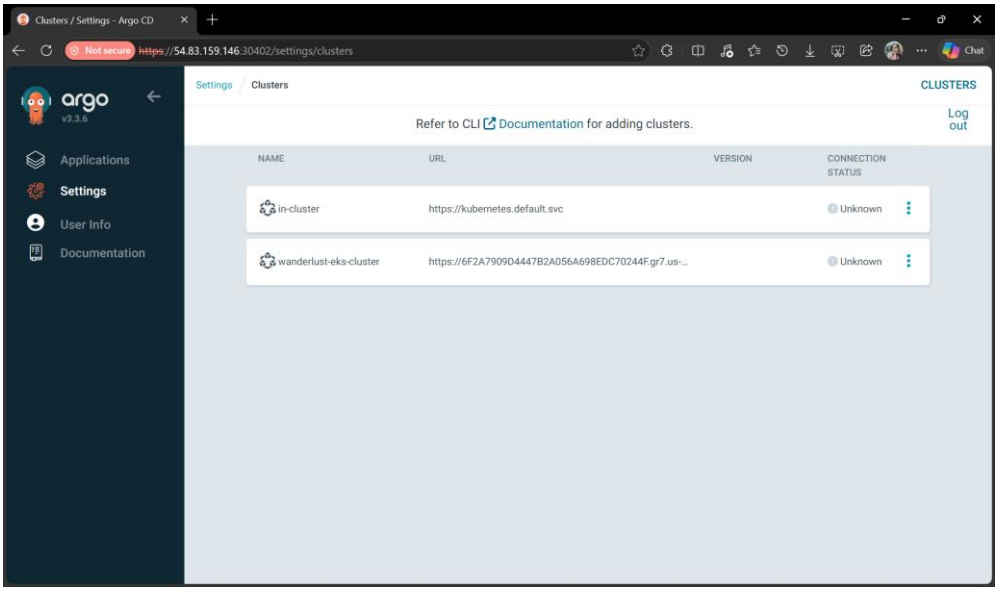
```
ubuntu@ip-172-31-41-235: ~/
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project$ cd Automations
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$ ls
updatebackendnew.sh  updatefrontendnew.sh
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$ vim updatebackendnew.sh
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$ vim updatebackendnew.sh
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$ ./updatebackendnew.sh
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$ vim updatebackendnew.sh
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   updatebackendnew.sh
        modified:   ../backend/.env.docker

no changes added to commit (use "git add" and/or "git commit -a")
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$ git diff
diff --git a/Automations/updatebackendnew.sh b/Automations/updatebackendnew.sh
index b1e12ab..368a249 100755
--- a/Automations/updatebackendnew.sh
+++ b/Automations/updatebackendnew.sh
@@ -1,7 +1,7 @@
 #!/bin/bash

 # Set the Instance ID and path to the .env file
-INSTANCE_ID="i-030da7d31a1dbbffc"
+INSTANCE_ID="i-06b291d2573c02ff1"

 # Retrieve the public IP address of the specified EC2 instance
 ipv4_address=$(aws ec2 describe-instances --instance-ids $INSTANCE_ID --query 'Reservations[0].Instances[0].PublicIpAddress' --output text)
diff --git a/backend/.env.docker b/backend/.env.docker
index 11da303..3e2c06a 100644
--- a/backend/.env.docker
+++ b/backend/.env.docker
@@ -1,7 +1,7 @@
 MONGODB_URI="mongodb://mongo-service/wanderlust"
 REDIS_URL="redis://redis-service:6379"
 PORT=8080
-FRONTEND_URL="http://18.224.72.54:5173"
+FRONTEND_URL="http://18.215.251.98:5173"
 ACCESS_COOKIE_MAXAGE=120000
 ACCESS_TOKEN_EXPIRES_IN='120s'
 REFRESH_COOKIE_MAXAGE=120000
```

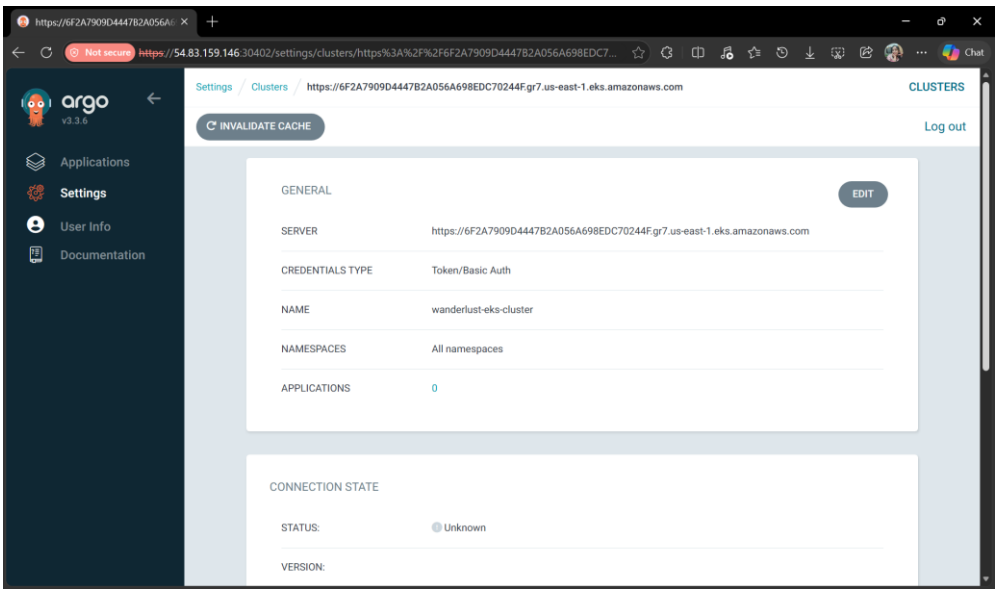
```
ubuntu@ip-172-31-41-235: ~/
#!/bin/bash

# Set the Instance ID and path to the .env file
INSTANCE_ID="i-06b291d2573c02ff1"

# Retrieve the public IP address of the specified EC2 instance
ipv4_address=$(aws ec2 describe-instances --instance-ids $INSTANCE_ID --query 'Reservations[0].Instances[0].PublicIpAddress' --output
 text)

# Path to the .env file
file_to_find="../frontend/.env.docker"

# Check the current VITE_API_PATH in the .env file
current_url=$(cat $file_to_find)

# Update the .env file if the IP address has changed
if [[ "$current_url" != "VITE_API_PATH=\"http://${ipv4_address}:31100\"" ]]; then
    if [ -f $file_to_find ]; then
        sed -i -e "s|VITE_API_PATH.*|VITE_API_PATH=\"http://${ipv4_address}:31100\"|g" $file_to_find
    else
        echo "ERROR: File not found."
    fi
fi
~
~
~
~
~
~
~
~
~
~
~
~
~
~
:wq
```

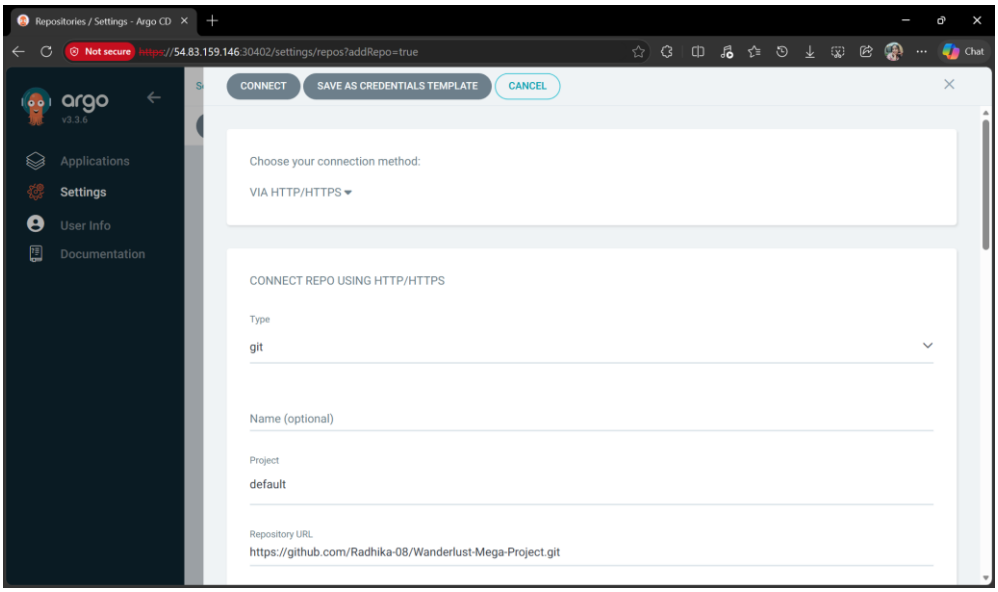

ubuntu@ip-172-31-41-235: ~/
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$ vim updatefrontendnew.sh
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$ ./updatefrontendnew.sh
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$ git status
On branch main
Your branch is up to date with 'origin/main'.
Changes not staged for commit:
(use "git add <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directory)
modified: updatebackendnew.sh
modified: updatefrontendnew.sh
modified: ../backend/.env.docker
modified: ../frontend/.env.docker
no changes added to commit (use "git add" and/or "git commit -a")
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$ git diff
diff --git a/Automations/updatebackendnew.sh b/Automations/updatebackendnew.sh
index b1e12ab..368a249 100755
--- a/Automations/updatebackendnew.sh
+++ b/Automations/updatebackendnew.sh
@@ -1,7 +1,7 @@
#!/bin/bash
Set the Instance ID and path to the .env file
-INSTANCE_ID="i-030da7d31a1dbbffc"
+INSTANCE_ID="i-06b291d2573c02ff1"
# Retrieve the public IP address of the specified EC2 instance
ipv4_address=$(aws ec2 describe-instances --instance-ids $INSTANCE_ID --query 'Reservations[0].Instances[0].PublicIpAddress' --outpu
t text)
diff --git a/Automations/updatefrontendnew.sh b/Automations/updatefrontendnew.sh
index 7ba7734..2909d5c 100755
--- a/Automations/updatefrontendnew.sh
+++ b/Automations/updatefrontendnew.sh
@@ -1,7 +1,7 @@
#!/bin/bash

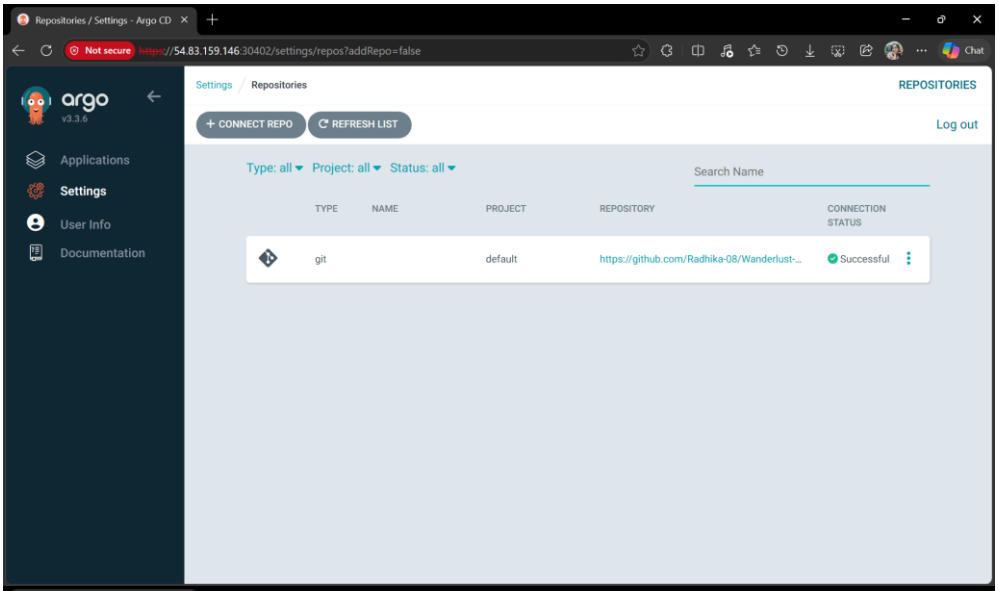

ubuntu@ip-172-31-41-235: ~/
diff --git a/Automations/updatebackendnew.sh b/Automations/updatebackendnew.sh
index b1e12ab..368a249 100755
--- a/Automations/updatebackendnew.sh
+++ b/Automations/updatebackendnew.sh
@@ -1,7 +1,7 @@
#!/bin/bash
Set the Instance ID and path to the .env file
-INSTANCE_ID="i-030da7d31a1dbbffc"
+INSTANCE_ID="i-06b291d2573c02ff1"
# Retrieve the public IP address of the specified EC2 instance
ipv4_address=$(aws ec2 describe-instances --instance-ids $INSTANCE_ID --query 'Reservations[0].Instances[0].PublicIpAddress' --outpu
t text)
diff --git a/Automations/updatefrontendnew.sh b/Automations/updatefrontendnew.sh
index 7ba7734..2909d5c 100755
--- a/Automations/updatefrontendnew.sh
+++ b/Automations/updatefrontendnew.sh
@@ -1,7 +1,7 @@
#!/bin/bash
Set the Instance ID and path to the .env file
-INSTANCE_ID="i-030da7d31a1dbbffc"
+INSTANCE_ID="i-06b291d2573c02ff1"
# Retrieve the public IP address of the specified EC2 instance
ipv4_address=$(aws ec2 describe-instances --instance-ids $INSTANCE_ID --query 'Reservations[0].Instances[0].PublicIpAddress' --outpu
t text)
diff --git a/backend/.env.docker b/backend/.env.docker
index 11da303..3e2c06a 100644
--- a/backend/.env.docker
+++ b/backend/.env.docker
@@ -1,7 +1,7 @@
MONGODB_URI="mongodb://mongo-service/wanderlust"
REDIS_URL="redis://redis-service:6379"
PORT=8080
:

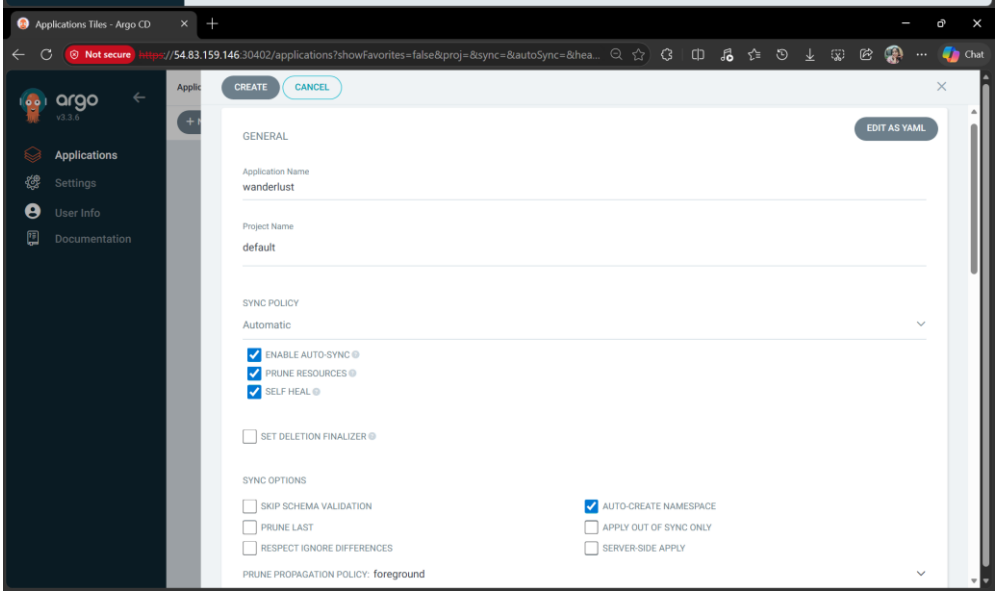

ubuntu@ip-172-31-41-235: ~/
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$ kubectl get nodes
NAME STATUS ROLES AGE VERSION
ip-192-168-24-5.ec2.internal Ready <none> 2d v1.31.14-eks-f69f56f
ip-192-168-60-110.ec2.internal Ready <none> 2d v1.31.14-eks-f69f56f
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$ argocd login 54.83.159.146:30402 --username admin
WARNING: server certificate had error: error creating connection: tls: failed to verify certificate: x509: cannot validate certificat
e for 54.83.159.146 because it doesn't contain any IP SANs. Proceed insecurely (y/n)? y
Password:
'admin:login' logged in successfully
Context '54.83.159.146:30402' updated
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$ argocd cluster list
SERVER NAME VERSION STATUS MESSAGE PROJECT
https://kubernetes.default.svc in-cluster Unknown Cluster has no applications and is not being monitored.
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$ kubectl config get-contexts
CURRENT NAME CLUSTER AUTHINFO
NAMESPACE
* terraform-admin@wanderlust.us-east-1.eksctl.io wanderlust.us-east-1.eksctl.io terraform-admin@wanderlust.us-east-1.eksc
tl.io
terraform-admin@wanderlust.us-east-2.eksctl.io wanderlust.us-east-2.eksctl.io terraform-admin@wanderlust.us-east-2.eksc
tl.io
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$ argocd cluster add terraform-admin@wanderlust.us-east-1.eksctl.io --na
me wanderlust-eks-cluster
WARNING: This will create a service account `argocd-manager` on the cluster referenced by context `terraform-admin@wanderlust.us-east
-1.eksctl.io` with full cluster level privileges. Do you want to continue [y/N]? y
{"level":"info","msg":"ServiceAccount \"argocd-manager\" created in namespace \"kube-system\"","time":"2026-04-02T19:12:19Z"}
{"level":"info","msg":"ClusterRole \"argocd-manager-role\" created","time":"2026-04-02T19:12:19Z"}
{"level":"info","msg":"ClusterRoleBinding \"argocd-manager-role-binding\" created","time":"2026-04-02T19:12:19Z"}
{"level":"info","msg":"Created bearer token secret \"argocd-manager-long-lived-token\" for ServiceAccount \"argocd-manager\"","time":
"2026-04-02T19:12:19Z"}
Cluster 'https://6F2A7909D4447B2A056A698EDC702A4F.gr7.us-east-1.eks.amazonaws.com' added
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$

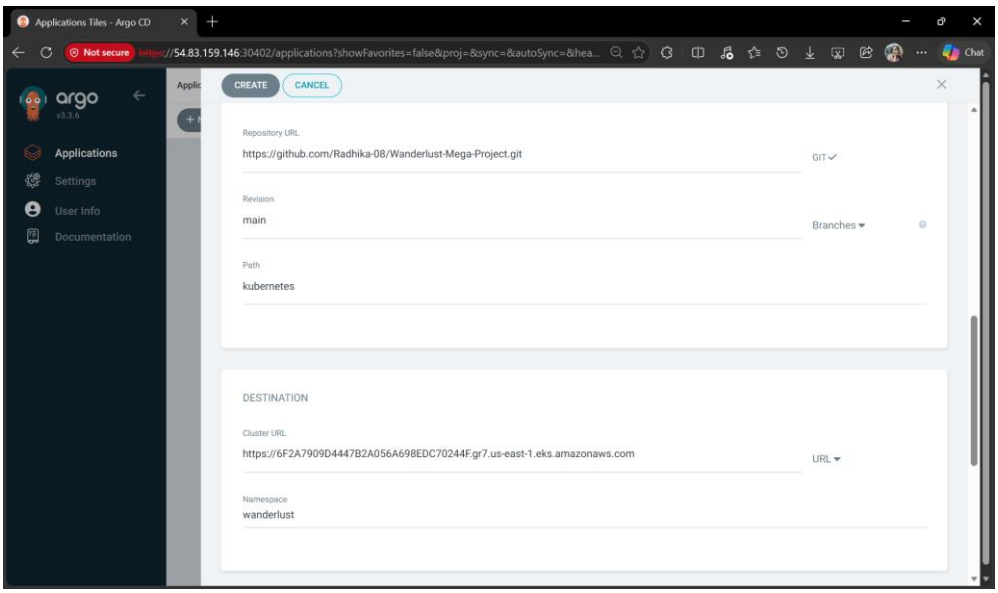

Clusters / Settings - Argo CD
Not secure https://54.83.159.146:30402/settings/clusters
Chat
argo
v3.3.6
Applications
Settings
User Info
Documentation
Settings / Clusters
CLUSTERS
Refer to CLI Documentation for adding clusters.
Log out
NAME
URL
VERSION
CONNECTION STATUS
in-cluster
https://kubernetes.default.svc
Unknown
wanderlust-eks-cluster
https://6F2A7909D4447B2A056A698EDC70244F.gr7.us-...
Unknown

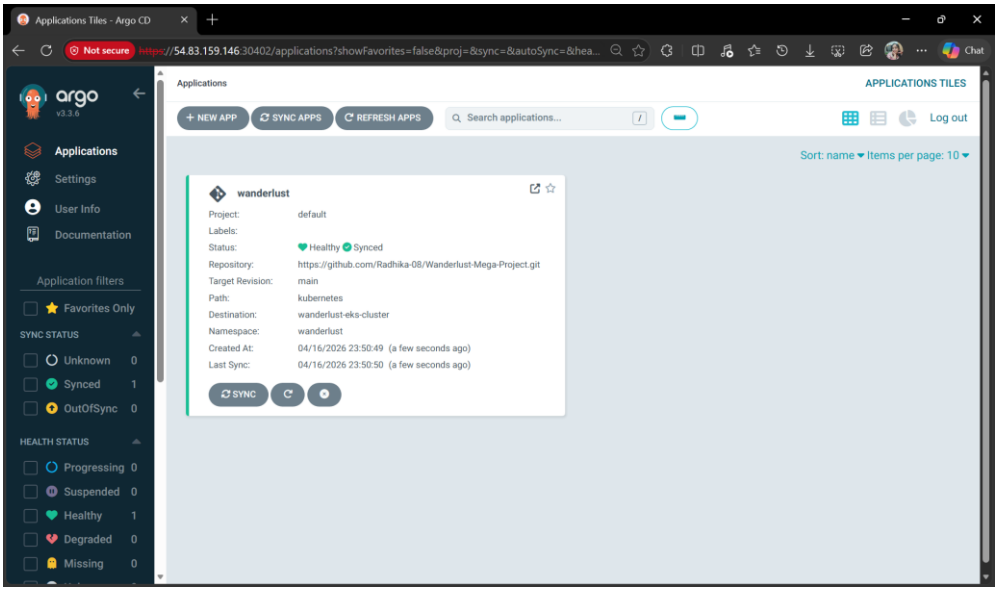

Not secure https://54.83.159.146:30402/settings/clusters/https%3A%2F%2F6F2A7909D4447B2A056A698EDC7...
Chat
argo
v3.3.6
Applications
Settings
User Info
Documentation
Settings / Clusters / https://6F2A7909D4447B2A056A698EDC70244F.gr7.us-east-1.eks.amazonaws.com
CLUSTERS
INVALIDATE CACHE
Log out
GENERAL
EDIT
SERVER
https://6F2A7909D4447B2A056A698EDC70244F.gr7.us-east-1.eks.amazonaws.com
CREDENTIALS TYPE
Token/Basic Auth
NAME
wanderlust-eks-cluster
NAMESPACES
All namespaces
APPLICATIONS
0
CONNECTION STATE
STATUS:
Unknown
VERSION:

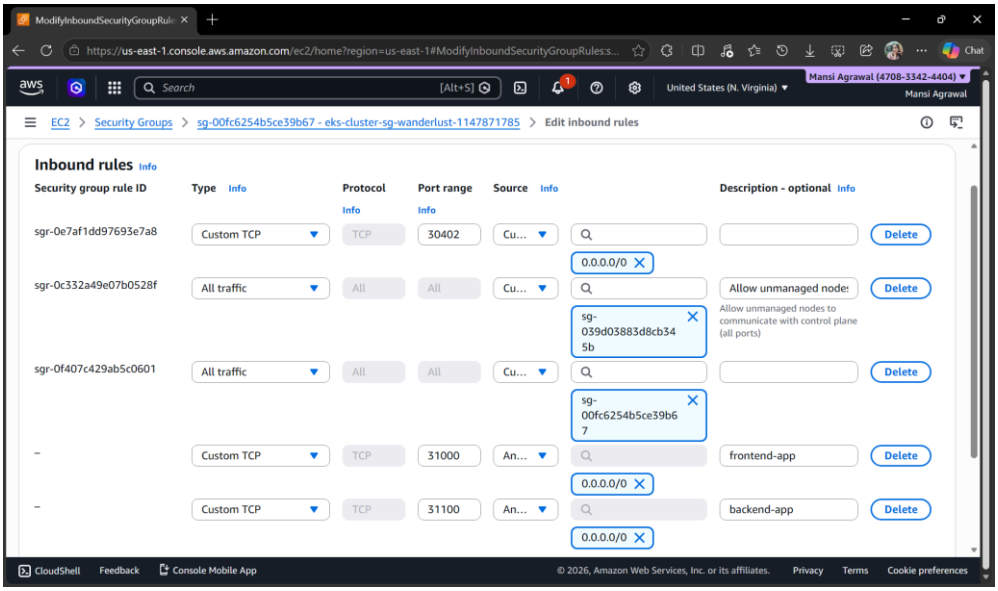

Repositories / Settings - Argo CD
Not secure https://54.83.159.146:30402/settings/repos?addRepo=true
Chat
argo
v3.3.6
Applications
Settings
User Info
Documentation
CONNECT
SAVE AS CREDENTIALS TEMPLATE
CANCEL
Choose your connection method:
VIA HTTP/HTTPS
CONNECT REPO USING HTTP/HTTPS
Type
git
Name (optional)
Project
default
Repository URL
https://github.com/Radhika-08/Wanderlust-Mega-Project.git


Repositories / Settings - Argo CD
Not secure https://54.83.159.146:30402/settings/repos?addRepo=false
argo
v3.3.6
Applications
Settings
User Info
Documentation
Settings / Repositories
REPOSITORIES
+ CONNECT REPO
REFRESH LIST
Log out
Type: all
Project: all
Status: all
Search Name
TYPE
NAME
PROJECT
REPOSITORY
CONNECTION STATUS
git
default
https://github.com/Radhika-08/Wanderlust-...
Successful

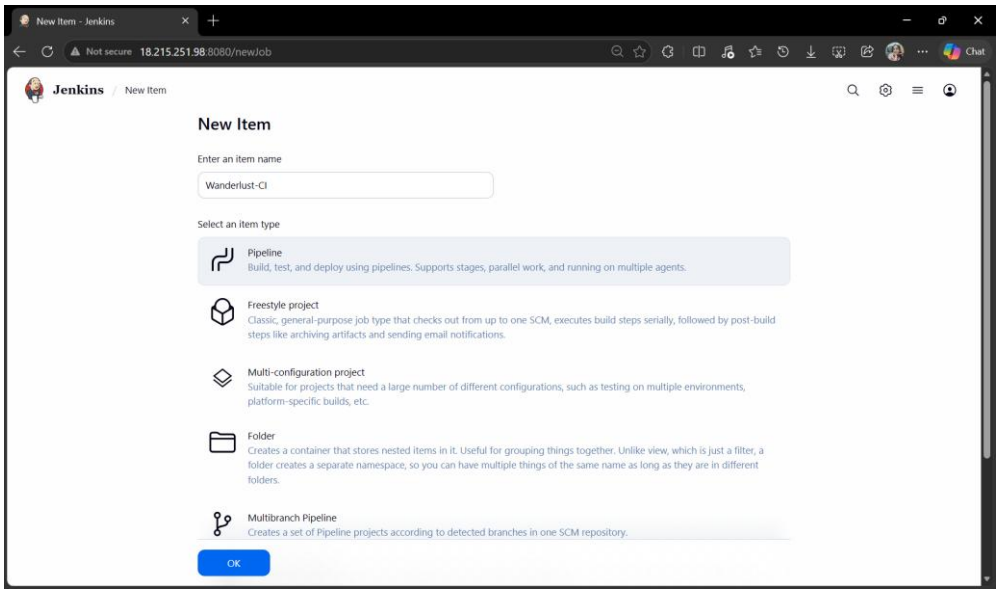

Applications Tiles - Argo CD
Not secure https://54.83.159.146:30402/applications?showFavorites=false&proj=&sync=&autoSync=&hea...
CREATE
CANCEL
EDIT AS YAML
GENERAL
Application Name
wanderlust
Project Name
default
SYNC POLICY
Automatic
ENABLE AUTO-SYNC
PRUNE RESOURCES
SELF HEAL
SET DELETION FINALIZER
SYNC OPTIONS
SKIP SCHEMA VALIDATION
AUTO-CREATE NAMESPACE
PRUNE LAST
APPLY OUT OF SYNC ONLY
RESPECT IGNORE DIFFERENCES
SERVER-SIDE APPLY
PRUNE PROPAGATION POLICY: foreground

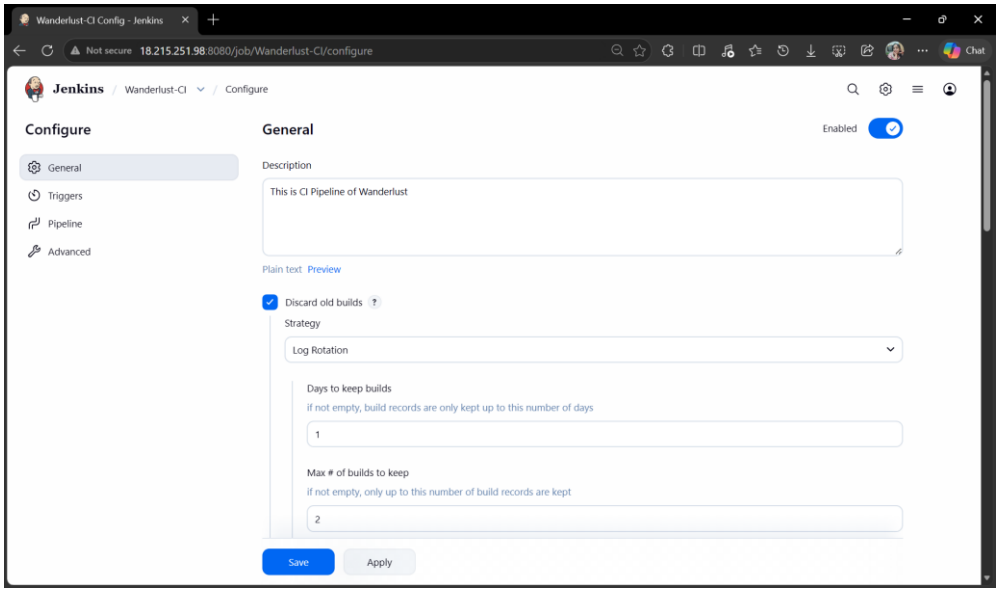

Applications Tiles - Argo CD
Not secure https://54.83.159.146:30402/applications?showFavorites=false&proj=&sync=&autoSync=&hea...
CREATE
CANCEL
Repository URL
https://github.com/Radhika-08/Wanderlust-Mega-Project.git
GIT
Revision
main
Branches
Path
kubernetes
DESTINATION
Cluster URL
https://6F2A7909D4447B2A056A698EDC70244F.gr7.us-east-1.eks.amazonaws.com
URL
Namespace
wanderlust

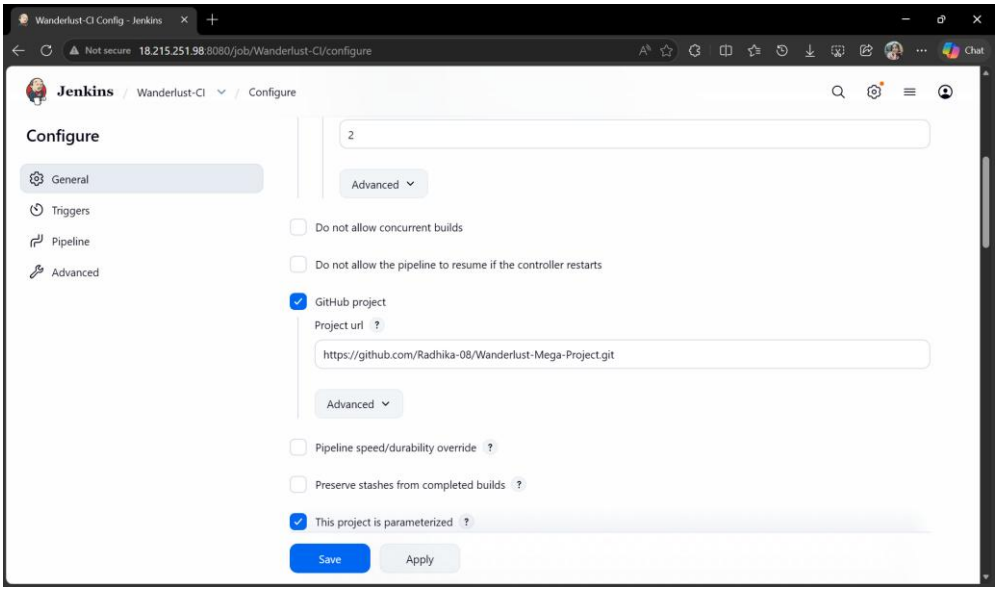

Applications Tiles - Argo CD
Not secure
https://54.83.159.146:30402/applications?showFavorites=false&proj=&sync=&autoSync=&hea...
Chat
argo
v3.3.6
Applications
Settings
User Info
Documentation
Application filters
Favorites Only
SYNC STATUS
Unknown 0
Synced 1
OutOfSync 0
HEALTH STATUS
Progressing 0
Suspended 0
Healthy 1
Degraded 0
Missing 0
Applications
APPLICATIONS TILES
+ NEW APP
SYNC APPS
REFRESH APPS
Search applications...
Log out
Sort: name
Items per page: 10
wanderlust
Project: default
Labels:
Status: Healthy Synced
Repository: https://github.com/Radhika-08/Wanderlust-Mega-Project.git
Target Revision: main
Path: kubernetes
Destination: wanderlust-eks-cluster
Namespace: wanderlust
Created At: 04/16/2026 23:50:49 (a few seconds ago)
Last Sync: 04/16/2026 23:50:50 (a few seconds ago)
SYNC

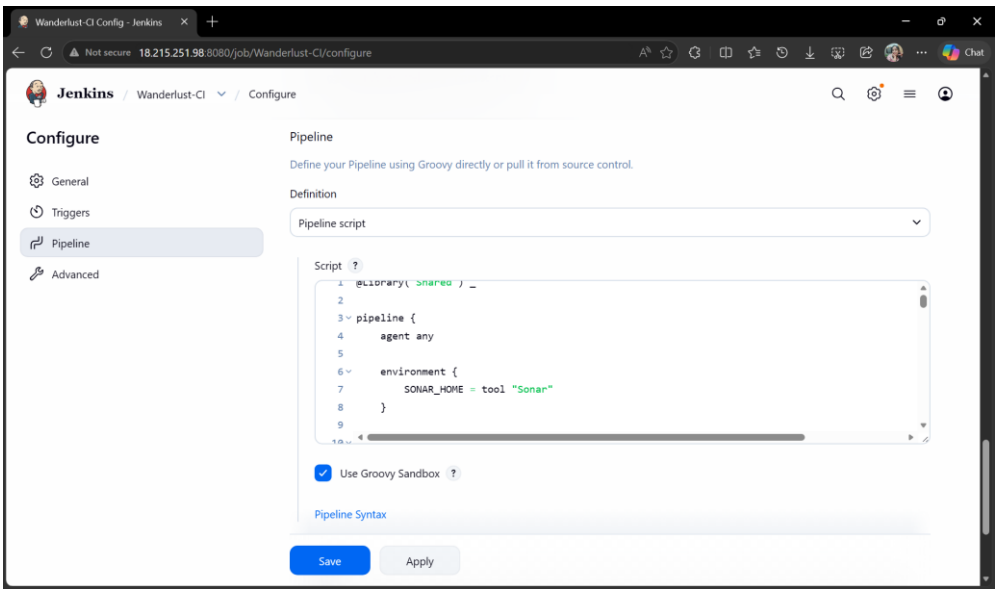

ModifyInboundSecurityGroupRule
https://us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#ModifyInboundSecurityGroupRules:s...
Chat
aws
Search
[Alt+S]
United States (N. Virginia)
Mansi Agrawal (4708-3342-4404)
Mansi Agrawal
EC2 > Security Groups > sg-00fc6254b5ce39b67 - eks-cluster-sg-wanderlust-1147871785 > Edit inbound rules
Inbound rules Info
Security group rule ID
Type Info
Protocol Info
Port range Info
Source Info
Description - optional Info
sgr-0e7af1dd97693e7a8
Custom TCP
TCP
30402
Cu...
0.0.0.0/0
Delete
sgr-0c332a49e07b0528f
All traffic
All
All
Cu...
sg-039d03883d8cb345b
Allow unmanaged node
Allow unmanaged nodes to communicate with control plane (all ports)
Delete
sgr-0f407c429ab5c0601
All traffic
All
All
Cu...
sg-00fc6254b5ce39b67
Delete
-
Custom TCP
TCP
31000
An...
0.0.0.0/0
frontend-app
Delete
-
Custom TCP
TCP
31100
An...
0.0.0.0/0
backend-app
Delete
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

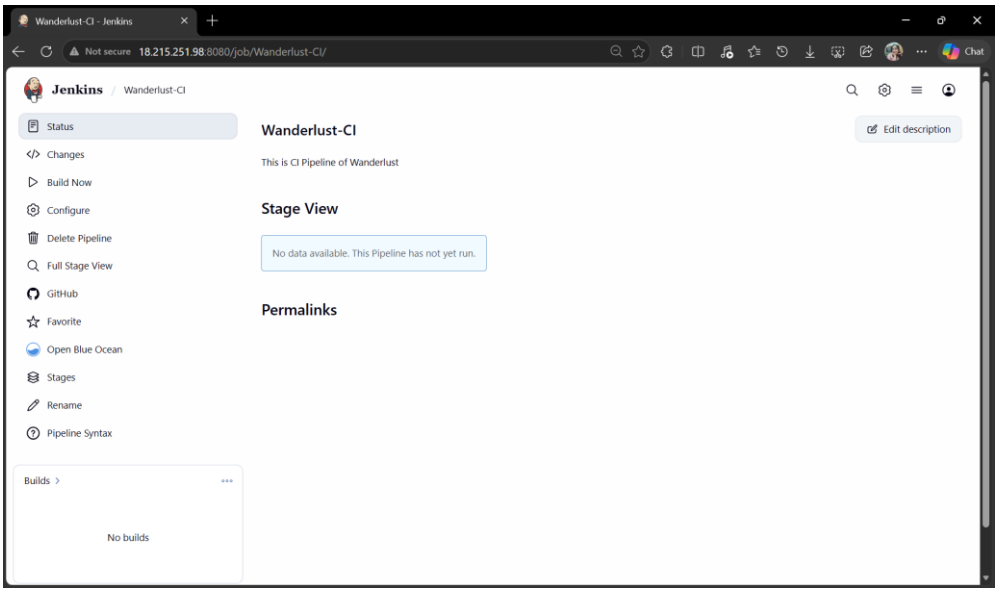
- Wanderlust CI & Wanderlust CD Pipeline Setup

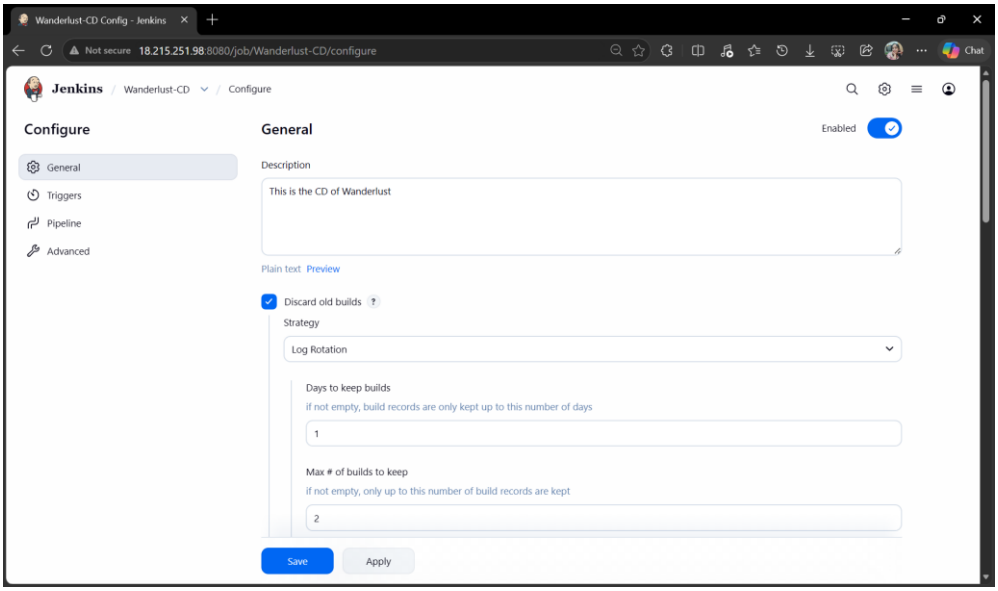


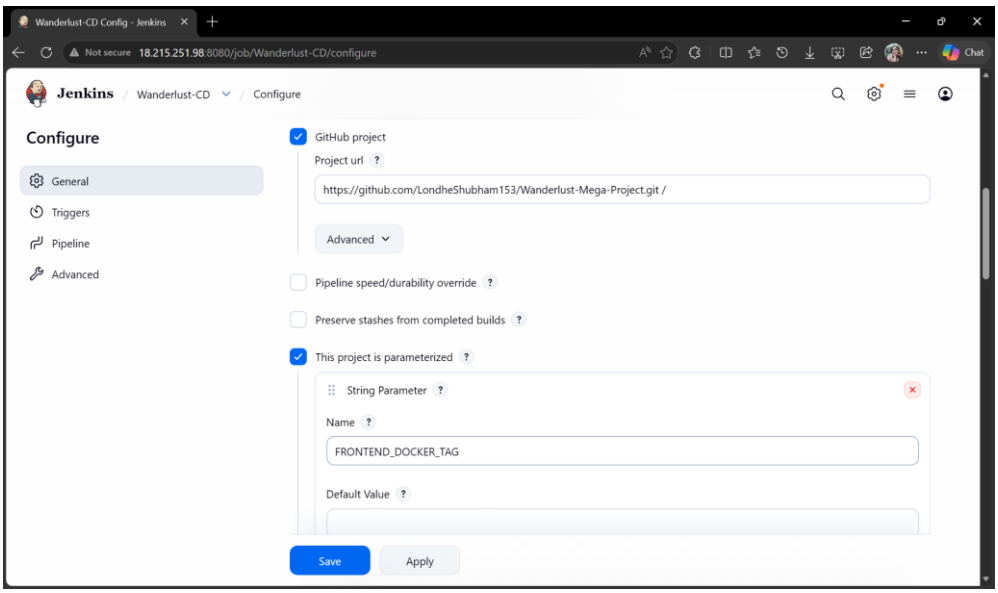


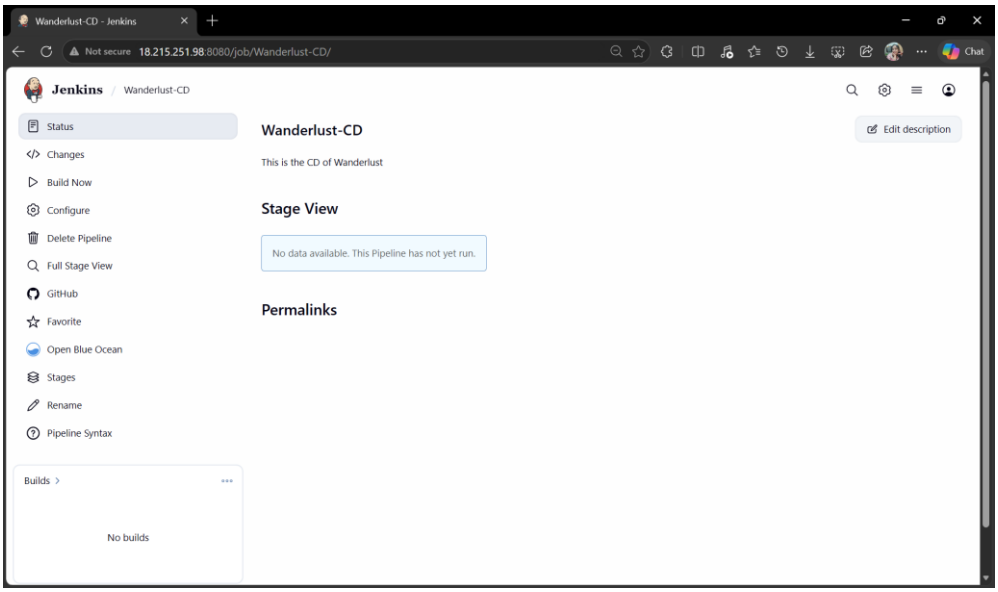

Wanderlust-CI Config - Jenkins
Not secure 18.215.251.98:8080/job/Wanderlust-CI/configure
Jenkins / Wanderlust-CI / Configure
Configure
General
Triggers
Pipeline
Advanced
2
Advanced
Do not allow concurrent builds
Do not allow the pipeline to resume if the controller restarts
GitHub project
Project url
https://github.com/Radhika-08/Wanderlust-Mega-Project.git
Advanced
Pipeline speed/durability override
Preserve stashes from completed builds
This project is parameterized
Save
Apply

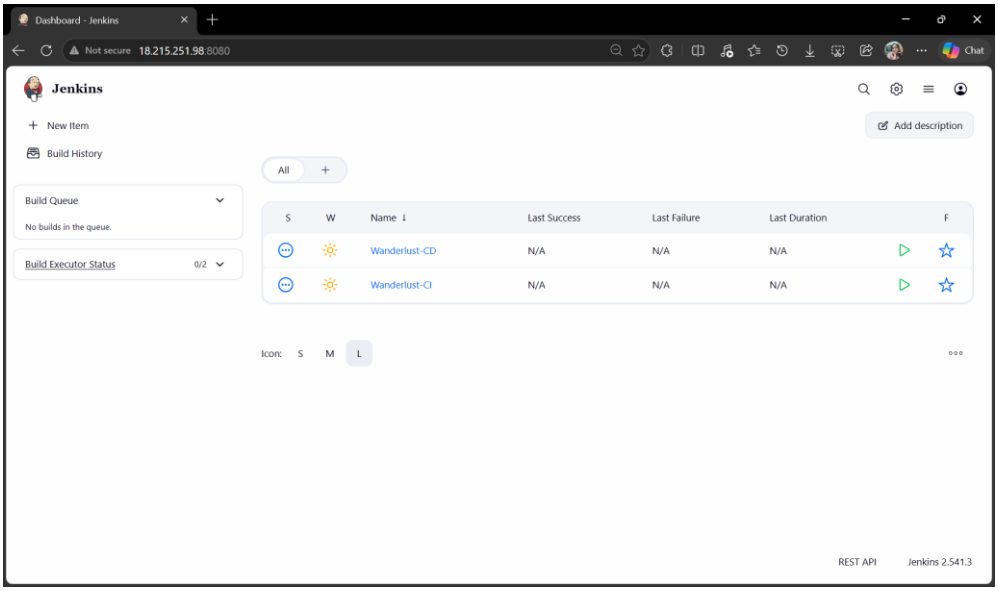

Wanderlust-CI Config - Jenkins
Not secure 18.215.251.98:8080/job/Wanderlust-CI/configure
Jenkins / Wanderlust-CI / Configure
Configure
General
Triggers
Pipeline
Advanced
Pipeline
Define your Pipeline using Groovy directly or pull it from source control.
Definition
Pipeline script
Script
@Library('Shared') _
pipeline {
agent any
environment {
SONAR_HOME = tool "Sonar"
}
Use Groovy Sandbox
Pipeline Syntax
Save
Apply


Wanderlust-CI - Jenkins
Not secure 18.215.251.98:8080/job/Wanderlust-CI/
Jenkins / Wanderlust-CI
Status
Changes
Build Now
Configure
Delete Pipeline
Full Stage View
GitHub
Favorite
Open Blue Ocean
Stages
Rename
Pipeline Syntax
Builds
No builds
Wanderlust-CI
Edit description
This is CI Pipeline of Wanderlust
Stage View
No data available. This Pipeline has not yet run.
Permalinks

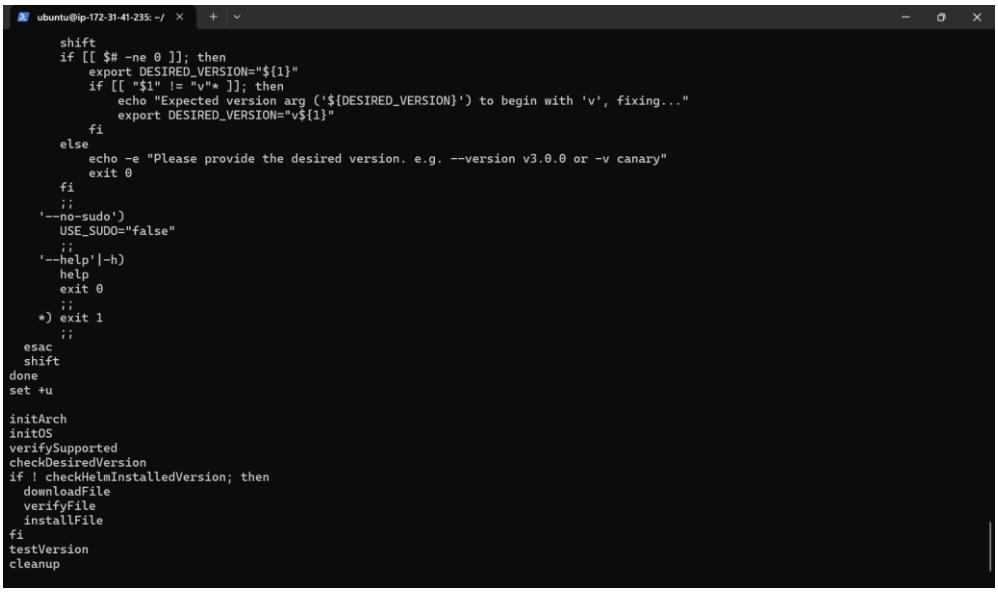

Wanderlust-CD Config - Jenkins
Not secure 18.215.251.98:8080/job/Wanderlust-CD/configure
Jenkins / Wanderlust-CD / Configure
Configure
General
Triggers
Pipeline
Advanced
General
Enabled
Description
This is the CD of Wanderlust
Plain text Preview
Discard old builds
Strategy
Log Rotation
Days to keep builds
if not empty, build records are only kept up to this number of days
1
Max # of builds to keep
if not empty, only up to this number of build records are kept
2
Save
Apply

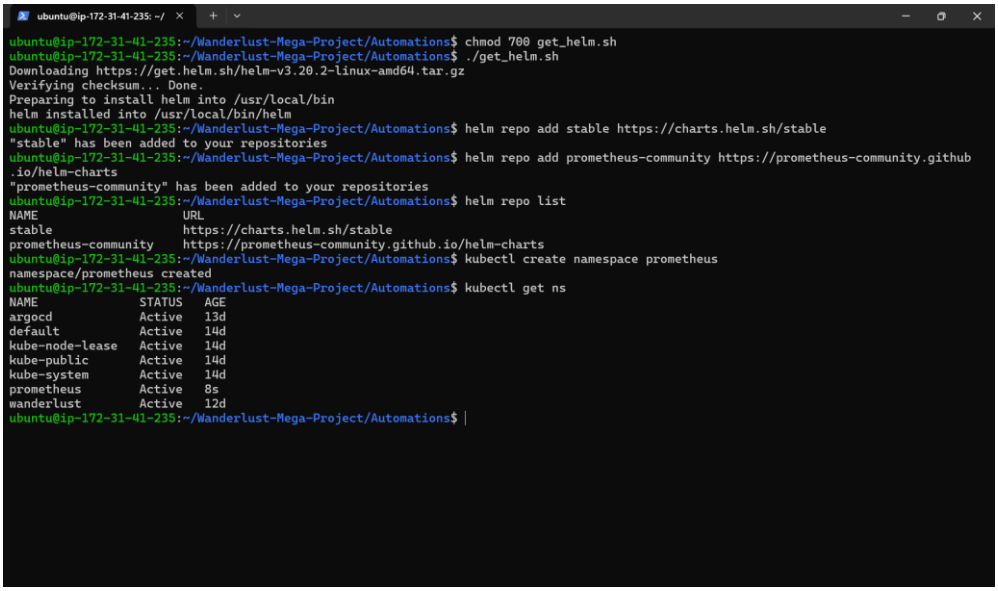

Wanderlust-CD Config - Jenkins
Not secure 18.215.251.98:8080/job/Wanderlust-CD/configure
Jenkins / Wanderlust-CD / Configure
Configure
General
Triggers
Pipeline
Advanced
GitHub project
Project url
https://github.com/LondheShubham153/Wanderlust-Mega-Project.git /
Advanced
Pipeline speed/durability override
Preserve stashes from completed builds
This project is parameterized
String Parameter
Name
FRONTEND_DOCKER_TAG
Default Value
Save
Apply

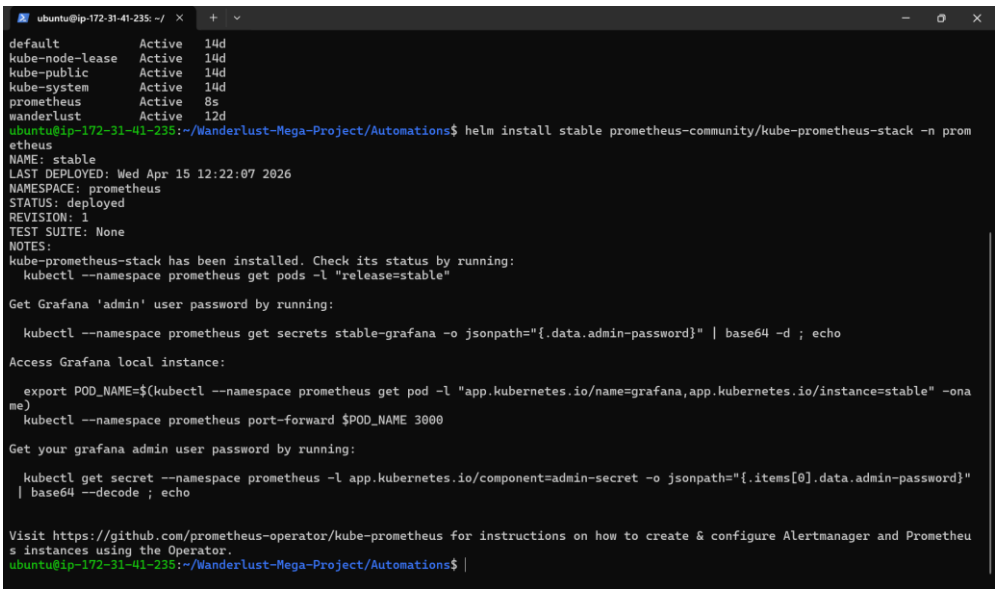

Wanderlust-CD - Jenkins
Not secure 18.215.251.98:8080/job/Wanderlust-CD/
Jenkins / Wanderlust-CD
Status
Changes
Build Now
Configure
Delete Pipeline
Full Stage View
GitHub
Favorite
Open Blue Ocean
Stages
Rename
Pipeline Syntax
Builds
No builds
Wanderlust-CD
Edit description
This is the CD of Wanderlust
Stage View
No data available. This Pipeline has not yet run.
Permalinks

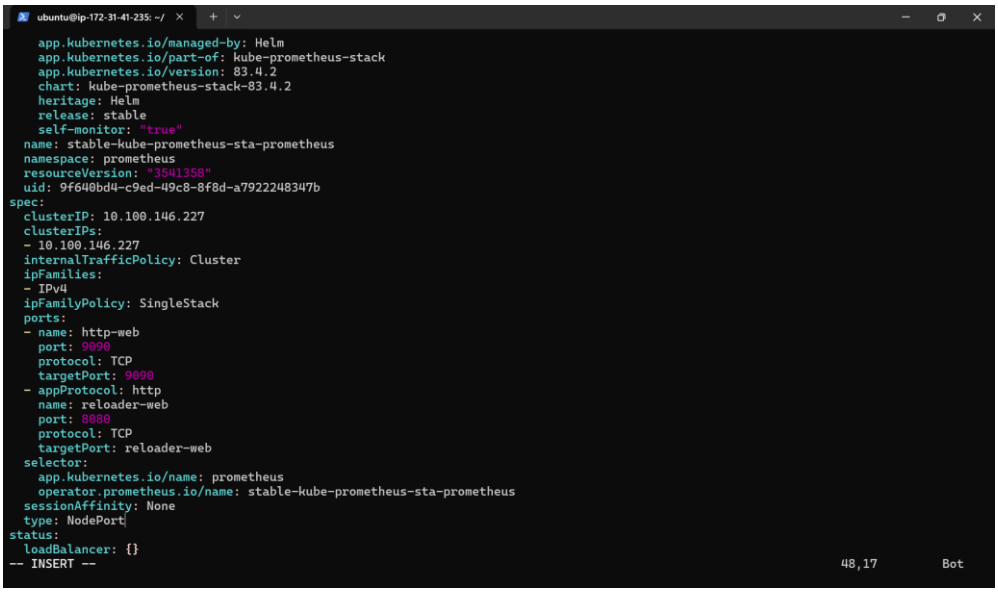


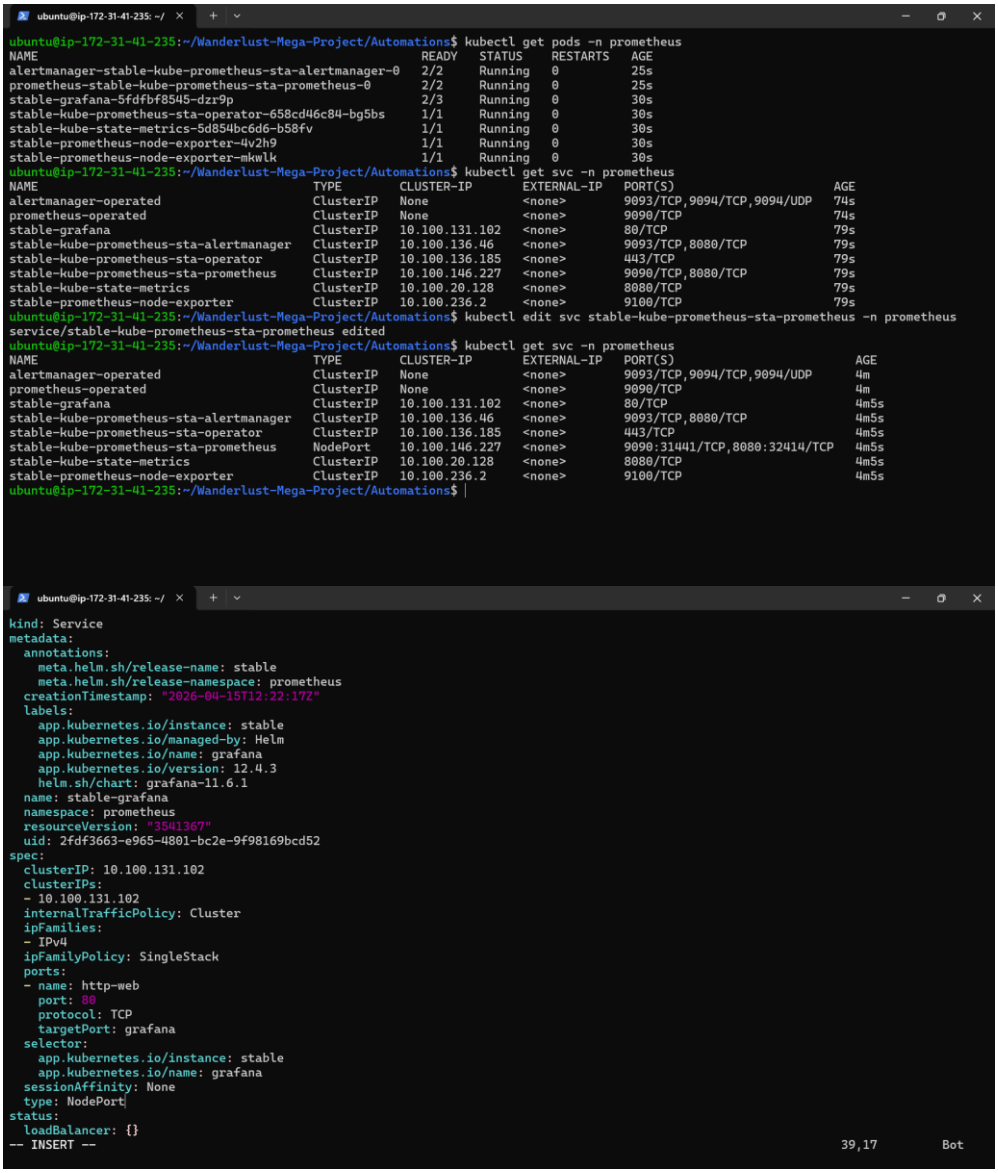
- Prometheus & Grafana Setup Using Helm Chart



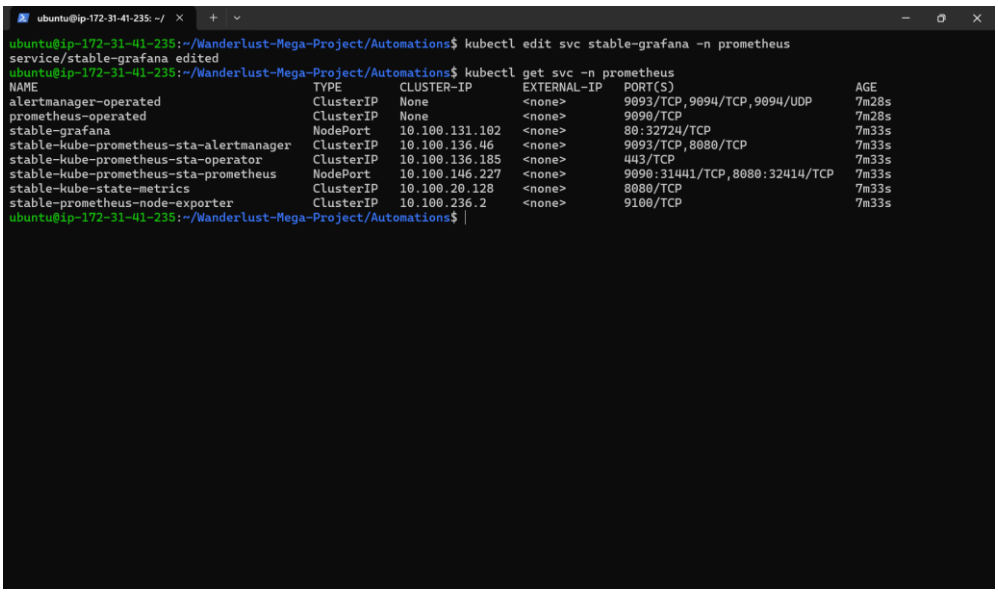

ubuntu@ip-172-31-41-235: ~/
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$ chmod 700 get_helm.sh
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$ ./get_helm.sh
Downloading https://get.helm.sh/helm-v3.20.2-linux-amd64.tar.gz
Verifying checksum... Done.
Preparing to install helm into /usr/local/bin
helm installed into /usr/local/bin/helm
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$ helm repo add stable https://charts.helm.sh/stable
"stable" has been added to your repositories
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$ helm repo add prometheus-community https://prometheus-community.github
.io/helm-charts
"prometheus-community" has been added to your repositories
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$ helm repo list
NAME URL
stable https://charts.helm.sh/stable
prometheus-community https://prometheus-community.github.io/helm-charts
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$ kubectl create namespace prometheus
namespace/prometheus created
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$ kubectl get ns
NAME STATUS AGE
argocd Active 13d
default Active 14d
kube-node-lease Active 14d
kube-public Active 14d
kube-system Active 14d
prometheus Active 8s
wanderlust Active 12d
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$

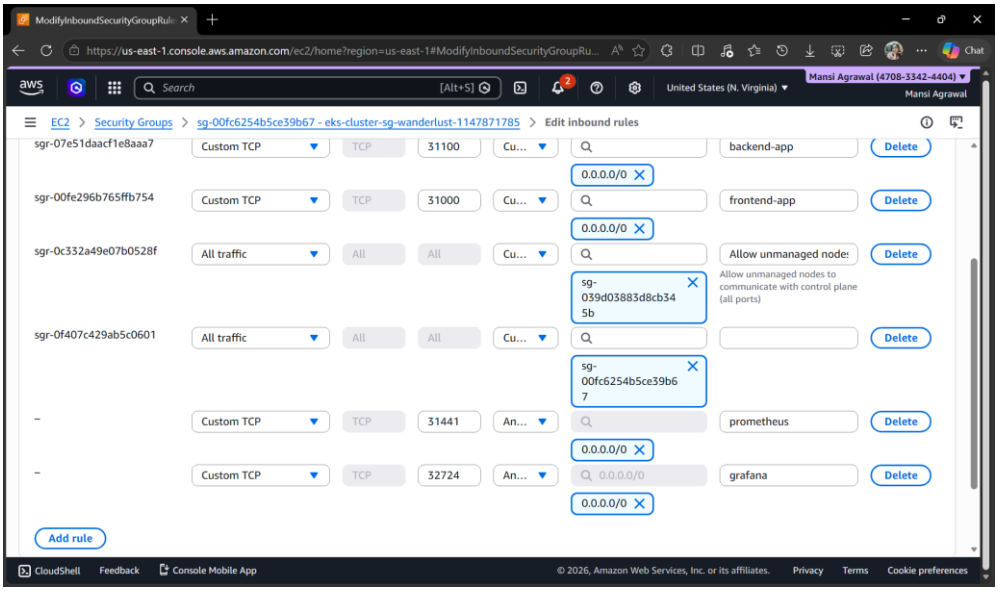

ubuntu@ip-172-31-41-235: ~/
default Active 14d
kube-node-lease Active 14d
kube-public Active 14d
kube-system Active 14d
prometheus Active 8s
wanderlust Active 12d
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$ helm install stable prometheus-community/kube-prometheus-stack -n prom
etheus
NAME: stable
LAST DEPLOYED: Wed Apr 15 12:22:07 2026
NAMESPACE: prometheus
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
kube-prometheus-stack has been installed. Check its status by running:
kubectl --namespace prometheus get pods -l "release=stable"
Get Grafana 'admin' user password by running:
kubectl --namespace prometheus get secrets stable-grafana -o jsonpath="{.data.admin-password}" | base64 -d ; echo
Access Grafana local instance:
export POD_NAME=$(kubectl --namespace prometheus get pod -l "app.kubernetes.io/name=grafana,app.kubernetes.io/instance=stable" -ona
me)
kubectl --namespace prometheus port-forward $POD_NAME 3000
Get your grafana admin user password by running:
kubectl get secret --namespace prometheus -l app.kubernetes.io/component=admin-secret -o jsonpath="{.items[0].data.admin-password}"
| base64 --decode ; echo
Visit https://github.com/prometheus-operator/kube-prometheus for instructions on how to create & configure Alertmanager and Prometheu
s instances using the Operator.
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$

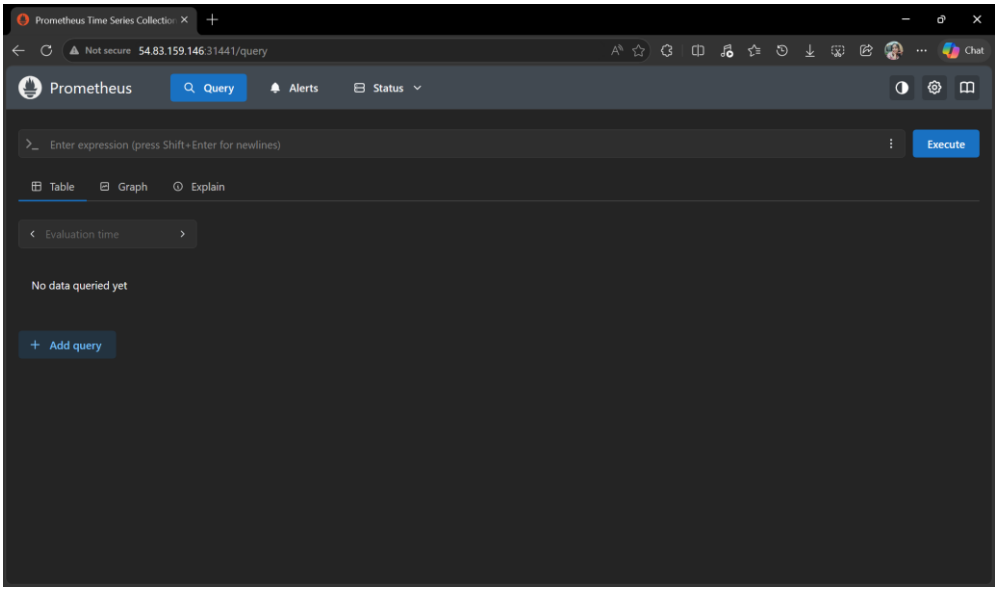

ubuntu@ip-172-31-41-235: ~/
app.kubernetes.io/managed-by: Helm
app.kubernetes.io/part-of: kube-prometheus-stack
app.kubernetes.io/version: 83.4.2
chart: kube-prometheus-stack-83.4.2
heritage: Helm
release: stable
self-monitor: "true"
name: stable-kube-prometheus-sta-prometheus
namespace: prometheus
resourceVersion: "3541358"
uid: 9f640bd4-c9ed-49c8-8f8d-a7922248347b
spec:
clusterIP: 10.100.146.227
clusterIPs:
- 10.100.146.227
internalTrafficPolicy: Cluster
ipFamilies:
- IPv4
ipFamilyPolicy: SingleStack
ports:
- name: http-web
port: 9090
protocol: TCP
targetPort: 9090
- appProtocol: http
name: reloader-web
port: 8080
protocol: TCP
targetPort: reloader-web
selector:
app.kubernetes.io/name: prometheus
operator.prometheus.io/name: stable-kube-prometheus-sta-prometheus
sessionAffinity: None
type: NodePort
status:
loadBalancer: {}
-- INSERT --
48,17
Bot

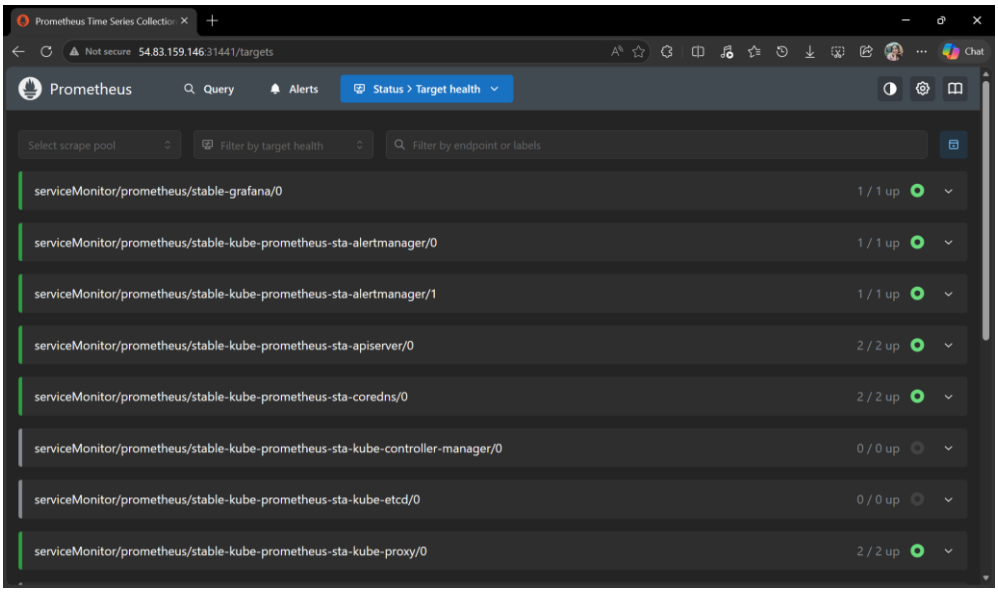

ubuntu@ip-172-31-41-235: ~/Wanderlust-Mega-Project/Automations$ kubectl get pods -n prometheus
NAME READY STATUS RESTARTS AGE
alertmanager-stable-kube-prometheus-sta-alertmanager-0 2/2 Running 0 25s
prometheus-stable-kube-prometheus-sta-prometheus-0 2/2 Running 0 25s
stable-grafana-5fdfbf8545-dzr9p 2/3 Running 0 30s
stable-kube-prometheus-sta-operator-658cd46c84-bg5bs 1/1 Running 0 30s
stable-kube-state-metrics-5d854bc6d6-b58fv 1/1 Running 0 30s
stable-prometheus-node-exporter-4v2h9 1/1 Running 0 30s
stable-prometheus-node-exporter-mkwlk 1/1 Running 0 30s
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$ kubectl get svc -n prometheus
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
alertmanager-operated ClusterIP None <none> 9093/TCP,9094/TCP,9094/UDP 74s
prometheus-operated ClusterIP None <none> 9090/TCP 74s
stable-grafana ClusterIP 10.100.131.102 <none> 80/TCP 79s
stable-kube-prometheus-sta-alertmanager ClusterIP 10.100.136.46 <none> 9093/TCP,8080/TCP 79s
stable-kube-prometheus-sta-operator ClusterIP 10.100.136.185 <none> 443/TCP 79s
stable-kube-prometheus-sta-prometheus ClusterIP 10.100.146.227 <none> 9090/TCP,8080/TCP 79s
stable-kube-state-metrics ClusterIP 10.100.20.128 <none> 8080/TCP 79s
stable-prometheus-node-exporter ClusterIP 10.100.236.2 <none> 9100/TCP 79s
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$ kubectl edit svc stable-kube-prometheus-sta-prometheus -n prometheus
service/stable-kube-prometheus-sta-prometheus edited
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$ kubectl get svc -n prometheus
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
alertmanager-operated ClusterIP None <none> 9093/TCP,9094/TCP,9094/UDP 4m
prometheus-operated ClusterIP None <none> 9090/TCP 4m
stable-grafana ClusterIP 10.100.131.102 <none> 80/TCP 4m5s
stable-kube-prometheus-sta-alertmanager ClusterIP 10.100.136.46 <none> 9093/TCP,8080/TCP 4m5s
stable-kube-prometheus-sta-operator ClusterIP 10.100.136.185 <none> 443/TCP 4m5s
stable-kube-prometheus-sta-prometheus NodePort 10.100.146.227 <none> 9090:31441/TCP,8080:32414/TCP 4m5s
stable-kube-state-metrics ClusterIP 10.100.20.128 <none> 8080/TCP 4m5s
stable-prometheus-node-exporter ClusterIP 10.100.236.2 <none> 9100/TCP 4m5s
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$

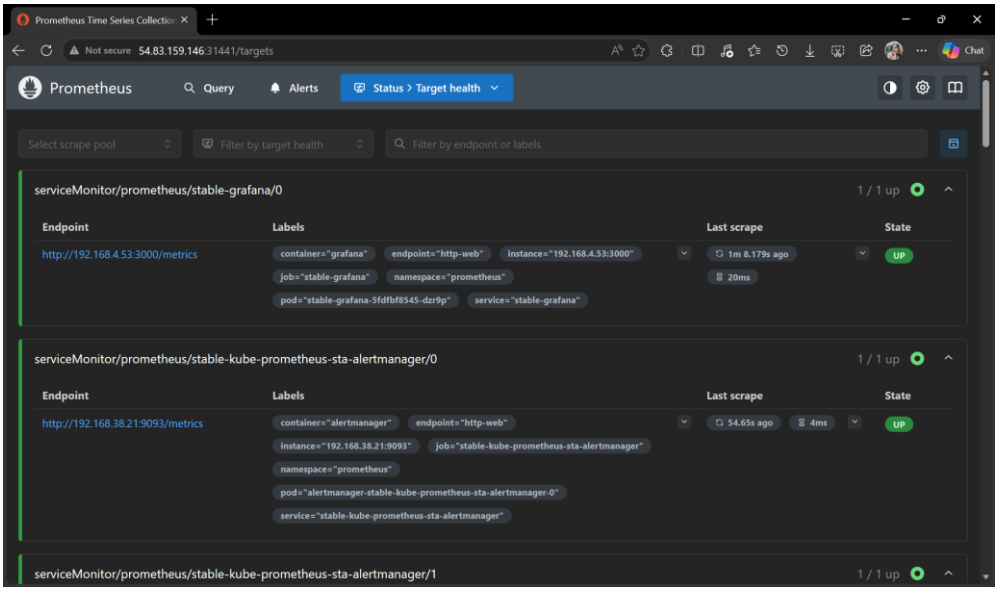
kind: Service
metadata:
annotations:
meta.helm.sh/release-name: stable
meta.helm.sh/release-namespace: prometheus
creationTimestamp: "2026-04-15T12:22:17Z"
labels:
app.kubernetes.io/instance: stable
app.kubernetes.io/managed-by: Helm
app.kubernetes.io/name: grafana
app.kubernetes.io/version: 12.4.3
helm.sh/chart: grafana-11.6.1
name: stable-grafana
namespace: prometheus
resourceVersion: "3541367"
uid: 2fdf3663-e965-4801-bc2e-9f98169bcd52
spec:
clusterIP: 10.100.131.102
clusterIPs:
- 10.100.131.102
internalTrafficPolicy: Cluster
ipFamilies:
- IPv4
ipFamilyPolicy: SingleStack
ports:
- name: http-web
port: 80
protocol: TCP
targetPort: grafana
selector:
app.kubernetes.io/instance: stable
app.kubernetes.io/name: grafana
sessionAffinity: None
type: NodePort
status:
loadBalancer: {}
-- INSERT --
39,17 Bot

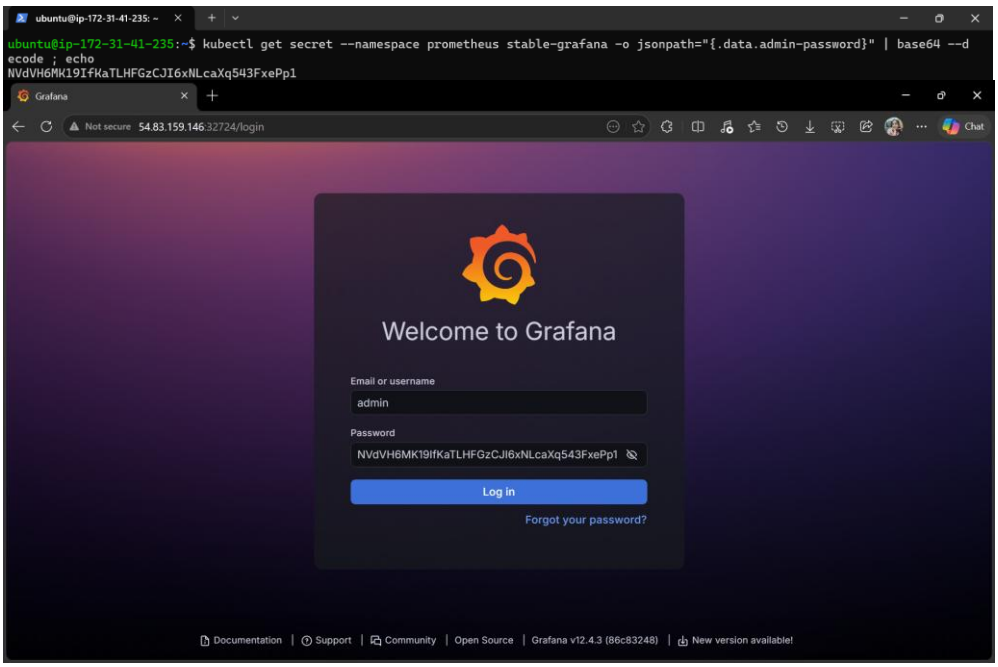

ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$ kubectl edit svc stable-grafana -n prometheus
service/stable-grafana edited
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$ kubectl get svc -n prometheus
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
alertmanager-operated ClusterIP None <none> 9093/TCP,9094/TCP,9094/UDP 7m28s
prometheus-operated ClusterIP None <none> 9090/TCP 7m28s
stable-grafana NodePort 10.100.131.102 <none> 80:32724/TCP 7m33s
stable-kube-prometheus-sta-alertmanager ClusterIP 10.100.136.46 <none> 9093/TCP,8080/TCP 7m33s
stable-kube-prometheus-sta-operator ClusterIP 10.100.136.185 <none> 443/TCP 7m33s
stable-kube-prometheus-sta-prometheus NodePort 10.100.146.227 <none> 9090:31441/TCP,8080:32414/TCP 7m33s
stable-kube-state-metrics ClusterIP 10.100.20.128 <none> 8080/TCP 7m33s
stable-prometheus-node-exporter ClusterIP 10.100.236.2 <none> 9100/TCP 7m33s
ubuntu@ip-172-31-41-235:~/Wanderlust-Mega-Project/Automations$

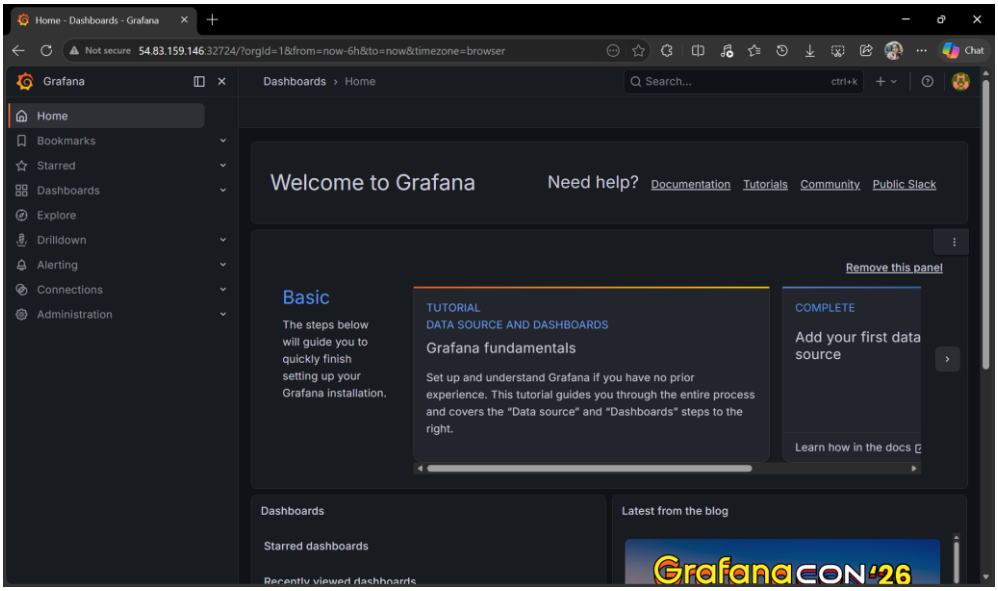

ModifyInboundSecurityGroupRules
https://us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#ModifyInboundSecurityGroupRu...
Search
[Alt+S]
United States (N. Virginia)
Mansi Agrawal (4708-3342-4404)
Mansi Agrawal
EC2 > Security Groups > sg-00fc6254b5ce39b67 - eks-cluster-sg-wanderlust-1147871785 > Edit inbound rules
sgr-07e51daacf1e8aaa7
Custom TCP
TCP
31100
Cu...
0.0.0.0/0
backend-app
Delete
sgr-00fe296b765ffb754
Custom TCP
TCP
31000
Cu...
0.0.0.0/0
frontend-app
Delete
sgr-0c332a49e07b0528f
All traffic
All
All
Cu...
sg-039d03883d8cb345b
Allow unmanaged nodes to communicate with control plane (all ports)
Allow unmanaged node
Delete
sgr-0f407c429ab5c0601
All traffic
All
All
Cu...
sg-00fc6254b5ce39b67
Delete
-
Custom TCP
TCP
31441
An...
0.0.0.0/0
prometheus
Delete
-
Custom TCP
TCP
32724
An...
0.0.0.0/0
grafana
Delete
Add rule
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Prometheus Time Series Collection
Not secure 54.83.159.146:31441/query
Prometheus
Query
Alerts
Status
Enter expression (press Shift+Enter for newlines)
Execute
Table
Graph
Explain
Evaluation time
No data queried yet
Add query

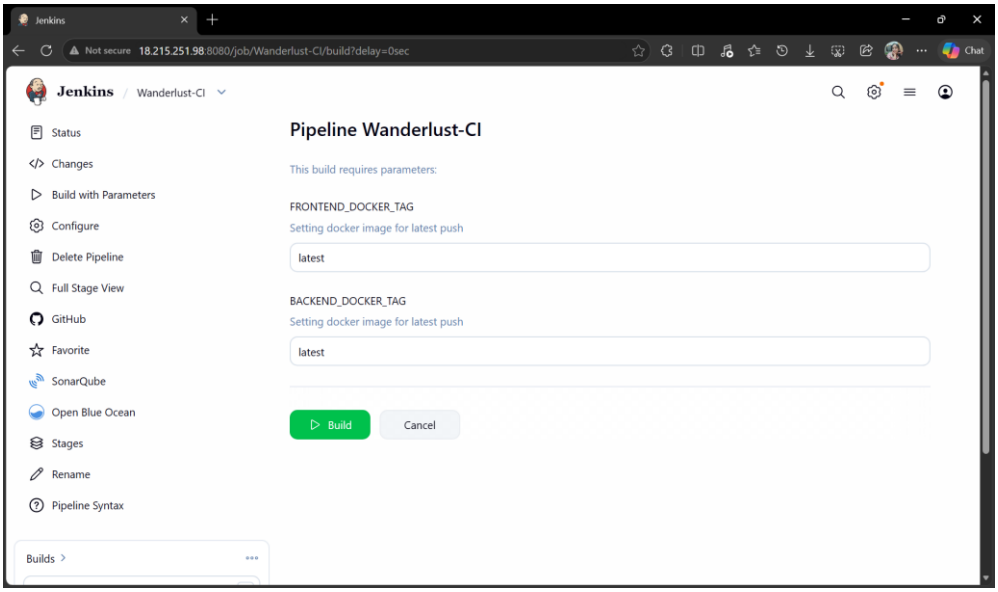

Prometheus Time Series Collection
Not secure 54.83.159.146:31441/targets
Prometheus
Query
Alerts
Status > Target health
Select scrape pool
Filter by target health
Filter by endpoint or labels
serviceMonitor/prometheus/stable-grafana/0
1 / 1 up
serviceMonitor/prometheus/stable-kube-prometheus-sta-alertmanager/0
1 / 1 up
serviceMonitor/prometheus/stable-kube-prometheus-sta-alertmanager/1
1 / 1 up
serviceMonitor/prometheus/stable-kube-prometheus-sta-apiserver/0
2 / 2 up
serviceMonitor/prometheus/stable-kube-prometheus-sta-coredns/0
2 / 2 up
serviceMonitor/prometheus/stable-kube-prometheus-sta-kube-controller-manager/0
0 / 0 up
serviceMonitor/prometheus/stable-kube-prometheus-sta-kube-etcd/0
0 / 0 up
serviceMonitor/prometheus/stable-kube-prometheus-sta-kube-proxy/0
2 / 2 up

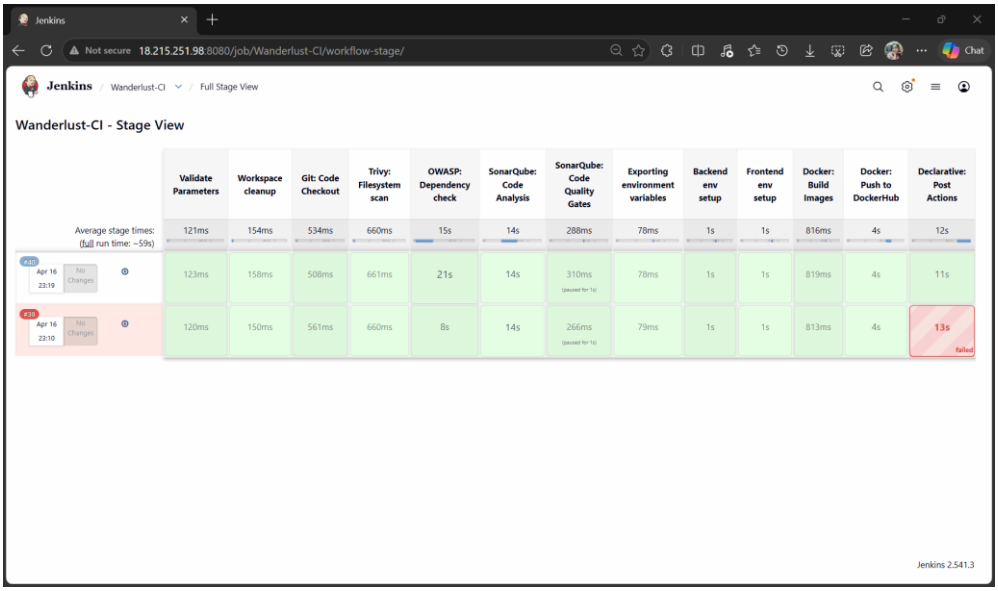

Prometheus Time Series Collection
Not secure 54.83.159.146:31441/targets
Prometheus
Query
Alerts
Status > Target health
Select scrape pool
Filter by target health
Filter by endpoint or labels
serviceMonitor/prometheus/stable-grafana/0
1 / 1 up
Endpoint
Labels
Last scrape
State
http://192.168.4.53:3000/metrics
container="grafana"
endpoint="http-web"
instance="192.168.4.53:3000"
job="stable-grafana"
namespace="prometheus"
pod="stable-grafana-5fdfbf8545-dzr9p"
service="stable-grafana"
1m 8.179s ago
20ms
UP
serviceMonitor/prometheus/stable-kube-prometheus-sta-alertmanager/0
1 / 1 up
Endpoint
Labels
Last scrape
State
http://192.168.38.21:9093/metrics
container="alertmanager"
endpoint="http-web"
instance="192.168.38.21:9093"
job="stable-kube-prometheus-sta-alertmanager"
namespace="prometheus"
pod="alertmanager-stable-kube-prometheus-sta-alertmanager-0"
service="stable-kube-prometheus-sta-alertmanager"
54.65s ago
4ms
UP
serviceMonitor/prometheus/stable-kube-prometheus-sta-alertmanager/1
1 / 1 up

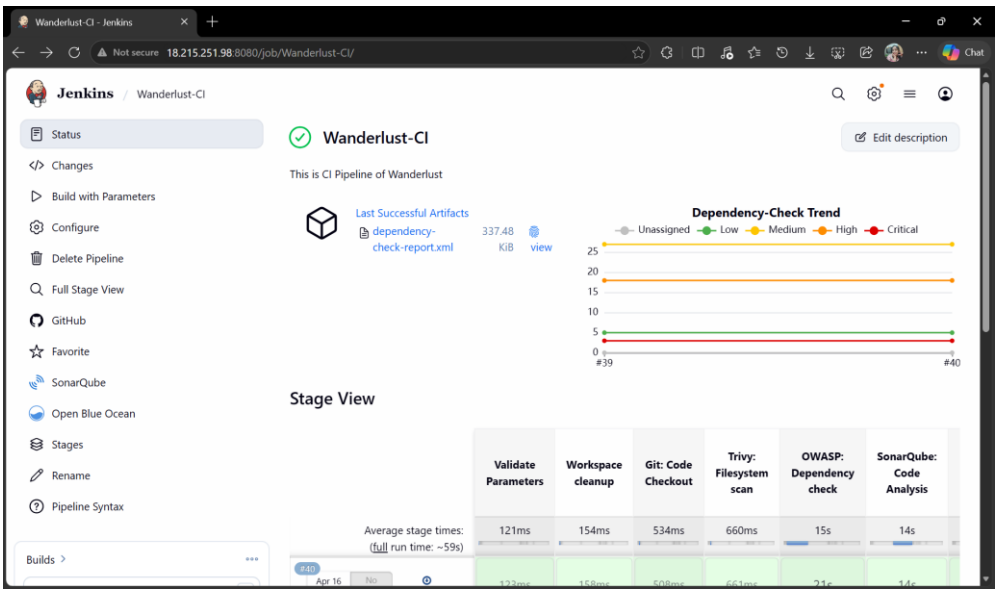

ubuntu@ip-172-31-41-235: ~
ubuntu@ip-172-31-41-235:~$ kubectl get secret --namespace prometheus stable-grafana -o jsonpath="{.data.admin-password}" | base64 --d
ecode ; echo
NVdVH6MK19IfKaTLHFGzCJI6xNLcaXq543FxePp1
Grafana
Not secure 54.83.159.146:32724/login
Welcome to Grafana
Email or username
admin
Password
NVdVH6MK19IfKaTLHFGzCJI6xNLcaXq543FxePp1
Log in
Forgot your password?
Documentation
Support
Community
Open Source
Grafana v12.4.3 (86c83248)
New version available!

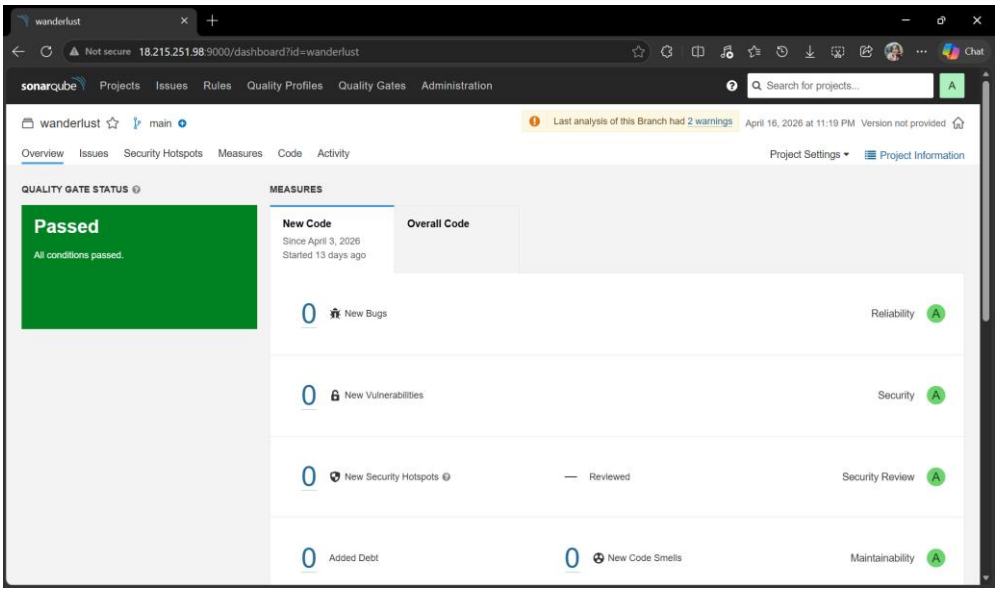

Home - Dashboards - Grafana
Not secure 54.83.159.146:32724/?orgId=1&from=now-6h&to=now&timezone=browser
Grafana
Dashboards › Home
Search...
Home
Bookmarks
Starred
Dashboards
Explore
Drilldown
Alerting
Connections
Administration
Welcome to Grafana
Need help?
Documentation
Tutorials
Community
Public Slack
Remove this panel
Basic
The steps below will guide you to quickly finish setting up your Grafana installation.
TUTORIAL
DATA SOURCE AND DASHBOARDS
Grafana fundamentals
Set up and understand Grafana if you have no prior experience. This tutorial guides you through the entire process and covers the "Data source" and "Dashboards" steps to the right.
COMPLETE
Add your first data source
Learn how in the docs
Dashboards
Starred dashboards
Recently viewed dashboards
Latest from the blog
GrafanaCON '26

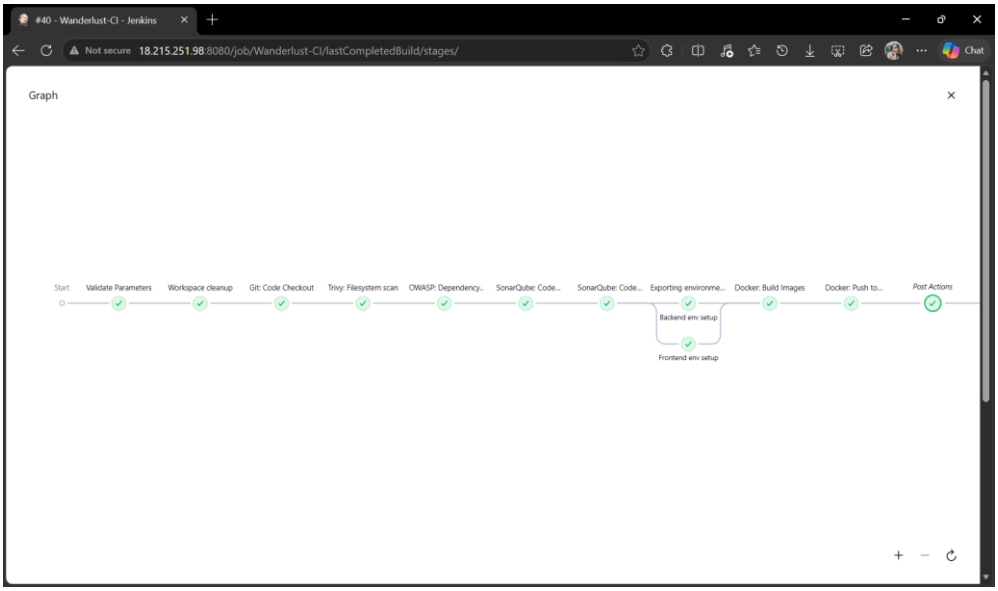
- Result

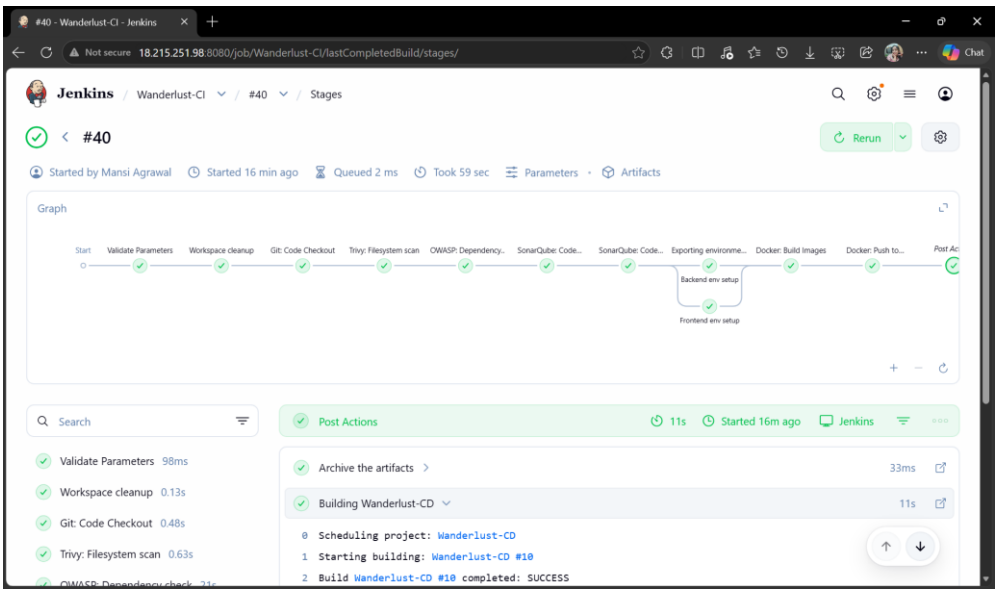

Jenkins
Not secure 18.215.251.98:8080/job/Wanderlust-CI/build?delay=0sec
Jenkins / Wanderlust-CI
Status
Changes
Build with Parameters
Configure
Delete Pipeline
Full Stage View
GitHub
Favorite
SonarQube
Open Blue Ocean
Stages
Rename
Pipeline Syntax
Builds
Pipeline Wanderlust-CI
This build requires parameters:
FRONTEND_DOCKER_TAG
Setting docker image for latest push
latest
BACKEND_DOCKER_TAG
Setting docker image for latest push
latest
Build
Cancel

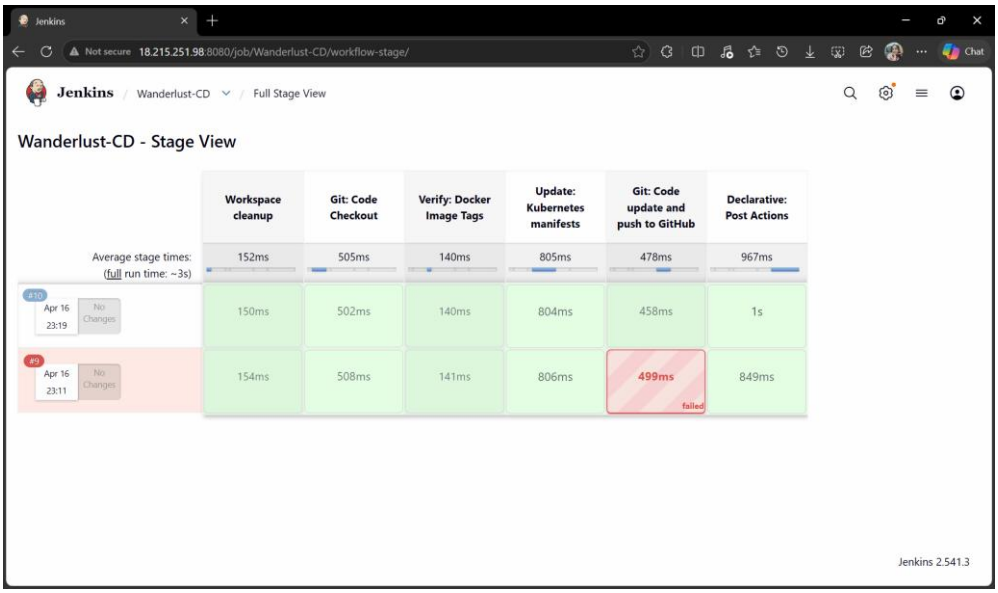

Jenkins
Not secure 18.215.251.98:8080/job/Wanderlust-CI/workflow-stage/
Jenkins / Wanderlust-CI / Full Stage View
Wanderlust-CI - Stage View
Validate Parameters
Workspace cleanup
Git: Code Checkout
Trivy: Filesystem scan
OWASP: Dependency check
SonarQube: Code Analysis
SonarQube: Code Quality Gates
Exporting environment variables
Backend env setup
Frontend env setup
Docker: Build Images
Docker: Push to DockerHub
Declarative: Post Actions
Average stage times:
(full run time: ~59s)
121ms 154ms 534ms 660ms 15s 14s 288ms 78ms 1s 1s 816ms 4s 12s
#40 Apr 16 23:19 No Changes
123ms 158ms 508ms 661ms 21s 14s 310ms 78ms 1s 1s 819ms 4s 11s
#39 Apr 16 23:10 No Changes
120ms 150ms 561ms 660ms 8s 14s 266ms 79ms 1s 1s 813ms 4s 13s
failed
Jenkins 2.541.3

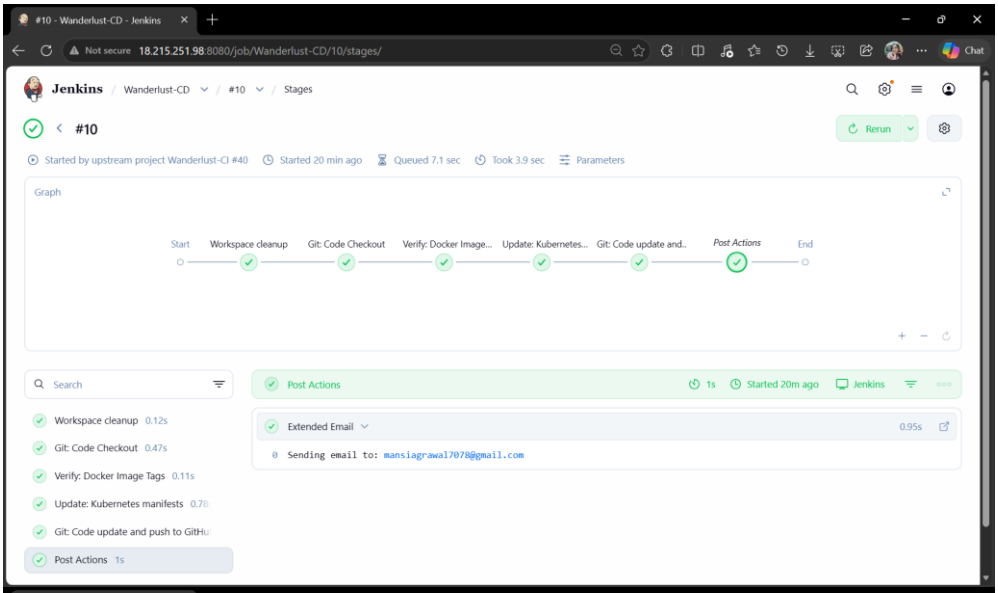

Wanderlust-CI - Jenkins
Not secure 18.215.251.98:8080/job/Wanderlust-CI/
Jenkins / Wanderlust-CI
Status
Changes
Build with Parameters
Configure
Delete Pipeline
Full Stage View
GitHub
Favorite
SonarQube
Open Blue Ocean
Stages
Rename
Pipeline Syntax
Builds
Wanderlust-CI
Edit description
This is CI Pipeline of Wanderlust
Last Successful Artifacts
dependency-check-report.xml 337.48 KiB view
Dependency-Check Trend
Unassigned Low Medium High Critical
25 20 15 10 5 0
#39 #40
Stage View
Validate Parameters
Workspace cleanup
Git: Code Checkout
Trivy: Filesystem scan
OWASP: Dependency check
SonarQube: Code Analysis
Average stage times:
(full run time: ~59s)
121ms 154ms 534ms 660ms 15s 14s
#40 Apr 16 No

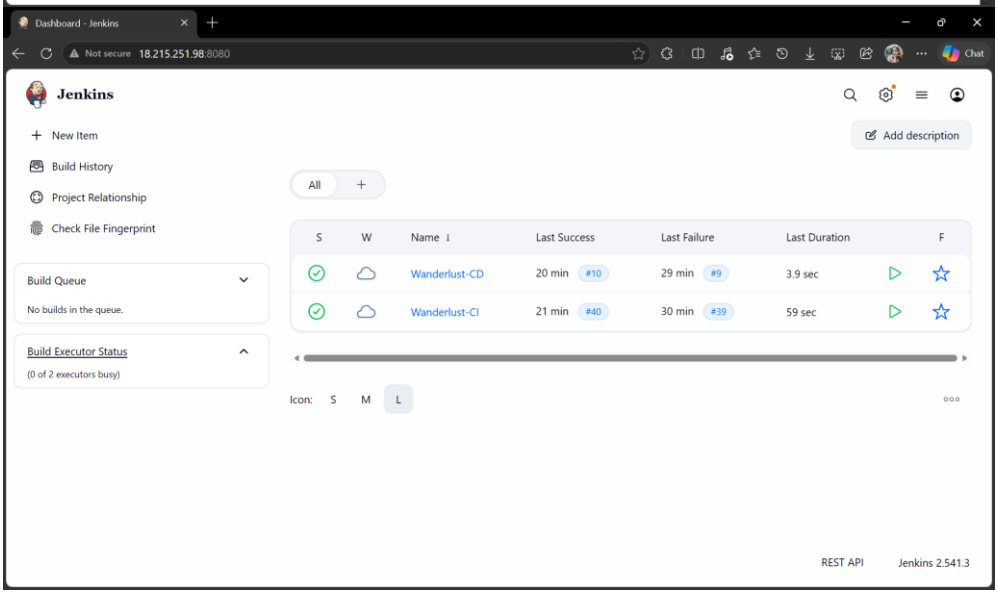

wanderlust
Not secure 18.215.251.98:9000/dashboard?id=wanderlust
sonarqube Projects Issues Rules Quality Profiles Quality Gates Administration
Search for projects...
wanderlust main
Last analysis of this Branch had 2 warnings
April 16, 2026 at 11:19 PM Version not provided
Overview Issues Security Hotspots Measures Code Activity
Project Settings Project Information
QUALITY GATE STATUS
Passed
All conditions passed.
MEASURES
New Code
Since April 3, 2026
Started 13 days ago
Overall Code
0 New Bugs Reliability A
0 New Vulnerabilities Security A
0 New Security Hotspots Reviewed Security Review A
0 Added Debt 0 New Code Smells Maintainability A

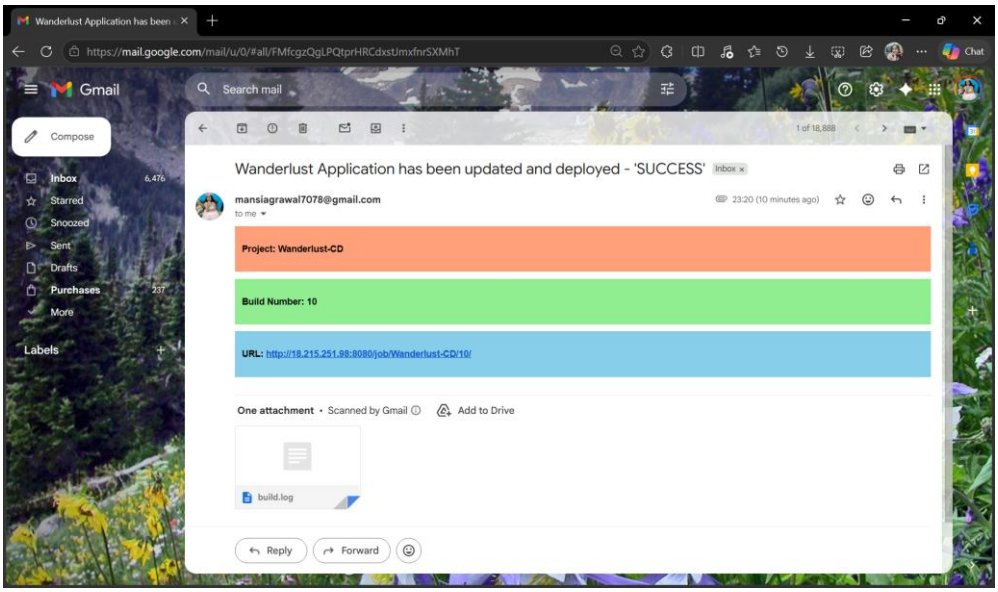

#40 - Wanderlust-CI - Jenkins
Not secure 18.215.251.98:8080/job/Wanderlust-CI/lastCompletedBuild/stages/
Graph
Start
Validate Parameters
Workspace cleanup
Git: Code Checkout
Trivy: Filesystem scan
OWASP: Dependency..
SonarQube: Code...
SonarQube: Code...
Exporting environme...
Backend env setup
Frontend env setup
Docker: Build Images
Docker: Push to...
Post Actions

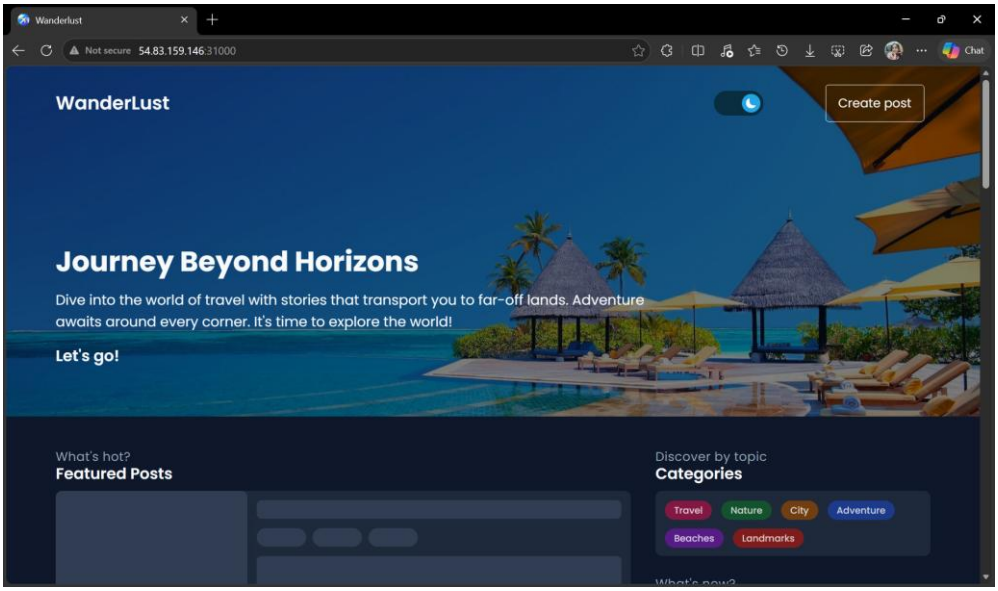

#40 - Wanderlust-CI - Jenkins
Not secure 18.215.251.98:8080/job/Wanderlust-CI/lastCompletedBuild/stages/
Jenkins / Wanderlust-CI / #40 / Stages
#40
Rerun
Started by Mansi Agrawal
Started 16 min ago
Queued 2 ms
Took 59 sec
Parameters
Artifacts
Graph
Search
Post Actions
11s
Started 16m ago
Jenkins
Validate Parameters 98ms
Workspace cleanup 0.13s
Git: Code Checkout 0.48s
Trivy: Filesystem scan 0.63s
Archive the artifacts
33ms
Building Wanderlust-CD
11s
0 Scheduling project: Wanderlust-CD
1 Starting building: Wanderlust-CD #10
2 Build Wanderlust-CD #10 completed: SUCCESS

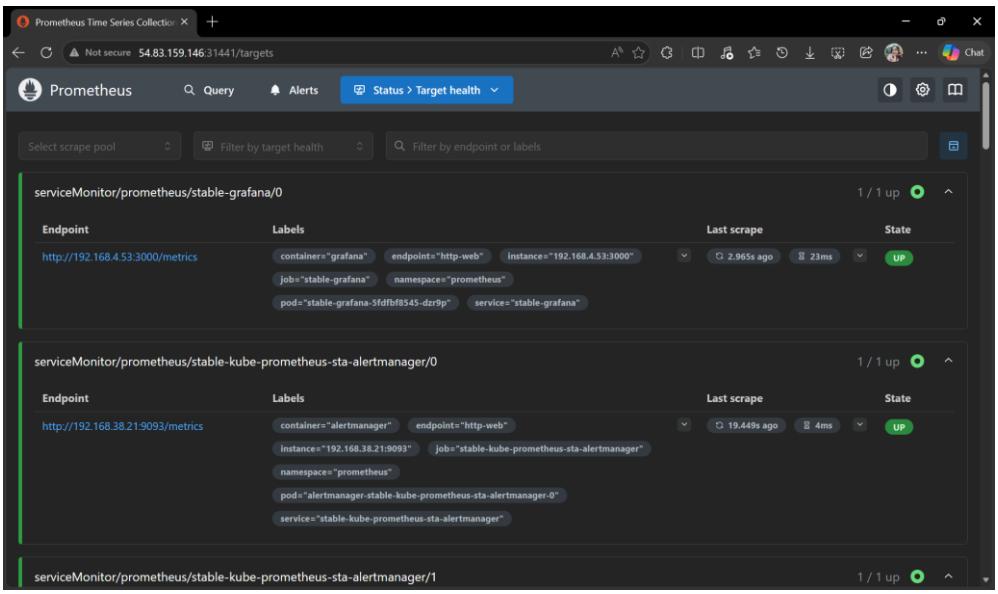

Jenkins
Not secure 18.215.251.98:8080/job/Wanderlust-CD/workflow-stage/
Jenkins / Wanderlust-CD / Full Stage View
Wanderlust-CD - Stage View
Workspace cleanup
Git: Code Checkout
Verify: Docker Image Tags
Update: Kubernetes manifests
Git: Code update and push to GitHub
Declarative: Post Actions
Average stage times:
(full run time: ~3s)
152ms
505ms
140ms
805ms
478ms
967ms
#10
Apr 16
23:19
No Changes
150ms
502ms
140ms
804ms
458ms
1s
#9
Apr 16
23:11
No Changes
154ms
508ms
141ms
806ms
499ms
failed
849ms
Jenkins 2.541.3

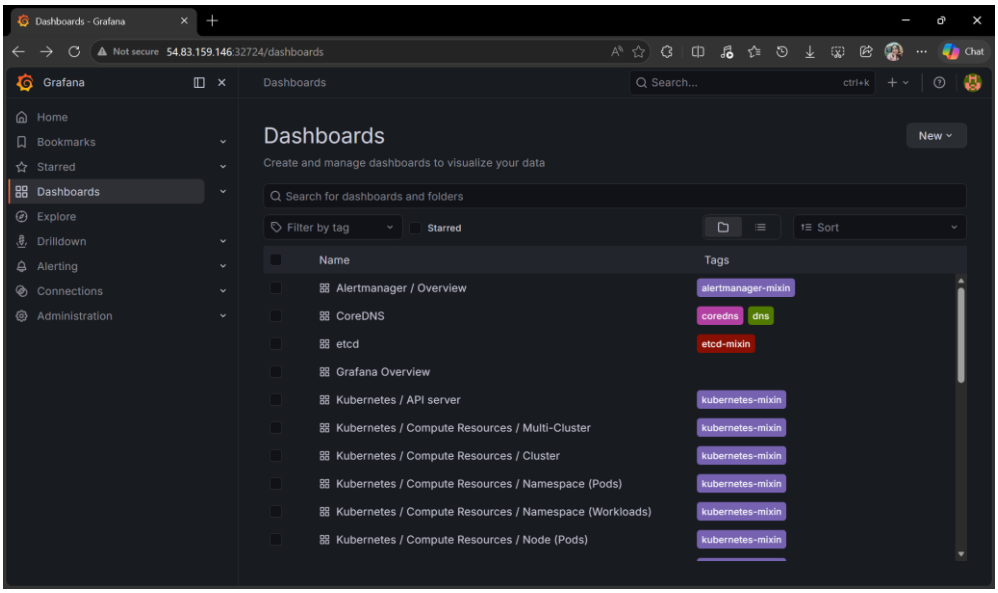

#10 - Wanderlust-CD - Jenkins
Not secure 18.215.251.98:8080/job/Wanderlust-CD/10/stages/
Jenkins / Wanderlust-CD / #10 / Stages
#10
Rerun
Started by upstream project Wanderlust-CI #40
Started 20 min ago
Queued 7.1 sec
Took 3.9 sec
Parameters
Graph
Start
Workspace cleanup
Git: Code Checkout
Verify: Docker Image...
Update: Kubernetes...
Git: Code update and..
Post Actions
End
Search
Post Actions
1s
Started 20m ago
Jenkins
Workspace cleanup 0.12s
Git: Code Checkout 0.47s
Verify: Docker Image Tags 0.11s
Update: Kubernetes manifests
Git: Code update and push to GitHu
Post Actions 1s
Extended Email
0.95s
Sending email to: mansiagrawal7078@gmail.com


Dashboard - Jenkins
Not secure 18.215.251.98:8080
Jenkins
New Item
Build History
Project Relationship
Check File Fingerprint
Add description
All
Build Queue
No builds in the queue.
Build Executor Status
(0 of 2 executors busy)
S
W
Name
Last Success
Last Failure
Last Duration
F
Wanderlust-CD
20 min #10
29 min #9
3.9 sec
Wanderlust-CI
21 min #40
30 min #39
59 sec
Icon: S M L
REST API
Jenkins 2.541.3


Wanderlust Application has been
https://mail.google.com/mail/u/0/#all/FMfcgzQgLPQtprHRCdxstJmxfnrSXMhT
Gmail
Search mail
Compose
Inbox 6,476
Starred
Snoozed
Sent
Drafts
Purchases 237
More
Labels
Wanderlust Application has been updated and deployed - 'SUCCESS'
Inbox
mansiagrawal7078@gmail.com
to me
23:20 (10 minutes ago)
Project: Wanderlust-CD
Build Number: 10
URL: http://18.215.251.98:8080/job/Wanderlust-CD/10/
One attachment • Scanned by Gmail
Add to Drive
build.log
Reply
Forward


Wanderlust
Not secure 54.83.159.146:31000
Chat
WanderLust
Create post
Journey Beyond Horizons
Dive into the world of travel with stories that transport you to far-off lands. Adventure awaits around every corer. It's time to explore the world!
Let's go!
What's hot?
Featured Posts
Discover by topic
Categories
Travel
Nature
City
Adventure
Beaches
Landmarks


Prometheus Time Series Collection
Not secure 54.83.159.146:31441/targets
Chat
Prometheus
Query
Alerts
Status > Target health
Select scrape pool
Filter by target health
Filter by endpoint or labels
serviceMonitor/prometheus/stable-grafana/0
1 / 1 up
Endpoint
Labels
Last scrape
State
http://192.168.4.53:3000/metrics
container="grafana"
endpoint="http-web"
instance="192.168.4.53:3000"
2.965s ago
23ms
UP
job="stable-grafana"
namespace="prometheus"
pod="stable-grafana-5fdfbf8545-dzr9p"
service="stable-grafana"
serviceMonitor/prometheus/stable-kube-prometheus-sta-alertmanager/0
1 / 1 up
Endpoint
Labels
Last scrape
State
http://192.168.38.21:9093/metrics
container="alertmanager"
endpoint="http-web"
19.449s ago
4ms
UP
instance="192.168.38.21:9093"
job="stable-kube-prometheus-sta-alertmanager"
namespace="prometheus"
pod="alertmanager-stable-kube-prometheus-sta-alertmanager-0"
service="stable-kube-prometheus-sta-alertmanager"
serviceMonitor/prometheus/stable-kube-prometheus-sta-alertmanager/1
1 / 1 up


Dashboards - Grafana
Not secure 54.83.159.146:32724/dashboards
Chat
Grafana
Dashboards
Search...
ctrl+k
Home
Bookmarks
Starred
Dashboards
Explore
Drilldown
Alerting
Connections
Administration
Dashboards
New
Create and manage dashboards to visualize your data
Search for dashboards and folders
Filter by tag
Starred
Sort
Name
Tags
Alertmanager / Overview
alertmanager-mixin
CoreDNS
coredns
dns
etcd
etcd-mixin
Grafana Overview
Kubernetes / API server
kubernetes-mixin
Kubernetes / Compute Resources / Multi-Cluster
kubernetes-mixin
Kubernetes / Compute Resources / Cluster
kubernetes-mixin
Kubernetes / Compute Resources / Namespace (Pods)
kubernetes-mixin
Kubernetes / Compute Resources / Namespace (Workloads)
kubernetes-mixin
Kubernetes / Compute Resources / Node (Pods)
kubernetes-mixin

CHAPTER 5: SOFTWARE TESTING

Testing is an important part of this project because it ensures that the application works correctly before it is deployed. In this system, testing is not done manually again and again. Instead, it is automated and becomes a part of the CI/CD pipeline.

Whenever the developer pushes code, the system automatically checks whether the code is working properly or not. This helps in finding errors at an early stage and avoids problems after deployment.

5.1 Testing Process

The testing process is fully automated and follows a proper flow:

1. Developer pushes code to Git
2. Jenkins pipeline gets triggered
3. Code is built
4. Automated tests are executed
5. Code quality and security checks are performed
6. If all tests pass, the process moves to deployment

If any error is found in any stage, the pipeline stops and shows a failure message. This helps the developer fix the issue before moving forward.

5.2 Test Cases

Some basic test cases used in this project are:

- Check if the pipeline triggers correctly after code push
- Verify that the build process completes successfully
- Check if test cases pass without errors
- Validate Docker image creation
- Verify successful deployment on Kubernetes
- Check system performance using monitoring tools

These test cases help ensure that each part of the system is working properly.

After performing all the testing steps, the system works as expected. The pipeline runs automatically, builds the application, performs testing, and deploys it successfully.

Errors are detected early in the process, which improves the overall quality of the application. The automated testing also saves time and reduces manual effort.

CHAPTER 6: CONCLUSION

This project presents a complete DevOps pipeline where different stages of software development are connected into a single automated workflow. The system starts from application development and continues up to deployment and monitoring, covering all major steps involved in modern software delivery.

In the beginning, the application code is managed using a Git repository. This acts as the central source from where the entire process starts. Once the code is updated, the CI process begins where Jenkins plays an important role in building and testing the application. Along with this, additional checks such as code quality and security scanning are performed to ensure that the application is reliable before moving forward.

After successful validation, the application is containerized using Docker, which helps in maintaining consistency across different environments. These containers are then deployed on a Kubernetes cluster, where multiple nodes are used to manage the application efficiently. Kubernetes ensures that the application runs smoothly and can handle scaling when required.

For deployment, a GitOps-based approach is followed using ArgoCD. Instead of manually deploying the application, the system automatically syncs changes from the repository and updates the Kubernetes cluster. This improves deployment consistency and provides better control over the system.

Finally, monitoring tools are used to track the performance and health of the system. This helps in identifying issues and maintaining system stability over time. Monitoring is an important part because it provides continuous feedback after deployment.

The overall flow of the project clearly shows how different tools work together in a DevOps pipeline. From code integration to deployment and monitoring, each stage is automated and connected. This reduces manual effort, improves efficiency, and ensures faster and more reliable software delivery.

In conclusion, the project successfully demonstrates how a complete DevOps system can be designed using tools like Jenkins, Docker, Kubernetes, and ArgoCD. It provides a clear understanding of how modern applications are developed, deployed, and maintained in real-world environments through automation and continuous integration and delivery.

CHAPTER 7: SUMMARY

This project focuses on building a complete DevOps pipeline by integrating different tools and technologies into a single workflow. The main aim was to automate the process of software development, starting from writing code to deploying and monitoring the application.

The project begins with application development, where the code is created and stored in a Git repository. This acts as the starting point of the pipeline. Once the code is pushed, Jenkins is used to trigger the CI/CD process. It automatically builds the application and performs necessary checks such as testing, security scanning, and code quality analysis.

After successful completion of these stages, the application is packaged using Docker. This ensures that the application runs consistently in any environment. The Docker containers are then deployed on a Kubernetes cluster, where multiple nodes are used to manage the application efficiently and handle scaling.

For deployment, ArgoCD is used, which follows the GitOps approach. It continuously monitors the repository and automatically updates the application in the Kubernetes cluster whenever changes are made. This makes the deployment process fully automated and easy to manage.

Monitoring tools are also included in the system to track the performance and health of the application. They help in identifying issues and maintaining stability after deployment.

Overall, the project successfully demonstrates how different DevOps tools can be integrated to create an automated pipeline. It highlights the importance of automation, continuous integration, and continuous delivery in modern software development. The project also provides practical knowledge of how real-world systems are built and managed using DevOps practices.

APPENDICES

References

1. Jenkins Documentation – https://www.jenkins.io/doc/
2. Docker Documentation – https://docs.docker.com/
3. Kubernetes Documentation – https://kubernetes.io/docs/
4. ArgoCD Documentation – https://argo-cd.readthedocs.io/
5. Git Documentation – https://git-scm.com/docs
6. SonarQube Documentation – https://docs.sonarqube.org/
7. OWASP Official Website – https://owasp.org/
8. Prometheus Documentation – https://prometheus.io/docs/
9. Grafana Documentation – https://grafana.com/docs/